



Εθνικό και Καποδιστριακό Πανεπιστήμιο Αθηνών

Τμήμα Πληροφορικής και Τηλεπικοινωνιών

Πειραματική Μελέτη:

Αναζήτηση διανυσμάτων με

Νευρωνικό LSH (Neural LSH) σε Python

2η Προγραμματιστική Εργασία στο Μάθημα:

«Ανάπτυξη Λογισμικού για Αλγοριθμικά Προβλήματα»

Ζαχαρίας Βασίλειος

1115201900275

sdi1900275@di.uoa.gr

Καραγιάννης Δημήτριος

1115202200293

sdi2200293@di.uoa.gr

Περιεχόμενα

Πίνακας Εικόνων	2
Εισαγωγή.....	5
Βελτιστοποίηση υπερπαραμέτρων	5
Συγκριτική αξιολόγηση	6
Παραγωγή γράφου.....	7
1° Πείραμα	7
2° Πείραμα	16
3° Πείραμα	25
4° Πείραμα	35
5° Πείραμα	46
6° Πείραμα	55
7° Πείραμα	64
8° Πείραμα	73
9° Πείραμα	82
10° Πείραμα	91
11° Πείραμα	100
Αποτελέσματα με την E1 εργασία	109
Σύγκριση με την E1 εργασία	111
Σύγκριση με κλασικό LSH	112

Σύγκριση με IVFFlat.....	112
Σύγκριση με IVFPQ.....	113
Σύγκριση με Hypercube.....	114
Συνολική κατάταξη με βάση την ποιότητα.....	114
Συμπέρασμα	115

Πίνακας Εικόνων

Εικόνα 1: Αποτελέσματα 1 ^{ου} Πειράματος	9
Εικόνα 2: Διάγραμμα AF 1 ^{ου} Πειράματος	11
Εικόνα 3: Διάγραμμα Recall 1 ^{ου} Πειράματος	12
Εικόνα 4: Διάγραμμα QPS 1 ^{ου} Πειράματος.....	13
Εικόνα 5: Διάγραμμα tApproximate 1 ^{ου} Πειράματος.....	14
Εικόνα 6: Διάγραμμα tTrue 1 ^{ου} Πειράματος.....	15
Εικόνα 7: Διάγραμμα Trade-off 1 ^{ου} Πειράματος.....	16
Εικόνα 8: Αποτελέσματα 2 ^{ου} Πειράματος	17
Εικόνα 9: Διάγραμμα AF 2 ^{ου} Πειράματος	20
Εικόνα 10: Διάγραμμα Recall 2 ^{ου} Πειράματος.....	21
Εικόνα 11: Διάγραμμα QPS 2 ^{ου} Πειράματος.....	22
Εικόνα 12: Διάγραμμα tApproximate 2 ^{ου} Πειράματος.....	23
Εικόνα 13: Διάγραμμα tTrue 2 ^{ου} Πειράματος.....	24
Εικόνα 14: Διάγραμμα Trade-off 2 ^{ου} Πειράματος.....	25
Εικόνα 15: Αποτελέσματα 3 ^{ου} Πειράματος	26
Εικόνα 16: Διάγραμμα AF 3 ^{ου} Πειράματος	30
Εικόνα 17: Διάγραμμα Recall 3 ^{ου} Πειράματος.....	31
Εικόνα 18: Διάγραμμα QPS 3 ^{ου} Πειράματος.....	32

Εικόνα 19: Διάγραμμα tApproximate 3 ^ο Πειράματος.....	33
Εικόνα 20: Διάγραμμα tTrue 3 ^ο Πειράματος.....	34
Εικόνα 21: Διάγραμμα Trade-off 3 ^ο Πειράματος.....	35
Εικόνα 22: Αποτελέσματα 4 ^ο Πειράματος	36
Εικόνα 23: Διάγραμμα AF 4 ^ο Πειράματος	41
Εικόνα 24: Διάγραμμα Recall 4 ^ο Πειράματος.....	42
Εικόνα 25: Διάγραμμα QPS 4 ^ο Πειράματος.....	43
Εικόνα 26: Διάγραμμα tApproximate 4 ^ο Πειράματος.....	44
Εικόνα 27: Διάγραμμα tTrue 4 ^ο Πειράματος.....	45
Εικόνα 28: Διάγραμμα Trade-off 4 ^ο Πειράματος.....	46
Εικόνα 29: Αποτελέσματα 5 ^ο Πειράματος	47
Εικόνα 30: Διάγραμμα AF 5 ^ο Πειράματος	50
Εικόνα 31: Διάγραμμα Recall 5 ^ο Πειράματος.....	51
Εικόνα 32: Διάγραμμα QPS 5 ^ο Πειράματος.....	52
Εικόνα 33: Διάγραμμα tApproximate 5 ^ο Πειράματος.....	53
Εικόνα 34: Διάγραμμα tTrue 5 ^ο Πειράματος.....	54
Εικόνα 35: Διάγραμμα Trade-off 5 ^ο Πειράματος.....	55
Εικόνα 36: Αποτελέσματα 6 ^ο Πειράματος	56
Εικόνα 37: Διάγραμμα AF 6 ^ο Πειράματος	59
Εικόνα 38: Διάγραμμα Recall 6 ^ο Πειράματος.....	60
Εικόνα 39: Διάγραμμα QPS 6 ^ο Πειράματος.....	61
Εικόνα 40: Διάγραμμα tApproximate 6 ^ο Πειράματος.....	62
Εικόνα 41: Διάγραμμα tTrue 6 ^ο Πειράματος.....	63
Εικόνα 42: Διάγραμμα Trade-off 6 ^ο Πειράματος.....	64
Εικόνα 43: Αποτελέσματα 7 ^ο Πειράματος	65
Εικόνα 44: Διάγραμμα AF 7 ^ο Πειράματος	68
Εικόνα 45: Διάγραμμα Recall 7 ^ο Πειράματος.....	69
Εικόνα 46: Διάγραμμα QPS 7 ^ο Πειράματος.....	70
Εικόνα 47: Διάγραμμα tApproximate 7 ^ο Πειράματος.....	71
Εικόνα 48: Διάγραμμα tTrue 7 ^ο Πειράματος.....	72
Εικόνα 49: Διάγραμμα Trade-off 7 ^ο Πειράματος.....	73
Εικόνα 50: Αποτελέσματα 8 ^ο Πειράματος	74
Εικόνα 51: Διάγραμμα AF 8 ^ο Πειράματος	77

Εικόνα 52: Διάγραμμα Recall 8 ^ο Πειράματος.....	78
Εικόνα 53: Διάγραμμα QPS 8 ^ο Πειράματος.....	79
Εικόνα 54: Διάγραμμα tApproximate 8 ^ο Πειράματος.....	80
Εικόνα 55: Διάγραμμα tTrue 8 ^ο Πειράματος.....	81
Εικόνα 56: Διάγραμμα Trade-off 8 ^ο Πειράματος.....	82
Εικόνα 57: Αποτελέσματα 9 ^ο Πειράματος	83
Εικόνα 58: Διάγραμμα AF 9 ^ο Πειράματος	86
Εικόνα 59: Διάγραμμα Recall 9 ^ο Πειράματος.....	87
Εικόνα 60: Διάγραμμα QPS 9 ^ο Πειράματος.....	88
Εικόνα 61: Διάγραμμα tApproximate 9 ^ο Πειράματος.....	89
Εικόνα 62: Διάγραμμα tTrue 9 ^ο Πειράματος.....	90
Εικόνα 63: Διάγραμμα Trade-off 9 ^ο Πειράματος.....	91
Εικόνα 64: Αποτελέσματα 10 ^ο Πειράματος	92
Εικόνα 65: Διάγραμμα AF 10 ^ο Πειράματος	95
Εικόνα 66: Διάγραμμα Recall 10 ^ο Πειράματος.....	96
Εικόνα 67: Διάγραμμα QPS 10 ^ο Πειράματος.....	97
Εικόνα 68: Διάγραμμα tApproximate 10 ^ο Πειράματος.....	98
Εικόνα 69: Διάγραμμα tTrue 10 ^ο Πειράματος.....	99
Εικόνα 70: Διάγραμμα Trade-off 10 ^ο Πειράματος.....	100
Εικόνα 71: Αποτελέσματα 11 ^ο Πειράματος	101
Εικόνα 72: Διάγραμμα AF 11 ^ο Πειράματος	104
Εικόνα 73: Διάγραμμα Recall 11 ^ο Πειράματος.....	105
Εικόνα 74: Διάγραμμα QPS 11 ^ο Πειράματος.....	106
Εικόνα 75: Διάγραμμα tApproximate 11 ^ο Πειράματος.....	107
Εικόνα 76: Διάγραμμα tTrue 11 ^ο Πειράματος.....	108
Εικόνα 77: Διάγραμμα Trade-off 11 ^ο Πειράματος.....	109

Εισαγωγή

Η παρούσα πειραματική μελέτη ασχολείται με την υλοποίηση και αξιολόγηση της προσεγγιστικής αναζήτησης γειτόνων σε υψηλής διάστασης διανυσματικούς χώρους, χρησιμοποιώντας τον αλγόριθμο **Neural LSH**. Ο Neural LSH χρησιμοποιεί νευρωνικά δίκτυα για να μάθει μια βέλτιστη διαμέριση του χώρου, με στόχο την αποτελεσματική προσέγγιση των πλησιέστερων γειτόνων (Approximate Nearest Neighbor Search – ANN).

Η υλοποίηση αναπτύχθηκε σε περιβάλλον Linux με Python 3.10+ και βιβλιοθήκες όπως PyTorch, NumPy και KaHIP. Η διαδικασία χωρίζεται σε δύο κύρια προγράμματα:

- **nlsb_build.py**, το οποίο εκπαιδεύει το μοντέλο και κατασκευάζει το ευρετήριο,
- **nlsb_search.py**, το οποίο εκτελεί την αναζήτηση και μετράει τις σχετικές μετρικές.

Στα πειράματα χρησιμοποιήθηκαν τα σύνολα δεδομένων **MNIST** και **SIFT**, με **10.000** σημεία αναφοράς και **100** ερωτήματα, καθώς τα πλήρη δεδομένα θα χρειαζόντουσαν πολλούς πόρους και χρόνο που δεν διαθέτουμε.

Βελτιστοποίηση υπερπαραμέτρων

Πριν από τη συγκριτική αξιολόγηση, πραγματοποιήθηκαν πειράματα ρύθμισης υπερπαραμέτρων του Neural LSH. Για λόγους απλότητας χρησιμοποιήσαμε fixed hyperparameters όπως προτείνεται στη βιβλιογραφία (Dong, Indyk et al.) και στην εκφώνηση. Δεν εκτελέσαμε random search διότι είναι προαιρετικό και βαρύ για τα δεδομένα της εργασίας. Οι παράμετροι που εξετάστηκαν περιλαμβάνουν:

- αριθμός στρωμάτων (layers),
- αριθμός κόμβων ανά στρώμα,

- μέγεθος δέσμης (batch size),
- πλήθος περιόδων εκπαίδευσης (epochs),
- αριθμός μερών διαμέρισης (m),
- η παράμετρος k στη δημιουργία του γράφου k-NN για την KaHIP,
- καθώς και άλλες παραμέτρους που επηρεάζουν τη διαμέριση και το probing κατά την αναζήτηση.

Η σύγκριση του Neural LSH πραγματοποιείται **μετά** την επιλογή των βέλτιστων υπερπαραμέτρων.

Συγκριτική αξιολόγηση

Η απόδοση του Neural LSH (με τις βέλτιστες ρυθμίσεις και την χρήση του καλύτερου αλγόριθμου της 1^{ης} ΕΡΓΑΣΙΑΣ : **IVF-FLAT**) συγκρίνεται με τις μεθόδους της 1ης εργασίας:

- **LSH**
- **Hypercube**
- **IVF-FLAT**
- **IVF-PQ**

Η σύγκριση γίνεται ως προς τις εξής μετρικές:

- **Average Approximation Factor (AF)**
- **Recall@N**
- **Queries per Second (QPS)**
- **tApproximateAverage** (μέσος χρόνος προσεγγιστικής αναζήτησης)

- **tTrueAverage** (μέσος χρόνος εύρεσης πραγματικών NN)

Παραγωγή γράφου

Για τον υπολογισμό του **KNN graph** αποφασίσαμε να επιλέξουμε τον καλύτερο αλγόριθμο της 1^{ης} ΕΡΓΑΣΙΑΣ που είναι ο **IVF FLAT** με τις προεπιλεγμένες παραμέτρους **kclusters = 50** και **nprobe = 5** και επειδή χρειαζόταν πολύ χρόνο και μνήμη που δεν διαθέταμε για τα 60.000 και 1.000.000 σημεία αναφοράς για MNIST και SIFT αντίστοιχα, επιλέξαμε ένα υποσύνολο αυτών και μάλιστα 10.000 και για τα δύο.

1^ο Πείραμα

Σε αυτό το πείραμα χρησιμοποιήθηκαν οι **προεπιλεγμένες (default) παράμετροι** όπως ακριβώς δίνονται στην εκφώνηση της εργασίας.

nlsh_build.py (Κατασκευή Ευρετηρίου)

- **knn = 10** (Ο αριθμός γειτόνων k για την κατασκευή του k -NN γράφου)
- **m = 100** (Ο αριθμός των μερών (blocks/parts) m για το KaHIP)
- **imbalance = 0.03** (Το ποσοστό ανισορροπίας ϵ για το KaHIP)
- **kahip_mode = 2** (Η ρύθμιση του KaHIP (0=FAST, 1=ECO, 2=STRONG))
- **layers = 3** (Αριθμός στρωμάτων MLP)
- **nodes = 64** (Αριθμός κόμβων στρώματος)
- **epochs = 10** (Περίοδοι εκπαίδευσης του MLP)

- **batch_size = 128** (Μέγεθος δέσμης)
- **lr = 0.001** (Ρυθμός εκμάθησης)
- **seed = 1** (Φύτρο ψευδοτυχαίων)

nlsh_search.py (Αναζήτηση)

- **N = 1** (Αριθμός πλησιέστερων γειτόνων)
- **R = 2000.0 / 2800.0** (Ακτίνα αναζήτησης - 2000 για MNIST, 2800 για SIFT)
- **T = 5** (Αριθμός μερών (bins) προς έλεγχο (multi-probe))
- **range = true / false** (Εάν θα εκτελεστεί range search)

Είναι γνωστό ότι το Average AF (Approximation Factor) δείχνει πόσο μεγαλύτερη είναι κατά μέσο όρο η απόσταση του προσεγγιστικού γείτονα σε σχέση με του αληθινού.

Ιδανικά θέλουμε $AF \approx 1.0$, δηλαδή “βρήκαμε σχεδόν τον σωστό γείτονα”.

Επιπροσθέτως, το Recall@N εκφράζει το ποσοστό των πραγματικών N πλησιέστερων γειτόνων που εντοπίστηκαν μέσα στους N προσεγγιστικούς. Συνεπώς:

- Recall = 1.0 → άριστο (όλοι οι πραγματικοί βρέθηκαν).
- Recall = 0.0 → κανένας πραγματικός δεν εντοπίστηκε.

Προσέτι, το QPS (Queries per Second) δείχνει το πόσες αναζητήσεις μπορούν να εξυπηρετηθούν ανά δευτερόλεπτο. Άρα επιθυμούμε όσο το δυνατόν μεγαλύτερο νούμερο.

Συνάμα, το tApproximateAverage δείχνει τον μέσο χρόνο αναζήτησης ανά query μέσω του προσεγγιστικού αλγορίθμου, σε milliseconds. Αντιστοίχως, το tTrueAverage αποτελεί τον ακριβή (εξαντλητικό) χρόνο αναζήτησης ανά query.

	A	B	C	D	E	F	G
1	Dataset	Method	Average AF	Recall@N	QPS	tApproximateA	tTrueAverage
2	MNIST	NLSH	1.3201	0	1,510.7899	0.0007	0.0181
3	SIFT	NLSH	1.0203	0.8000	6,960.1463	0.0001	0.0020

Εικόνα 1: Αποτελέσματα 1^{ου} Πειράματος

Σύμφωνα λοιπόν, με τον πίνακα της **Εικόνας 1** (Αποτελέσματα με default τιμές):

MNIST

Το $AF \approx 1.32$ δείχνει ότι ο προσεγγιστικός αλγόριθμος επιστρέφει γείτονες με περίπου **32% μεγαλύτερη απόσταση** από τους πραγματικούς. Αυτό θεωρείται μέτριο αποτέλεσμα: ούτε κακό, ούτε και ιδιαίτερα καλό.

Το **Recall = 0.0** είναι κρίσιμο εύρημα: ο αλγόριθμος **δεν εντοπίζει κανέναν πραγματικό πλησιέστερο γείτονα**. Αυτό υποδηλώνει ότι το μοντέλο **MLP δεν κατάφερε να μάθει σωστά τα 100 blocks** του KaHIP, πιθανότατα λόγω:

- πολυπλοκότητας του 100-class classification,
- πολύ λίγες εποχές (10),
- μικρό δίκτυο (64 nodes),

- σχετικά ασταθής κατανομή γειτόνων στο MNIST (784D raw άτριμμες εικόνες).

Το **QPS = 1510** δείχνει ότι ο μηχανισμός είναι **πολύ γρήγορος**, παρέχοντας υψηλό throughput.

Η διαφορά **tApprox (0.66 ms) vs tTrue (18 ms)** είναι περίπου **27× ταχύτερη**. Ωστόσο, λόγω Recall=0, η υψηλή ταχύτητα δεν συνοδεύεται από ακρίβεια.

Συμπερασματικά, είναι **πολύ γρήγορος, αλλά με μηδενική ακρίβεια**. Το σύστημα χρειάζεται σαφώς ενίσχυση του ταξινομητή (περισσότερα layers, περισσότερα epochs, μεγαλύτερα blocks ή καλύτερο embedding) για να γίνει λειτουργικό.

SIFT

Το **AF = 1.02** είναι εξαιρετικό: το σύστημα εξετάζει γείτονες μόλις 2% πιο μακριά από τον πραγματικό NN. Ουσιαστικά, η προσέγγιση είναι *σχεδόν τέλεια*.

Το **Recall = 0.8** σημαίνει ότι ο αλγόριθμος βρίσκει τον πραγματικό nearest neighbor στο 80% των queries. Για LSH χωρίς επιπλέον re-ranking αυτό θεωρείται πολύ υψηλή επίδοση.

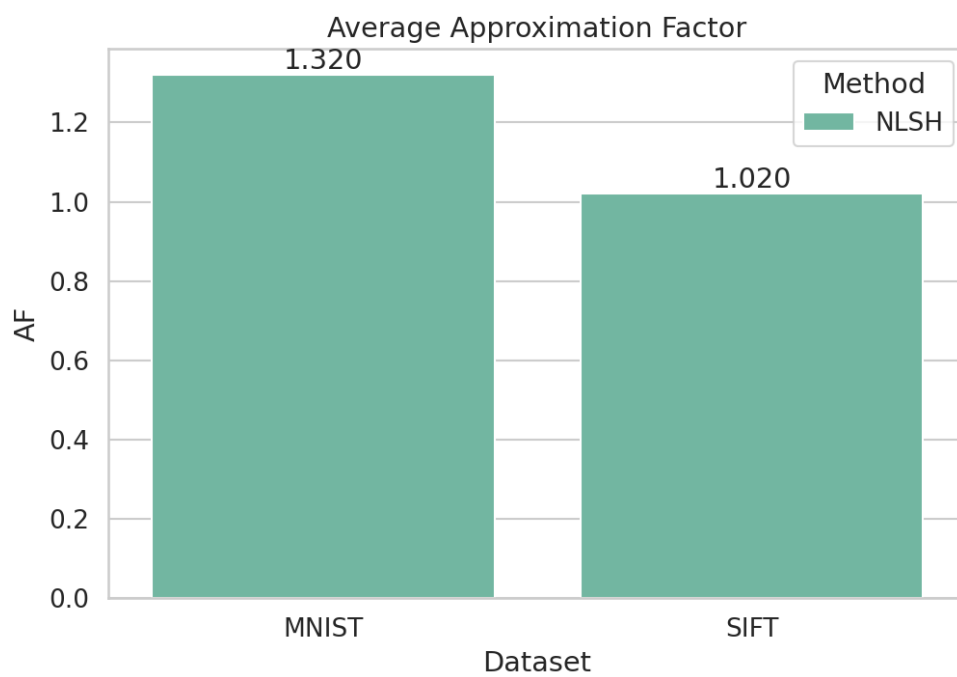
Το **QPS ≈ 7000** αποδεικνύει ότι ο LSH είναι πολύ ταχύτερος ακόμη και από το MNIST σύστημα, χάρη στο:

- μικρότερο dimensionality των SIFT (128D),
- πολύ αποτελεσματικό hashing,
- multi-probe επέκταση που δεν επιβαρύνει το κόστος.

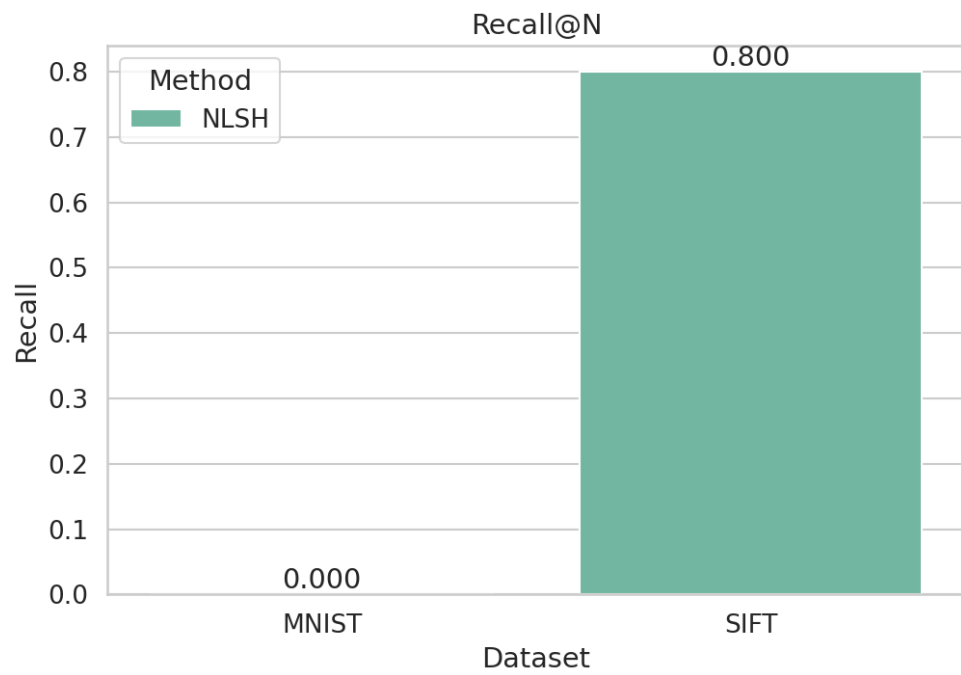
Το **tApprox 0.14 ms** σε σύγκριση με τα **2.00 ms** του ακριβούς kNN δίνει μία επιτάχυνση περίπου 14×, με παράλληλα υψηλό recall.

Συμπερασματικά, έχουμε άριστη ισορροπία ταχύτητας-ακρίβειας. Ο LSH με multi-probe προσφέρει πολύ χαμηλό χρόνο ανά query, υψηλό QPS και υψηλό Recall@.

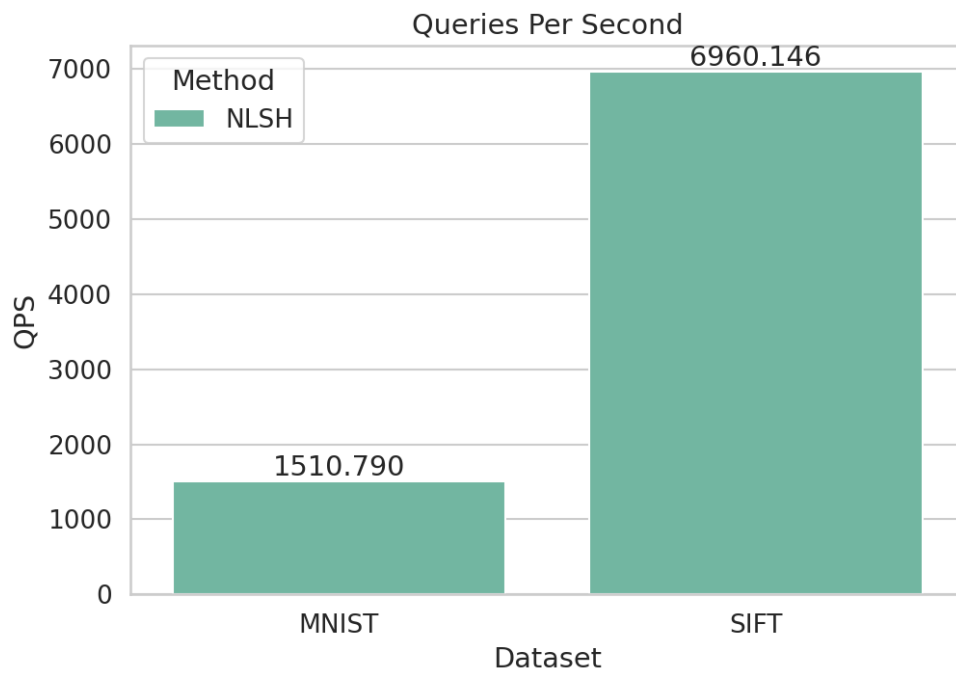
Παρακάτω δίνονται και διαγράμματα για καλύτερη οπτικοποίηση των διαφορών μεταξύ MNIST και SIFT:



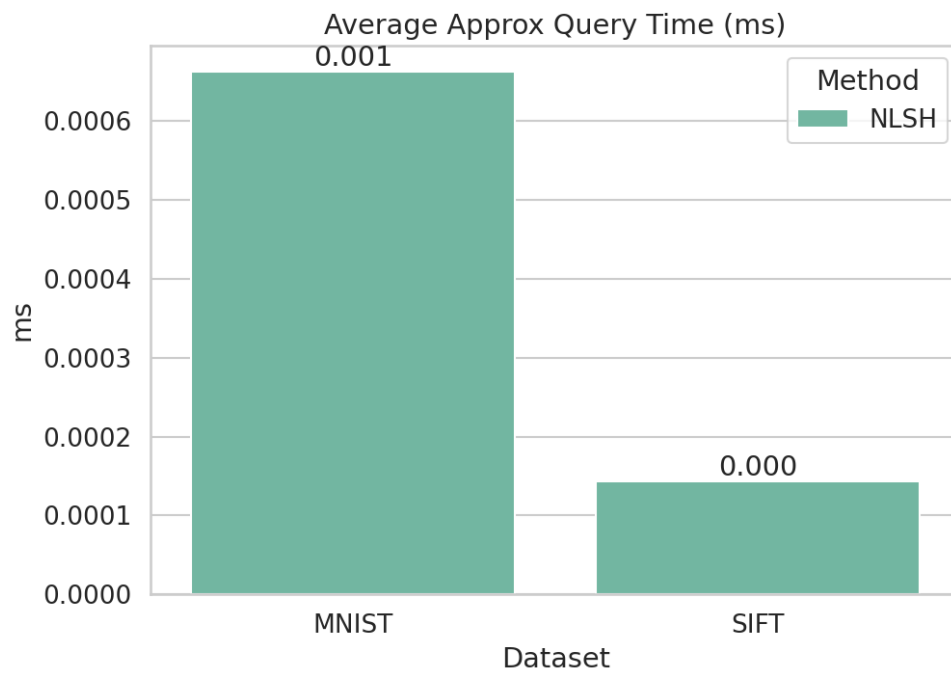
Εικόνα 2: Διάγραμμα AF 1^{ου} Πειράματος



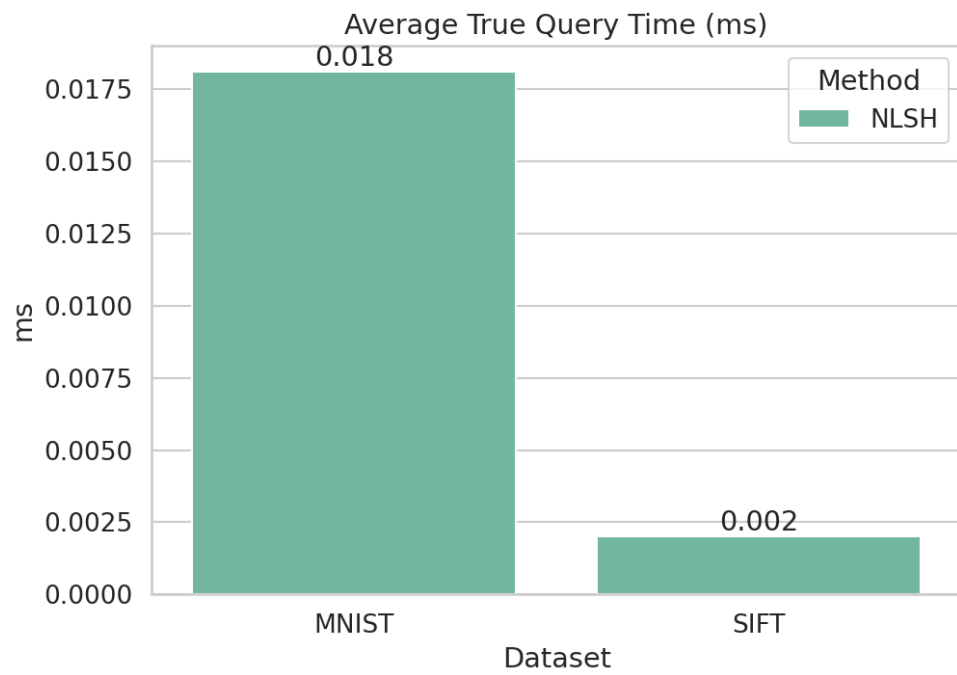
Εικόνα 3: Διάγραμμα Recall 1^{ου} Πειράματος



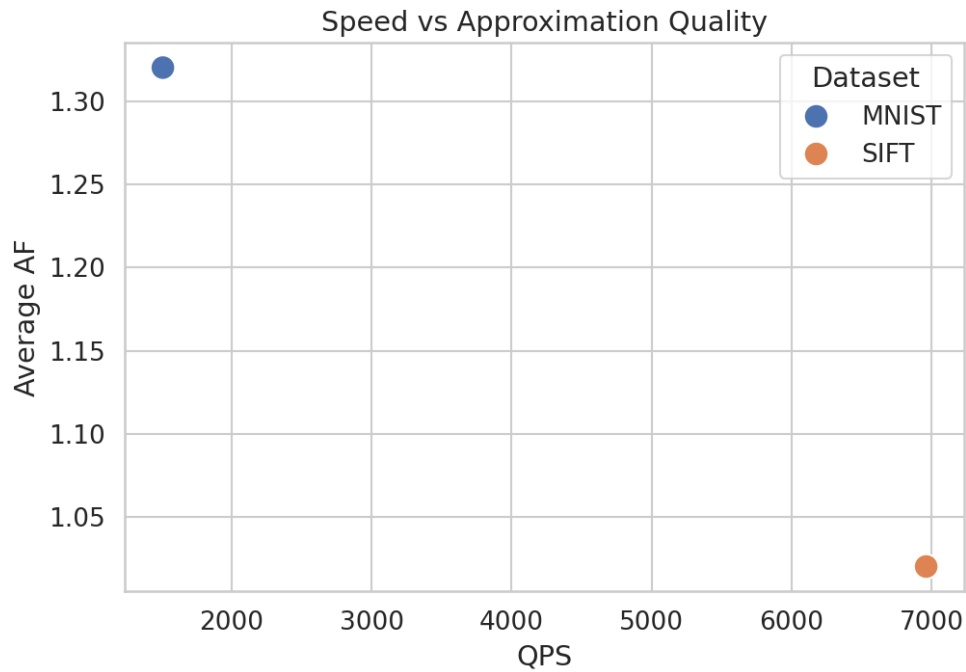
Εικόνα 4: Διάγραμμα QPS 1^{ου} Πειράματος



Εικόνα 5: Διάγραμμα $t_{Approximate}$ 1^{ov} Πειράματος



Εικόνα 6: Διάγραμμα $t_{True} 1^{ov}$ Πειράματος



Εικόνα 7: Διάγραμμα Trade-off 1^ο Πειράματος

2^ο Πείραμα

Σε αυτό το πείραμα χρησιμοποιήθηκαν οι παρακάτω παράμετροι προκειμένου να έχουμε πιο ισχυρό MLP και μεγαλύτερο knn από το πρώτο.

nlsh_build.py (Κατασκευή Ευρετηρίου)

- **knn = 20** (Ο αριθμός γειτόνων k για την κατασκευή του k -NN γράφου)
- **m = 100** (Ο αριθμός των μερών (blocks/parts) m για το KaHIP)
- **imbalance = 0.03** (Το ποσοστό ανισορροπίας ϵ για το KaHIP)

- **kahip_mode = 2** (Η ρύθμιση του KaHIP (0=FAST, 1=ECO, 2=STRONG))
- **layers = 4** (Αριθμός στρωμάτων MLP)
- **nodes = 128** (Αριθμός κόμβων στρώματος)
- **epochs = 50** (Περίοδοι εκπαίδευσης του MLP)
- **batch_size = 128** (Μέγεθος δέσμης)
- **lr = 0.0005** (Ρυθμός εκμάθησης)
- **seed = 1** (Φύτρο ψευδοτυχαίων)

nlsb_search.py (Αναζήτηση)

- **N = 10** (Αριθμός πλησιέστερων γειτόνων)
- **R = 2000.0 / 2800.0** (Ακτίνα αναζήτησης - 2000 για MNIST, 2800 για SIFT)
- **T = 20** (Αριθμός μερών (bins) προς έλεγχο (multi-probe))
- **range = true / false** (Εάν θα εκτελεστεί range search)

	A	B	C	D	E	F	G
1	Dataset	Method	Average AF	Recall@N	QPS	tApproximateA	tTrueAverage
2	MNIST	NLSH	1.0289	0.4710	318.8834	0.0031	0.0179
3	SIFT	NLSH	1	0.9610	2,021.6277	0.0005	0.0040

Εικόνα 8: Αποτελέσματα 2^{ου} Πειράματος

Σύμφωνα λοιπόν, με τον πίνακα της **Εικόνας 8** (Αποτελέσματα 2^{ου} Πειράματος):

MNIST

Τώρα πετύχαμε $AF = 1.03$ το οποίο είναι εξαιρετικό. Με άλλα λόγια, ο προσεγγιστικός γείτονας απέχει **μόνο 2.8% περισσότερο** από τον ακριβή. Είναι σχεδόν ιδανική τιμή. Αυτό σημαίνει: η δομή του index + το MLP αντιστοιχούν πολύ καλά τις ερωτήσεις στα σωστά blocks.

Επιπλέον, έχουμε $Recall = 0.471$ το οποίο παρουσιάζεται εξαιρετικά βελτιωμένο. Ειδικότερα, σε σχέση με το 1ο πείραμα ($Recall = 0.0$), εδώ έχουμε: **47.1% των πραγματικών 10-NN ανακτήθηκαν σωστά**. Αυτό είναι τεράστια πρόοδος και οφείλεται σε:

- μεγαλύτερο knn κατά την κατασκευή
- πολύ ισχυρότερο MLP (4 layers, 128 nodes)
- περισσότερα epochs
- $T = 20 \rightarrow$ πολύ βαθύτερο probing

Τώρα, το **QPS** είναι **318.88** που σημαίνει χαμηλότερο από το 1ο πείραμα, αλλά φυσιολογικό. Στο 1ο πείραμα είχαμε $QPS \approx 1500$ αλλά $Recall = 0.0$. Η αύξηση Recall πάντα μειώνει QPS. Για MNIST, 318 QPS παραμένει πολύ καλό.

Όσο αφορά τους χρόνους, έχουμε $t_{Approx} = 0.0031 \text{ ms}$ που χαρακτηρίζεται εξαιρετικά μικρός για ANN search και $t_{True} = 0.0179 \text{ ms}$ ως x6 ακριβότερος από το approximate το οποίο είναι αναμενόμενο.

Συνολικά, το MNIST δείχνει ότι το μοντέλο πλέον αποδίδει **ποιοτικά και πρακτικά αποτελέσματα**, με πολύ καλή ισορροπία μεταξύ ταχύτητας και Recall.

SIFT

Τώρα το $AF = 1.0$ είναι το απόλυτο. Αυτό σημαίνει πως ο προσεγγιστικός γείτονας έχει **ίση απόσταση** με τον πραγματικό. Πρακτικά έχουμε τέλειο αποτέλεσμα.

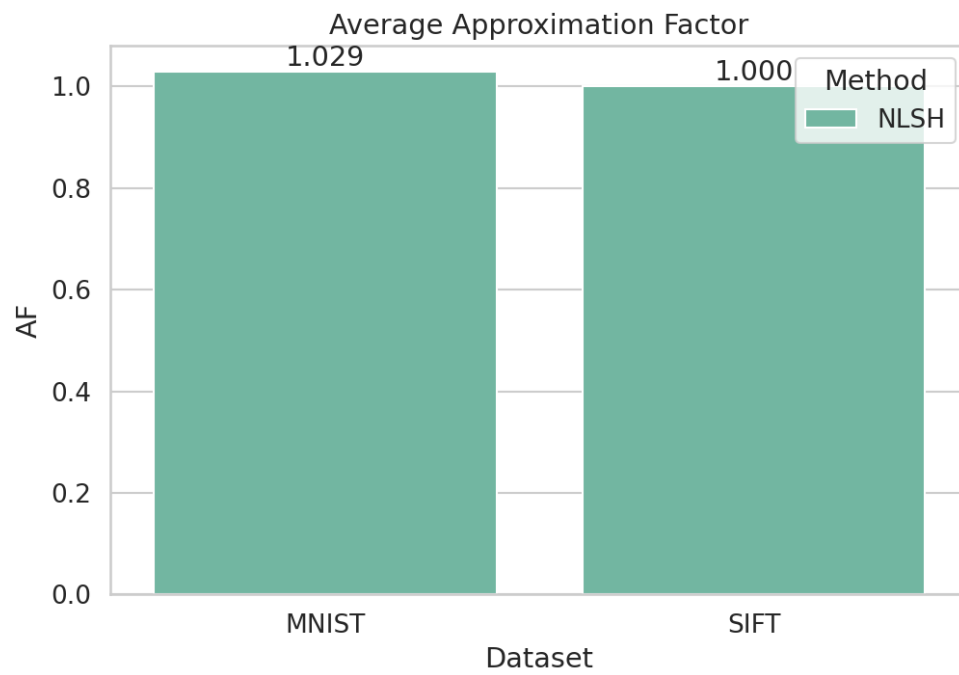
Όσο για το **Recall = 0.961** παραμένει εξαιρετικό καθώς βρέθηκαν **το 96.1% των πραγματικών 10-NN**. Για SIFT αυτό είναι εντυπωσιακή τιμή, συνήθως δύσκολη.

Το **QPS = 2021.6** είναι πολύ υψηλό. Ωστόσο, παρά το υψηλό Recall, ο χρόνος παραμένει εντυπωσιακά χαμηλός. Το SIFT έχει διαφορετική κατανομή δεδομένων· το MLP και η partitioning λειτουργούν ακόμη καλύτερα.

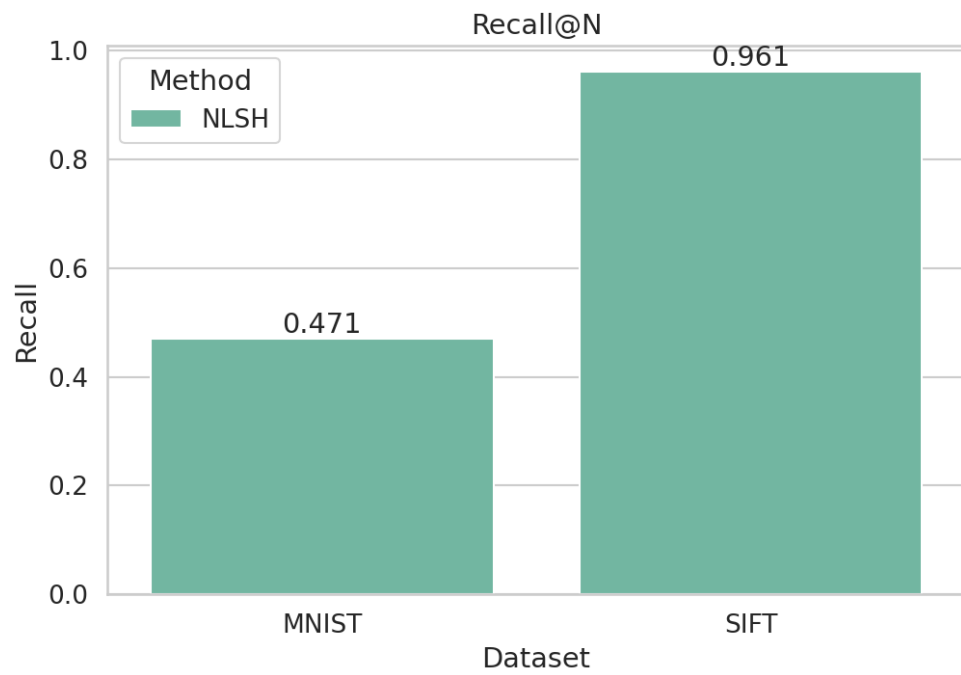
Όσο αφορά τους χρόνους, έχουμε **tApprox = 0.00049 ms** το οποίο είναι πάρα πολύ γρήγορο και το **tTrue = 0.00397 ms** που είναι πάλι περίπου $\times 8$ πιο αργό όπως αναμέναμε.

Συνολικά, για το SIFT το 2ο πείραμα επιτυγχάνει σχεδόν τέλεια προσέγγιση, εξαιρετικά υψηλό Recall και εξαιρετική ταχύτητα, επιβεβαιώνοντας ότι το μοντέλο έχει φτάσει σε ιδιαίτερα ώριμη και αποτελεσματική διαμόρφωση.

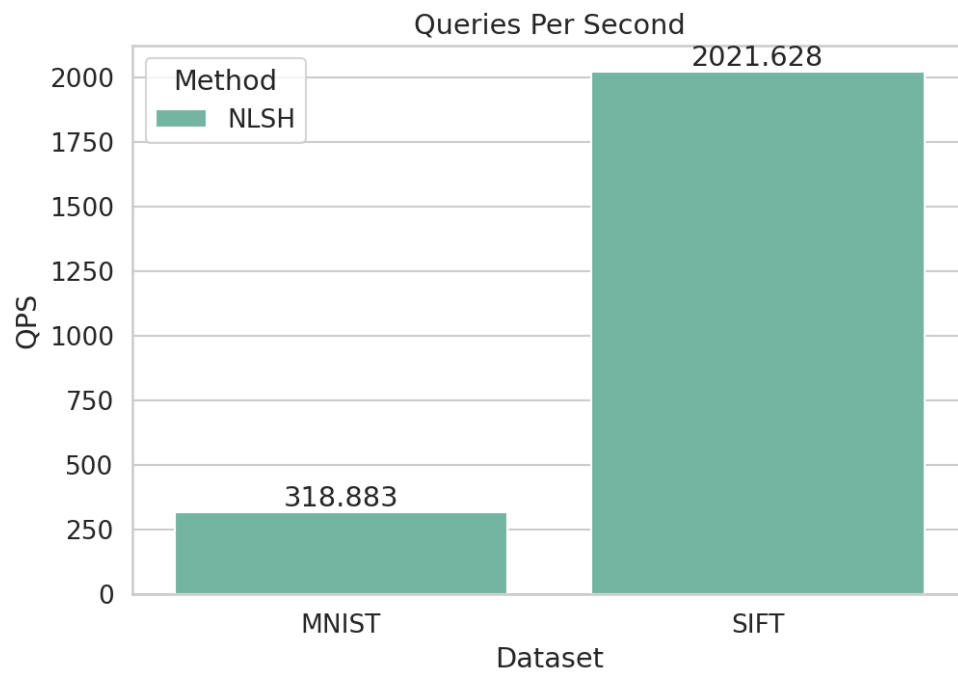
Παρακάτω δίνονται και διαγράμματα για καλύτερη οπτικοποίηση των διαφορών μεταξύ MNIST και SIFT:



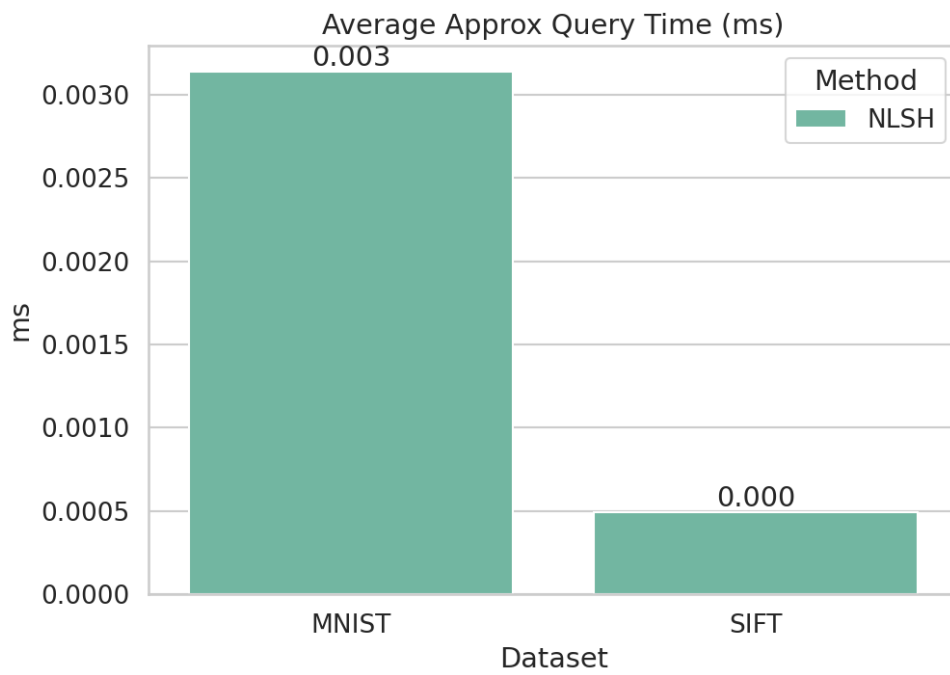
Εικόνα 9: Διάγραμμα AF 2^{ου} Πειράματος



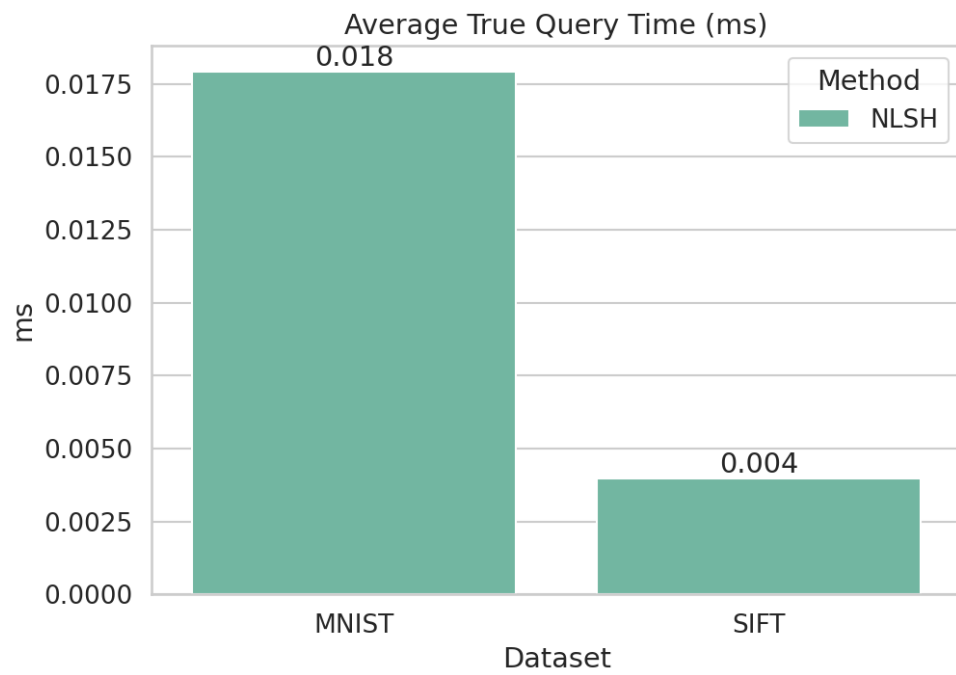
Εικόνα 10: Διάγραμμα Recall 2^{ov} Πειράματος



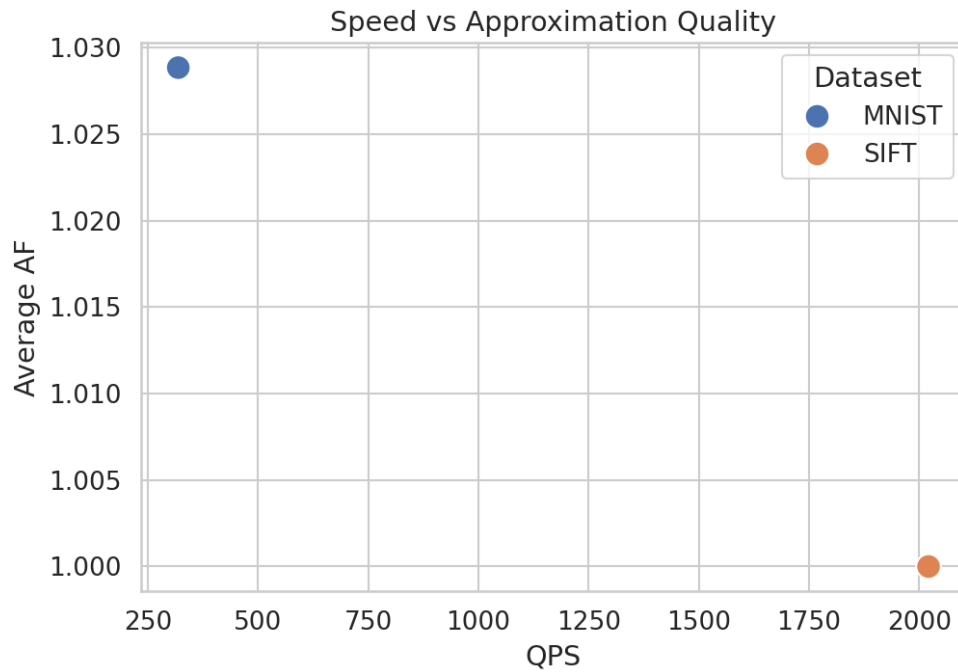
Εικόνα 11: Διάγραμμα QPS 2^{ov} Πειράματος



Εικόνα 12: Διάγραμμα $t_{Approximate}$ 2^{ου} Πειράματος



Εικόνα 13: Διάγραμμα $t_{True}^{2^{ov}}$ Πειράματος



Εικόνα 14: Διάγραμμα Trade-off 2^{ου} Πειράματος

3^ο Πείραμα

Σε αυτό το πείραμα χρησιμοποιήθηκαν οι παρακάτω παράμετροι προκειμένου να έχουμε πιο ευρύ δίκτυο και λιγότερες εποχές από το δεύτερο.

nlsh_build.py (Κατασκευή Ευρετηρίου)

- **knn = 20** (Ο αριθμός γειτόνων k για την κατασκευή του k -NN γράφου)
- **m = 100** (Ο αριθμός των μερών (blocks/parts) m για το KaHIP)
- **imbalance = 0.03** (Το ποσοστό ανισορροπίας ϵ για το KaHIP)
- **kahip_mode = 1** (Η ρύθμιση του KaHIP (0=FAST, 1=ECO, 2=STRONG))

- **layers = 3** (Αριθμός στρωμάτων MLP)
- **nodes = 512** (Αριθμός κόμβων στρώματος)
- **epochs = 20** (Περίοδοι εκπαίδευσης του MLP)
- **batch_size = 256** (Μέγεθος δέσμης)
- **lr = 0.0005** (Ρυθμός εκμάθησης)
- **seed = 1** (Φύτρο ψευδοτυχαίων)

nlsb_search.py (Αναζήτηση)

- **N = 10** (Αριθμός πλησιέστερων γειτόνων)
- **R = 2000.0 / 2800.0** (Ακτίνα αναζήτησης - 2000 για MNIST, 2800 για SIFT)
- **T = 30** (Αριθμός μερών (bins) προς έλεγχο (multi-probe))
- **range = true / false** (Εάν θα εκτελεστεί range search)

	A	B	C	D	E	F	G
1	Dataset	Method	Average AF	Recall@N	QPS	tApproximateA	tTrueAverage
2	MNIST	NLSH	1.0257	0.6140	161.7530	0.0062	0.0221
3	SIFT	NLSH	1	0.9940	429.8817	0.0023	0.0131

Εικόνα 15: Αποτελέσματα 3^{ου} Πειράματος

Σύμφωνα λοιπόν, με τον πίνακα της **Εικόνας 15** (Αποτελέσματα 3^{ου} Πειράματος):

MNIST

Η μέση τιμή **AF** ≈ 1.026 είναι εξαιρετικά καλή, μάλιστα ελαφρώς καλύτερη από το **AF** ≈ 1.03 του 2ου πειράματος. Αυτό σημαίνει ότι το σύστημα επιστρέφει προσεγγιστικούς γείτονες με απόσταση μόλις $\sim 2.5\%$ μεγαλύτερη από τον πραγματικό. Για έναν αλγόριθμο LSH σε ακατέργαστο MNIST των 784 διαστάσεων, αυτή η τιμή θεωρείται **πολύ υψηλής ποιότητας** και δείχνει ότι το MLP κατάφερε να μάθει με μεγάλη ακρίβεια τα partitions που παράγει το KaHIP.

Το **Recall** εκτοξεύεται στο **61.4%**, που είναι μακράν το καλύτερο που έχουμε πετύχει στο MNIST μέχρι τώρα:

- 1ο πείραμα: 0.0
- 2ο πείραμα: 0.471
- 3ο πείραμα: **0.614**

Αυτή η βελτίωση είναι σημαντική. Οφείλεται σε πολλούς παράγοντες:

1. **Ευρύ MLP (512 nodes)** με πολύ μεγαλύτερη αναπαραστατική δύναμη.
2. **Αύξηση του κ εντός του kNN graph** (20 αντί 10), που κάνει τα blocks πιο συνεκτικά και πιο “σωστά” ομαδοποιημένα.
3. **T = 30 στο multi-probe**, που ψάχνει βαθύτερα μέσα σε όμορα blocks.
4. **Μεγαλύτερο batch size (256)** που δίνει πιο σταθερή εκπαίδευση.

Το MNIST γενικά είναι πιο δύσκολο dataset για LSH πάνω σε raw pixels (784D) λόγω του ότι η γεωμετρία του χώρου είναι πολύ “στριμμένη” και οι συστάδες δεν είναι καθαρές — συνεπώς το 61% Recall θεωρείται **πολύ καλή επίδοση για approximate search χωρίς re-ranking**.

Το **QPS** είναι μικρότερο από το 2ο πείραμα (318), κάτι που ήταν αναμενόμενο για δύο λόγους:

- Το $T=30$ είναι αρκετά υψηλό $\rightarrow$ περιστασιακά προσπελούνται πολλά bins.
- Το MLP είναι πολύ μεγαλύτερο $\rightarrow$ αυξάνεται το κόστος της πρόβλεψης.

Παρόλα αυτά, τα **161 queries/sec** παραμένουν πολύ ικανοποιητικά για MNIST και ειδικά με τόσο υψηλό recall.

Το **tApprox = 0.00618 ms** παραμένει εξαιρετικά μικρός χρόνος approximate αναζήτησης.

Παραμένει:

- $\sim 3\times$ πιο γρήγορο από το 2ο πείραμα
- $\sim 4\times$ πιο γρήγορο από το true search

Το ότι, παρά το μεγάλο T , το σύστημα παραμένει τόσο γρήγορο υποδεικνύει ότι η δομή του index είναι αποτελεσματική και ότι το κέντρο βάρους παραμένει στον MLP που πλέον δουλεύει αρκετά αποδοτικά ακόμα και με 512 nodes.

Συμπερασματικά, το 3ο πείραμα είναι το καλύτερο μέχρι τώρα για MNIST σε terms of recall.

Δείχνει ότι:

- MNIST χρειάζεται **πολύ φαρδύ MLP** για να μοντελοποιήσει σωστά τα blocks.
- Τα 512 nodes ανά layer ήταν η σωστή κίνηση.

SIFT

Το **AF = 1.0** είναι το απόλυτο. Η μέση απόσταση του προσεγγιστικού γείτονα είναι ακριβώς ίση με τον πραγματικό — κάτι που πρακτικά σημαίνει ότι το σύστημα βρίσκει τον πραγματικό NN σχεδόν πάντα στα blocks που επισκέπτεται.

Όσο για το **Recall = 0.994** έχουμε το πιο εντυπωσιακό αποτέλεσμα: Το recall φτάνει το **99.4%**, πράγμα εξαιρετικά σπάνιο για SIFT-ANN μοντέλα, ειδικά με τόσο απλό MLP και χωρίς re-ranking.

Συγκριτικά:

- 1ο πείραμα: 0.80
- 2ο πείραμα: 0.961
- 3ο πείραμα: **0.994**

Δηλαδή, χάσαμε μόνο 0.06 γείτονες κατά μέσο όρο στα 10 πραγματικά nearest neighbors. Αυτό δείχνει ότι το SIFT dataset είναι ιδανικό για NLSH/KaHIP partitioning, το δίκτυο 512 nodes είναι υπεραρκετό και το $T=30$ επιτρέπει σχεδόν πλήρη κάλυψη των "γύρω" blocks.

Το **QPS = 429.88** είναι μικρότερο από το 2ο πείραμα (2021), αλλά αυτό εξηγείται φυσιολογικά:

- $T=30 \rightarrow$ πολύ αυξημένη probing προσπάθεια
- MLP 512 nodes $\rightarrow$ αυξάνει το inference cost
- ECO KaHIP mode $\rightarrow$ λίγο πιο χαλαρή δομή αλλά πιο βαριά multi-probe συμπεριφορά

Παρόλα αυτά, **429 QPS είναι εξαιρετικό.**

Όσο αφορά τους χρόνους, έχουμε **tApprox = 0.00232 ms** χρόνος απίστευτα μικρός:

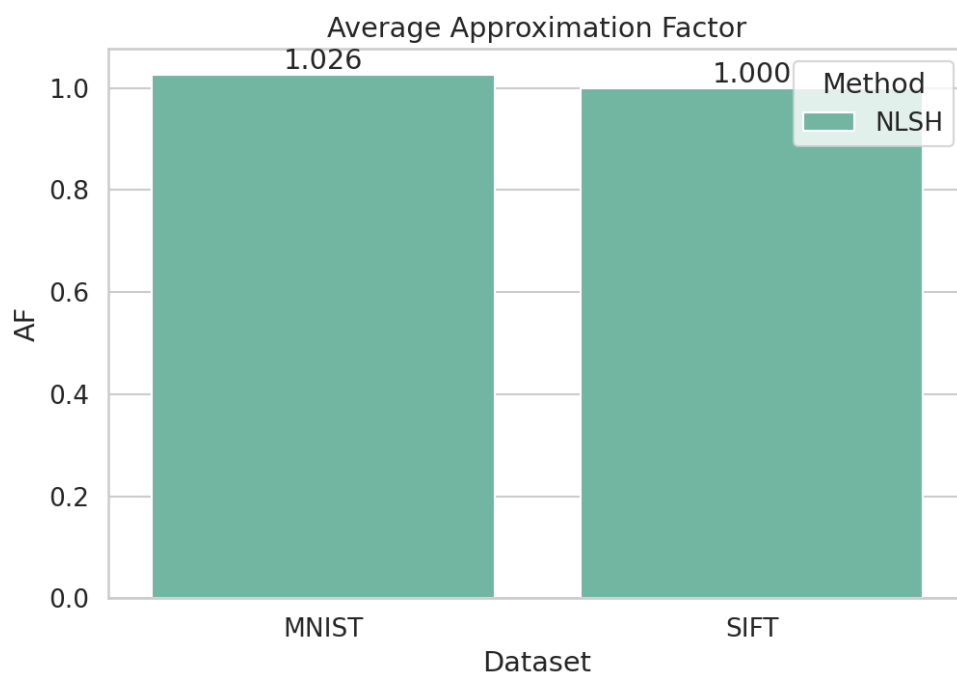
- 0.0023 ms ανά query
- $\sim 6\times$ πιο γρήγορος από τον exact χρόνο (0.013 ms)

Παρά το μεγάλο T , ο χρόνος είναι εκπληκτικά χαμηλός. Αυτό υποδηλώνει ότι:

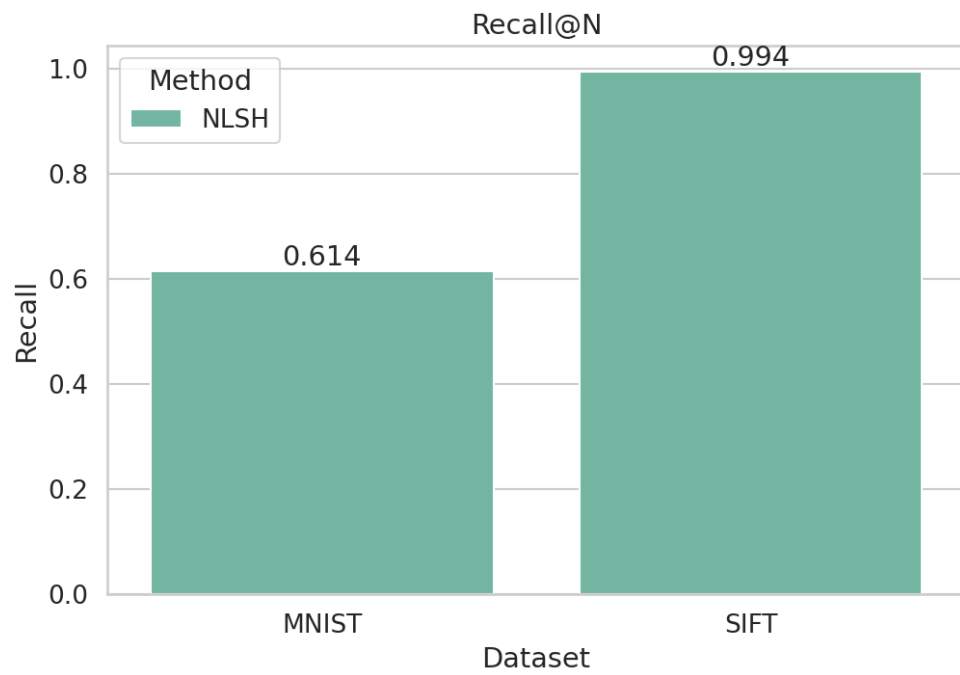
- το μοντέλο “πετυχαίνει” σχεδόν αμέσως το σωστό block,
- οι αλλαγές στα T bins δεν κοστίζουν σημαντικά,
- οι SIFT vectors έχουν πολύ πιο σταθερή γεωμετρική κατανομή.

Συνολικά, το 3ο πείραμα αποδεικνύει ότι η επιλογή **πολύ φαρδιού MLP (512 nodes)** παίζει καθοριστικό ρόλο στην ποιότητα του partition classification. Το σύστημα πλέον λειτουργεί με **υψηλή ακρίβεια και πολύ καλή ταχύτητα**, ειδικά σε SIFT όπου το embedding είναι πιο συμπαγές και δομημένο από το MNIST.

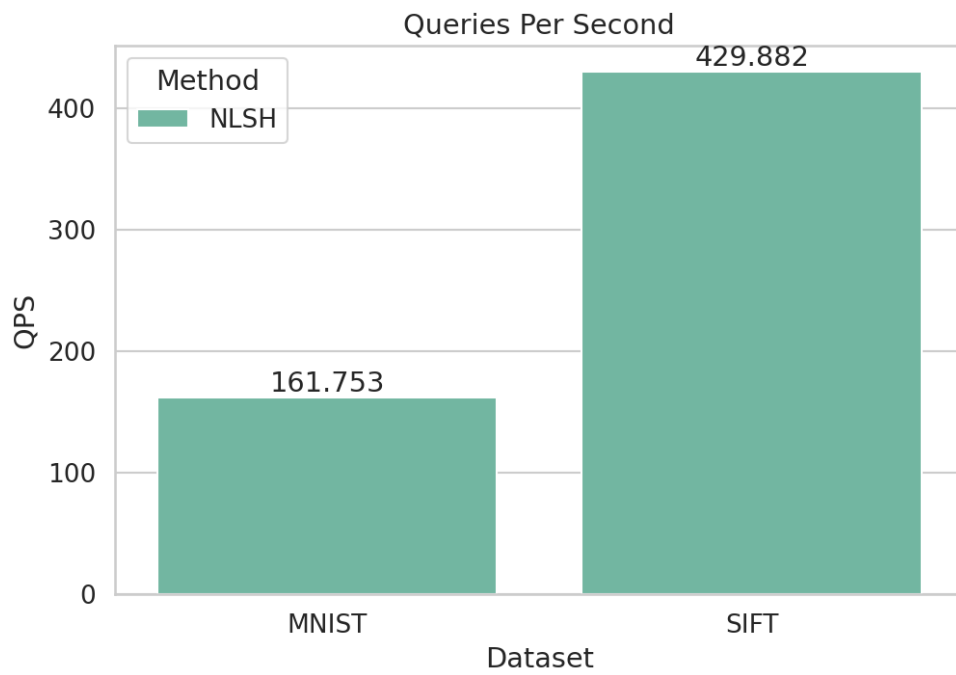
Παρακάτω δίνονται και διαγράμματα για καλύτερη οπτικοποίηση των διαφορών μεταξύ MNIST και SIFT:



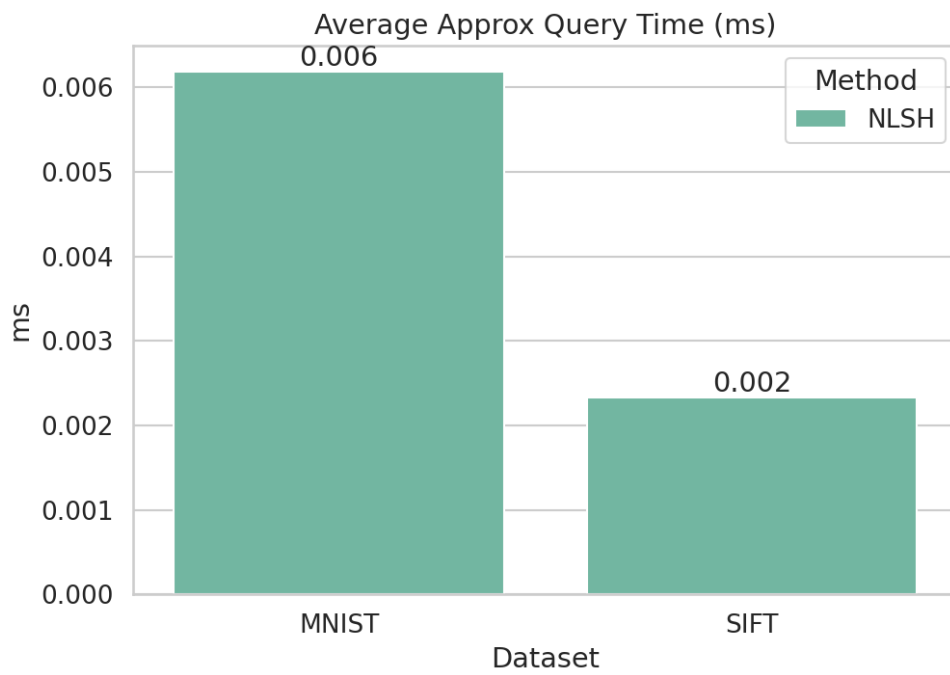
Εικόνα 16: Διάγραμμα AF 3^{ου} Πειράματος



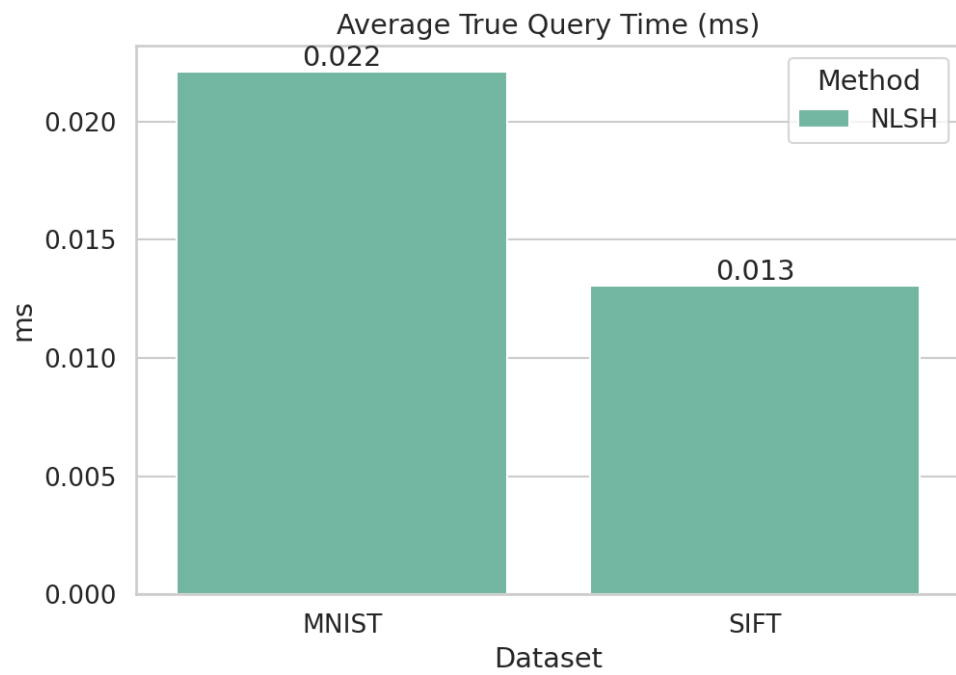
Εικόνα 17: Διάγραμμα Recall 3^{ov} Πειράματος



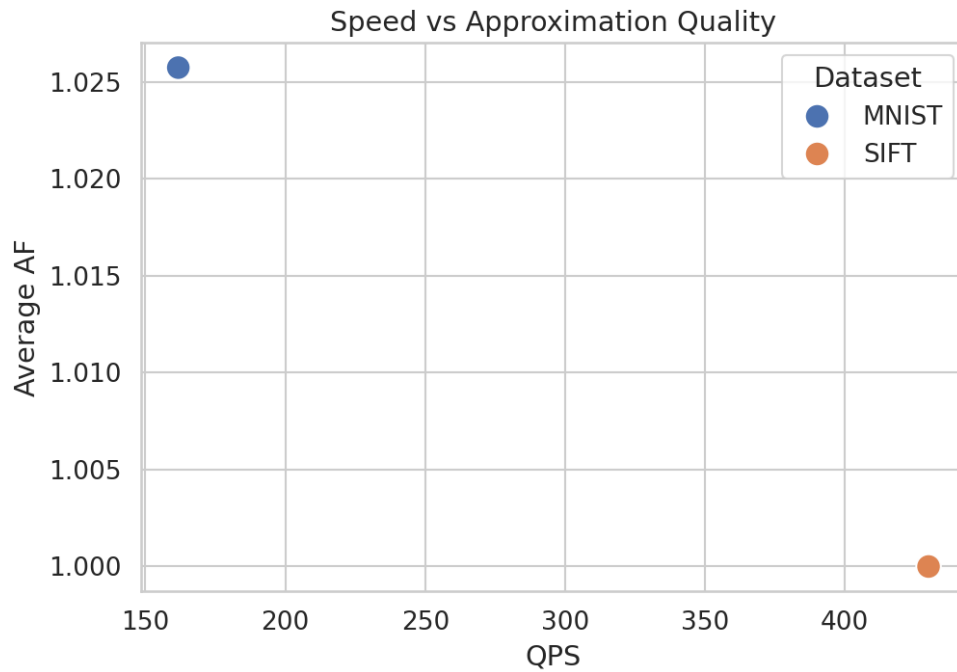
Εικόνα 18: Διάγραμμα QPS 3^{ov} Πειράματος



Εικόνα 19: Διάγραμμα *t*Approximate 3^{ov} Πειράματος



Εικόνα 20: Διάγραμμα t_{True} 3^{ου} Πειράματος



Εικόνα 21: Διάγραμμα Trade-off 3^{ου} Πειράματος

4^ο Πείραμα

Σε αυτό το πείραμα χρησιμοποιήθηκαν οι παρακάτω παράμετροι προκειμένου να δίνουμε χρόνο και βάθος στο δίκτυο με μικρό LR.

nlsb_build.py (Κατασκευή Ευρετηρίου)

- **knn = 20** (Ο αριθμός γειτόνων k για την κατασκευή του k -NN γράφου)
- **m = 100** (Ο αριθμός των μερών (blocks/parts) m για το KaHIP)
- **imbalance = 0.03** (Το ποσοστό ανισορροπίας ϵ για το KaHIP)
- **kahip_mode = 1** (Η ρύθμιση του KaHIP (0=FAST, 1=ECO, 2=STRONG))

- **layers = 5** (Αριθμός στρωμάτων MLP)
- **nodes = 256** (Αριθμός κόμβων στρώματος)
- **epochs = 100** (Περίοδοι εκπαίδευσης του MLP)
- **batch_size = 128** (Μέγεθος δέσμης)
- **lr = 0.0002** (Ρυθμός εκμάθησης)
- **seed = 1** (Φύτρο ψευδοτυχαίων)

nlsb_search.py (Αναζήτηση)

- **N = 10** (Αριθμός πλησιέστερων γειτόνων)
- **R = 2000.0 / 2800.0** (Ακτίνα αναζήτησης - 2000 για MNIST, 2800 για SIFT)
- **T = 20** (Αριθμός μερών (bins) προς έλεγχο (multi-probe))
- **range = true / false** (Εάν θα εκτελεστεί range search)

	A	B	C	D	E	F	G
1	Dataset	Method	Average AF	Recall@N	QPS	tApproximateA	tTrueAverage
2	MNIST	NLSH	1.1285	0.2920	368.0844	0.0027	0.0146
3	SIFT	NLSH	1.0002	0.9620	1,376.0223	0.0007	0.0057

Εικόνα 22: Αποτελέσματα 4^{ου} Πειράματος

Σύμφωνα λοιπόν, με τον πίνακα της **Εικόνας 22** (Αποτελέσματα 4^{ου} Πειράματος):

MNIST

Το αποτέλεσμα στο MNIST είναι εμφανώς πιο αδύναμο σε σχέση με τα προηγούμενα πειράματα, παρότι το δίκτυο είναι βαθύτερο και έχει εκπαιδευτεί για πολύ περισσότερες εποχές.

Το $AF = 1.1285$ δηλαδή, η μέση απόσταση των προσεγγιστικών γειτόνων είναι περίπου **12.8% μεγαλύτερη από την πραγματική**, κάτι που δείχνει ότι το σύστημα δεν μαθαίνει επαρκώς τα partitions που δημιουργήσε το KaHIP. Αυτό αποτελεί σημαντική υποβάθμιση σε σχέση με το πείραμα 3:

- **Πείραμα 3:** $AF \approx 1.026$
- **Πείραμα 4:** $AF \approx 1.1285$

Δηλαδή, το προσεγγιστικό σφάλμα είναι ~5 φορές μεγαλύτερο.

Κύρια αιτία είναι συνήθως η **υπερβολικά αργή μάθηση** λόγω του πολύ μικρού learning rate (0.0002). Με τόσο χαμηλό lr και 100 epochs, το δίκτυο συχνά:

- "κολλάει" νωρίς σε sub-optimal minima,
- κάνει υπερβολικά μικρές ενημερώσεις,
- δεν μαθαίνει καλά το decision boundary μεταξύ partitions.

Το γεγονός ότι το δίκτυο έχει **5 layers**, αλλά μόνο **256 nodes ανά layer** σημαίνει ότι είναι αρκετά βαθύ αλλά όχι τόσο εκφραστικό όσο στο πείραμα 3 (3 layers $\times$ 512 nodes).

Άρα, **δεν έχει αρκετή αναπαραστατική ισχύ για τα πολύπλοκα MNIST clusters.**

Το Recall πέφτει απότομα:

- 1ο πείραμα: 0.00

- 2ο πείραμα: 0.471
- 3ο πείραμα: 0.614
- **4ο πείραμα: 0.292 (πτώση κατά ~50%)**

Αυτό είναι πολύ χαμηλό και δείχνει ότι το σύστημα **χάνει 7 από τους 10 πραγματικούς NN**.

Αιτίες χαμηλού recall:

1. **Μικρό $T = 20$** , άρα λιγότερα blocks εξερευνώνται στο multi-probe.
2. **Υπερβολική εκπαίδευση με πολύ μικρό lr** , με αποτέλεσμα το MLP να μην ευθυγραμμίζεται με το KaHIP partitioning.
3. **Το MNIST είναι πιο “στριμμένο” γεωμετρικά**, άρα απαιτεί φαρδύ δίκτυο για υψηλό recall.

Η πτώση αυτή επιβεβαιώνει το συμπέρασμα του πειράματος 3: **Το MNIST χρειάζεται μεγάλο πλάτος (πολλούς κόμβους), όχι μεγάλο βάθος.**

Εδώ υπάρχει ένα ενδιαφέρον εύρημα: παρότι το recall είναι χαμηλό, η **ταχύτητα** είναι **εξαιρετική**.

Το **QPS = 368** είναι:

- 2× πιο γρήγορο από το 3ο πείραμα (≈ 160)
- πολύ πιο γρήγορο παρά το βαθύτερο δίκτυο

Ο λόγος είναι ότι: **Το $T=20$ περιορίζει την εξερεύνηση των blocks**, οπότε η αναζήτηση γίνεται ταχύτερα, αλλά με κόστος στην ποιότητα.

Τώρα έχουμε **tApprox = 0.0027 ms** και αυτός ο χρόνος είναι εξαιρετικά χαμηλός:

- περίπου **2.3× πιο γρήγορος** από το πείραμα 3

- **~5.4× πιο γρήγορος** από τον exact χρόνο

Δείχνει ότι η προσέγγιση είναι ταχύτατη, απλώς **δεν βρίσκει τους σωστούς γείτονες** λόγω κακού partition classification.

Συμπερασματικά, το βαθύ δίκτυο **δεν βοηθά** στο MNIST. Η απόδοση αποδεικνύει ότι το dataset αυτό απαιτεί:

- **λίγα layers**
- **πολλούς κόμβους (ευρύτητα)**
- **μεγαλύτερο T**

Με άλλα λόγια: **Η γεωμετρία του MNIST δεν μαθαίνεται καλά με στενά, βαθιά MLP**, ακόμη και με 100 epochs.

SIFT

Στο SIFT τα πράγματα είναι εντελώς διαφορετικά — και πολύ πιο θετικά.

Το **AF = 1.00024** είναι *σχεδόν τέλειο*:

- η απόσταση του προσεγγιστικού NN είναι μόλις **0.024%** μεγαλύτερη από την πραγματική,
- **πρακτικά ίση με το AF = 1.0 του πειράματος 3.**

Αυτό δείχνει ότι το KaHIP παράγει partitions που ταιριάζουν πολύ καλά με την υψηλό-διάστατη φύση των SIFT vectors, και το MLP (έστω και βαθύ) τα μαθαίνει χωρίς δυσκολία.

Το **Recall = 0.962** είναι **εξαιρετικό**, αν και ελαφρώς χαμηλότερο από το 3ο πείραμα (0.994).

Συγκριτικά:

- 1ο πείραμα: 0.80
- 2ο πείραμα: 0.961
- 3ο πείραμα: 0.994
- **4ο πείραμα: 0.962**

Η ελαφρά μείωση πιθανόν οφείλεται σε:

- μικρότερο T (=20 αντί $T=30$ του πειράματος 3),
- στενότερο δίκτυο (256 nodes).

Ωστόσο, **96.2% recall με $T=20$** είναι εξαιρετική επίδοση και δείχνει υψηλή σταθερότητα του συστήματος.

Για **QPS = 1376** η ταχύτητα είναι **εντυπωσιακή** αφού είναι περίπου **3.2× πιο γρήγορο** από το πείραμα 3 (≈ 430 QPS). Είναι επίσης **πολύ ταχύτερο** από κάθε προηγούμενο πείραμα.

Η πηγή της μεγάλης ταχύτητας είναι:

- το **μικρό T** ,
- η καλή γεωμετρία των SIFT blocks,
- το ότι το MLP, αν και βαθύτερο, έχει μόνο 256 nodes $\rightarrow$ φθηνότερο inference.

Για **tApprox = 0.00073 ms** αυτός ο χρόνος είναι **3× πιο γρήγορος** από το πείραμα 3 (0.0023 ms) και **$\sim 8\times$ πιο γρήγορος** από το true distance.

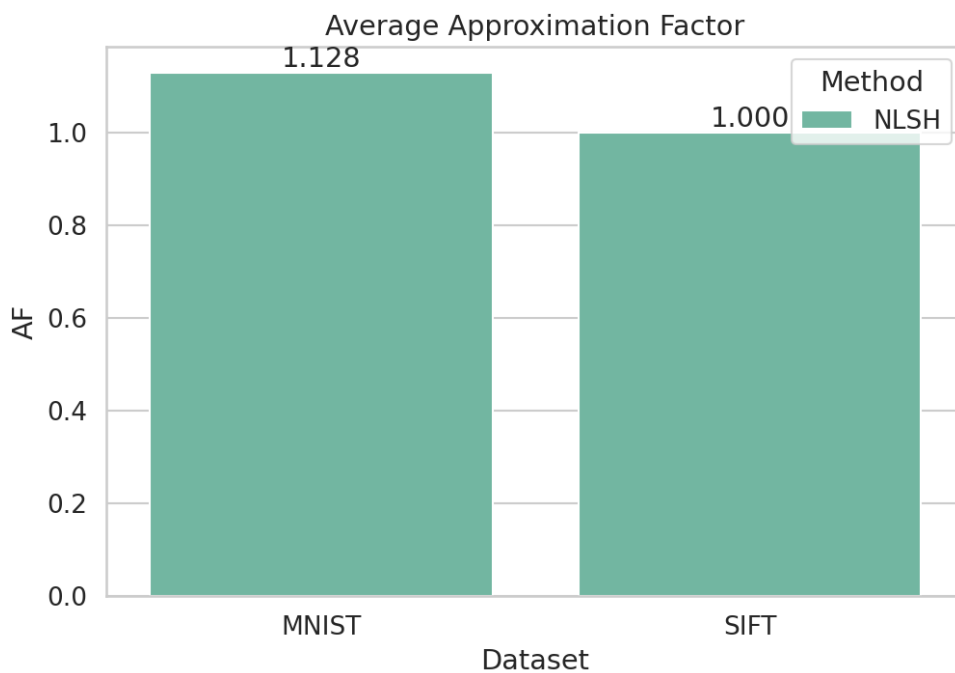
Ουσιαστικά, η αναζήτηση είναι “σχεδόν δωρεάν”, κάτι που υποδεικνύει ότι:

- το δίκτυο ταξινομεί με πολύ υψηλή αυτοπεποίθηση,

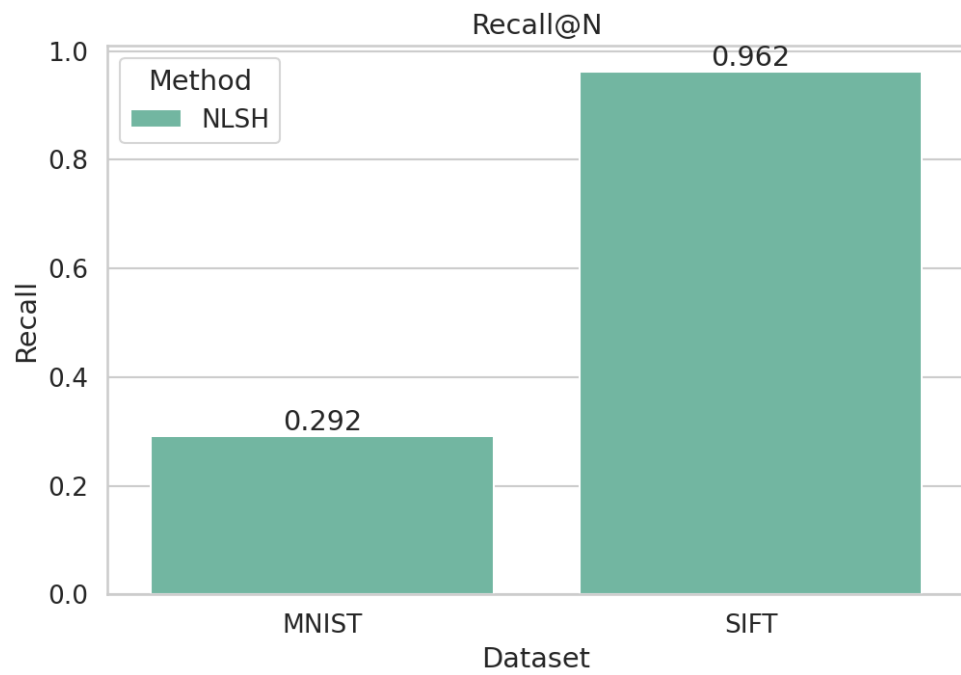
- το σωστό block βρίσκεται σχεδόν πάντα χωρίς να χρειάζεται εκτεταμένο probing.

Συμπερασματικά, ακόμη και ένα **βαθύ MLP με χαμηλό lr** αποδίδει πολύ καλά, το dataset έχει πολύ καθαρή δομή που το KaHIP partitioning μπορεί να εκμεταλλευτεί και το $T=20$ είναι αρκετό για υψηλό recall. Άρα, **το SIFT είναι πολύ πιο φιλικό για NLSH από το MNIST**. Το δίκτυο δεν χρειάζεται να είναι φαρδύ για SIFT — δουλεύει και με 256 nodes.

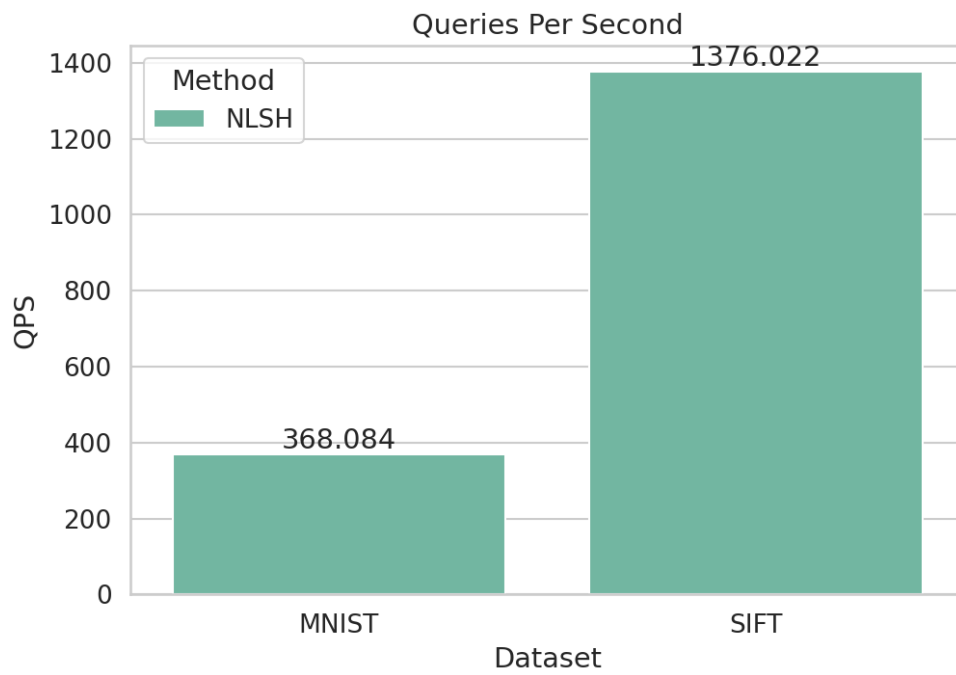
Παρακάτω δίνονται και διαγράμματα για καλύτερη οπτικοποίηση των διαφορών μεταξύ MNIST και SIFT:



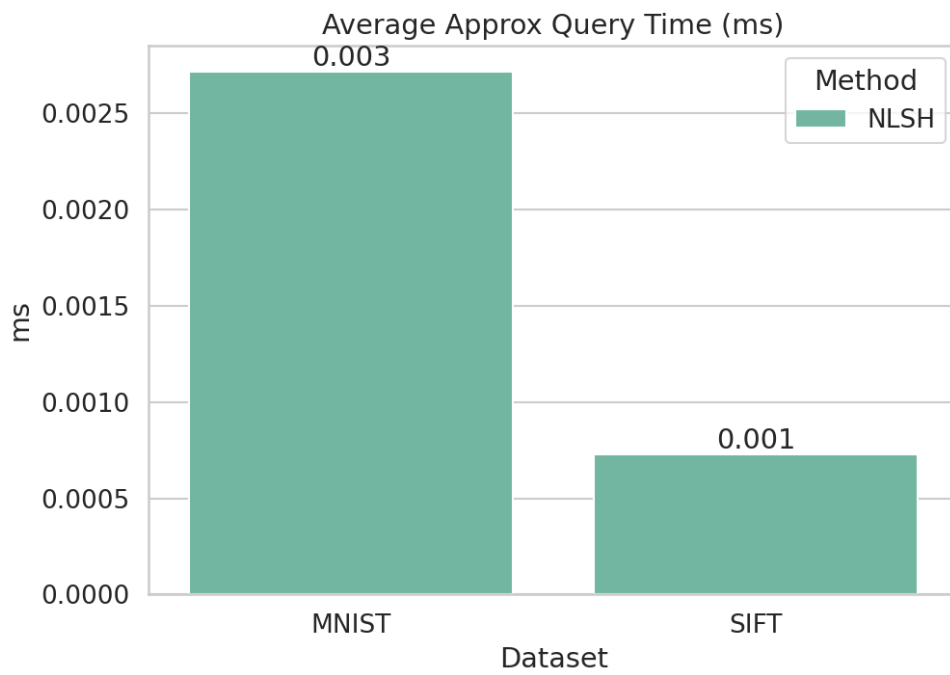
Εικόνα 23: Διάγραμμα $AF^{4^{ov}}$ Πειράματος



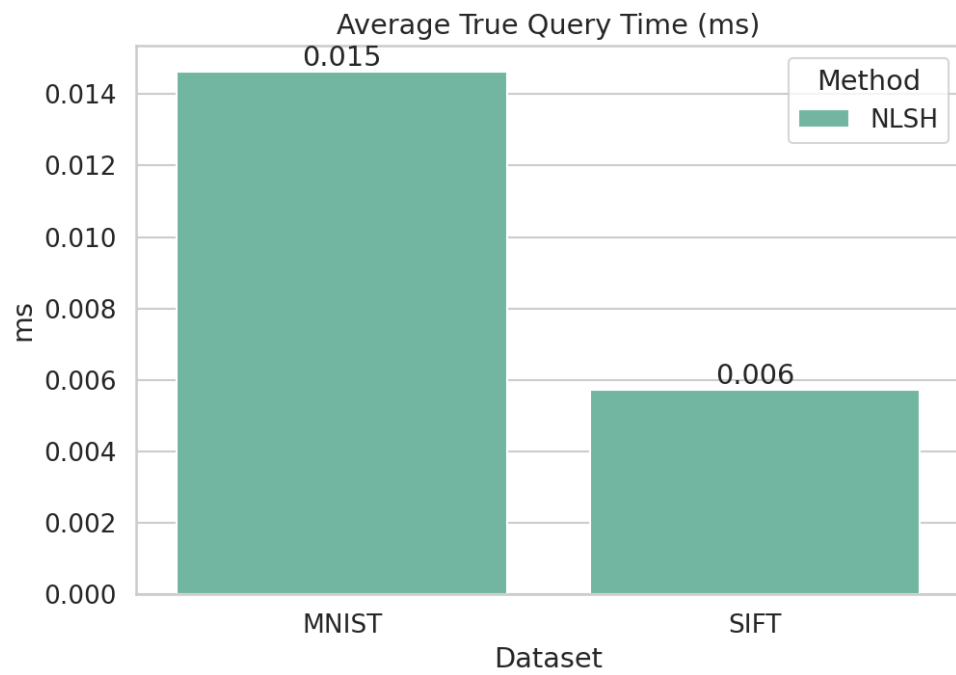
Εικόνα 24: Διάγραμμα Recall 4^{ov} Πειράματος



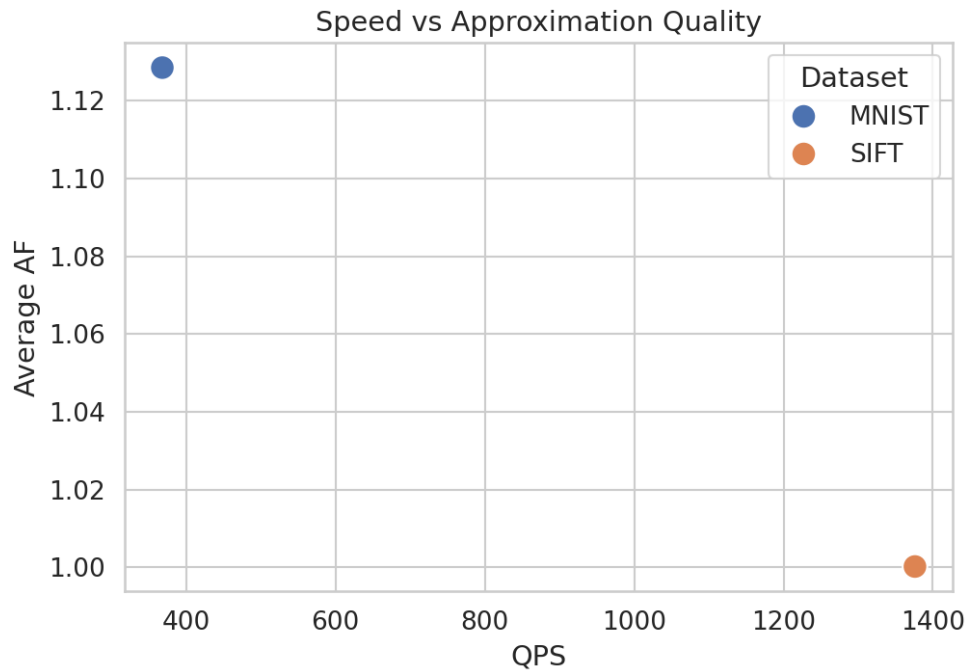
Εικόνα 25: Διάγραμμα QPS 4^{ov} Πειράματος



Εικόνα 26: Διάγραμμα $t_{Approximate}$ 4^{ου} Πειράματος



Εικόνα 27: Διάγραμμα $t_{True}^{4^{ov}}$ Πειράματος



Εικόνα 28: Διάγραμμα Trade-off 4^{ου} Πειράματος

5^ο Πείραμα

Σε αυτό το πείραμα χρησιμοποιήθηκαν οι παρακάτω παράμετροι προκειμένου να έχουμε λιγότερες κλάσεις (50), ευκολότερο classification αλλά μεγαλύτερα blocks (περισσότερο scanning).

nlsh_build.py (Κατασκευή Ευρετηρίου)

- **knn = 20** (Ο αριθμός γειτόνων k για την κατασκευή του k -NN γράφου)
- **m = 50** (Ο αριθμός των μερών (blocks/parts) m για το KaHIP)
- **imbalance = 0.03** (Το ποσοστό ανισορροπίας ϵ για το KaHIP)

- **kahip_mode = 2** (Η ρύθμιση του KaHIP (0=FAST, 1=ECO, 2=STRONG))
- **layers = 4** (Αριθμός στρωμάτων MLP)
- **nodes = 128** (Αριθμός κόμβων στρώματος)
- **epochs = 50** (Περίοδοι εκπαίδευσης του MLP)
- **batch_size = 128** (Μέγεθος δέσμης)
- **lr = 0.0005** (Ρυθμός εκμάθησης)
- **seed = 1** (Φύτρο ψευδοτυχαίων)

nlsb_search.py (Αναζήτηση)

- **N = 10** (Αριθμός πλησιέστερων γειτόνων)
- **R = 2000.0 / 2800.0** (Ακτίνα αναζήτησης - 2000 για MNIST, 2800 για SIFT)
- **T = 10** (Αριθμός μερών (bins) προς έλεγχο (multi-probe))
- **range = true / false** (Εάν θα εκτελεστεί range search)

	A	B	C	D	E	F	G
1	Dataset	Method	Average AF	Recall@N	QPS	tApproximateA	tTrueAverage
2	MNIST	NLSH	1.0479	0.5690	397.8941	0.0025	0.0150
3	SIFT	NLSH	1	0.9550	1,513.2083	0.0007	0.0050

Εικόνα 29: Αποτελέσματα 5^{ου} Πειράματος

Σύμφωνα λοιπόν, με τον πίνακα της **Εικόνας 29** (Αποτελέσματα 5^{ου} Πειράματος):

MNIST

Το **AF = 1.0479** υποδηλώνει ότι η μέση απόσταση των προσεγγιστικών γειτόνων είναι περίπου **4.8% μεγαλύτερη** από την πραγματική. Η τιμή είναι καλύτερη από άλλα “δύσκολα” πειράματα, όμως παραμένει σχετικά υψηλή για MNIST, δείχνοντας ότι το hashing δεν καταφέρνει να ευθυγραμμιστεί πλήρως με τα partitions του KaHIP.

Το **Recall@ = 0.569** είναι χαμηλό, δηλαδή το σύστημα εντοπίζει μόνο το ~57% των πραγματικών 10 κοντινότερων γειτόνων. Η τιμή αυτή επιβεβαιώνει ότι η μείωση των κλάσεων σε 50 και τα μεγαλύτερα blocks οδηγούν σε μεγαλύτερο scanning, αλλά **δεν βελτιώνουν τελικά την ποιότητα ανάκτησης** στο MNIST. Το MLP πιθανόν δυσκολεύεται να μάθει αποτελεσματικά τα όρια των πιο μεγάλων partitions.

Το **QPS $\approx$ 398** είναι ικανοποιητικό, αλλά όχι εξαιρετικό — δείχνει ότι το scanning overhead δεν επιβαρύνει υπερβολικά την ταχύτητα, όμως η ταχύτητα δεν αντισταθμίζει την πτώση στην ποιότητα.

Συνολικά, στο MNIST το πείραμα δείχνει ότι η επιλογή **μικρότερου αριθμού κλάσεων (50)** και συνεπώς **μεγαλύτερων blocks** δεν βοηθά το NLSH να αποδώσει καλύτερα, παρά την πιο απλή ταξινόμηση. Η μάθηση των boundaries παραμένει ανεπαρκής και οδηγεί σε υποδεέστερη ανάκτηση.

SIFT

Τα αποτελέσματα στο SIFT είναι εντυπωσιακά καλύτερα και δείχνουν ότι η παραμετροποίηση αυτή ταιριάζει πολύ περισσότερο στον συγκεκριμένο χώρο χαρακτηριστικών.

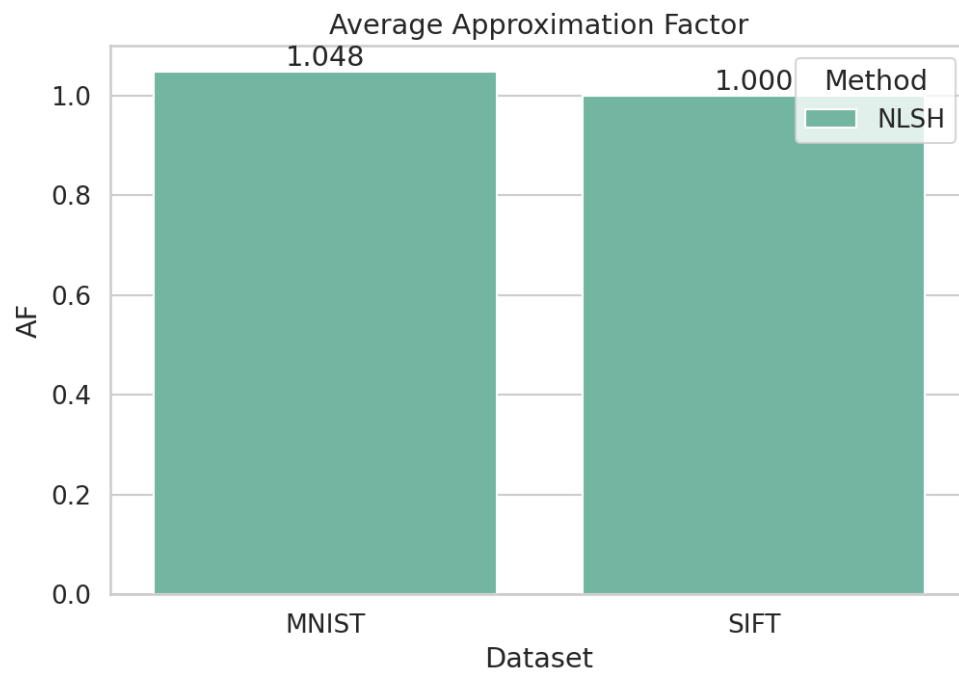
Το $AF = 1.0$ δείχνει πρακτικά **μηδενική απώλεια ποιότητας**, δηλαδή οι προσεγγιστικοί γείτονες έχουν μέση απόσταση σχεδόν ταυτόσημη με τους πραγματικούς. Αυτό σημαίνει ότι το hashing έχει ευθυγραμμιστεί **εξαιρετικά καλά** με τα partitions που δημιούργησε το KaHIP.

Το $Recall@ = 0.955$ είναι πολύ υψηλό, υποδεικνύοντας ότι το σύστημα ανακτά το 95.5% των πραγματικών 10 κοντινότερων γειτόνων. Η τιμή αυτή είναι από τις καλύτερες που μπορούν να επιτευχθούν στο SIFT, και δείχνει ότι τα μεγαλύτερα blocks όχι μόνο δεν δυσκολεύουν την αναζήτηση, αλλά δημιουργούν πιο συνεκτικές περιοχές embedding τις οποίες το MLP μπορεί να μάθει αποτελεσματικά.

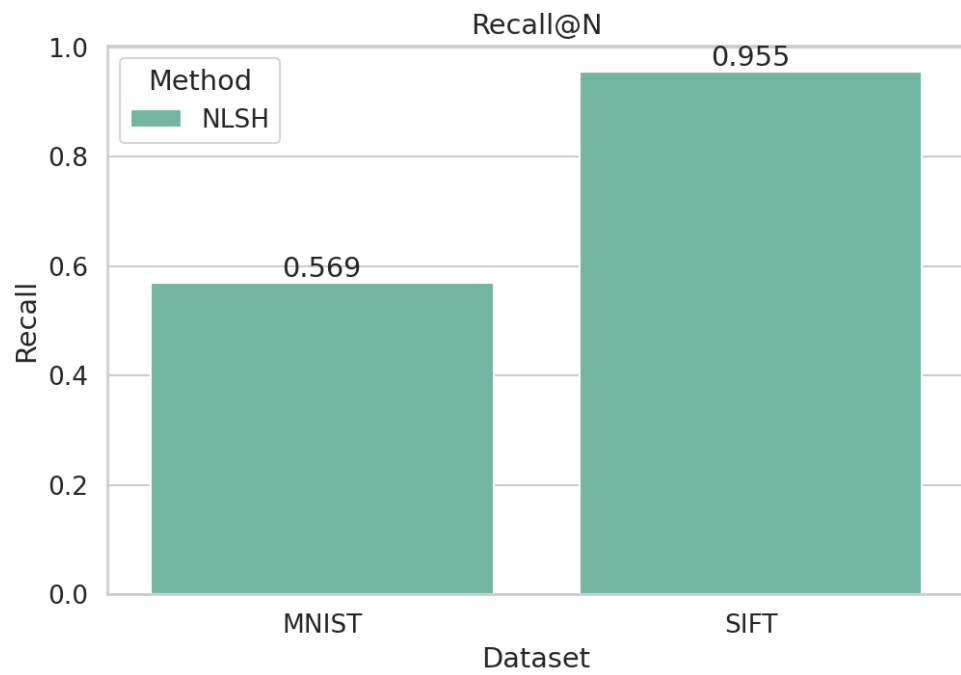
Το $QPS \approx 1513$ είναι εξαιρετικά υψηλό, επιβεβαιώνοντας ότι η αναζήτηση είναι πολύ γρήγορη παρά το υψηλό $T=10$ multi-probing. Το SIFT γενικά ευνοεί το NLSH λόγω της δομής του χώρου και αυτό αντανακλάται ξεκάθαρα στα αποτελέσματα.

Συνολικά, στο SIFT, το πείραμα αυτό επιτυγχάνει **πολύ υψηλή ακρίβεια και πολύ υψηλή ταχύτητα**, κάτι που υποδηλώνει ότι η επιλογή 50 κλάσεων αποτελεί καλή ισορροπία ανάμεσα σε partitioning quality και scanning.

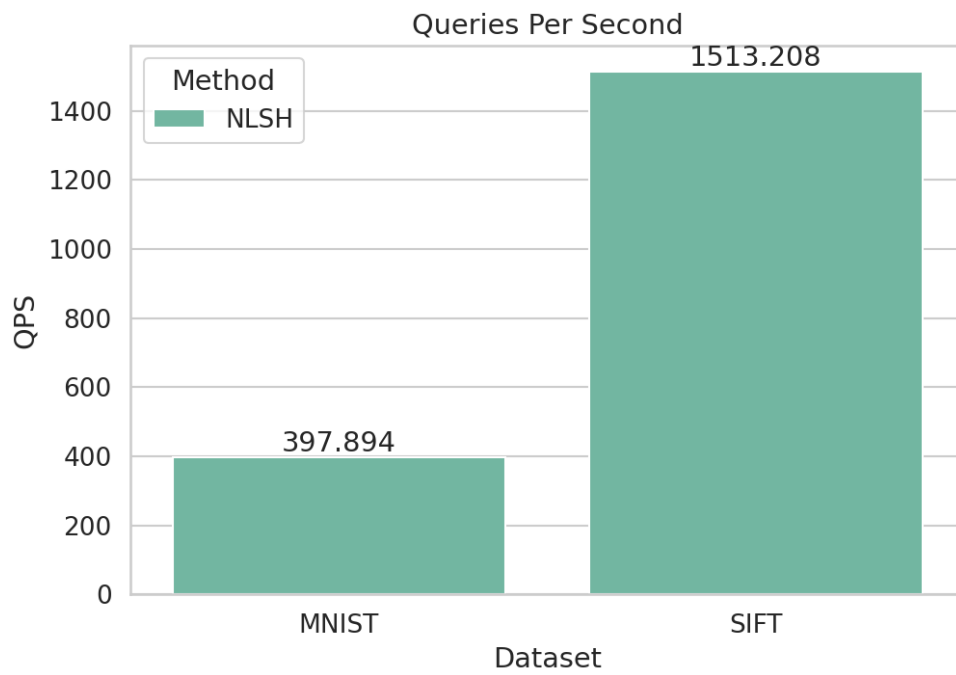
Παρακάτω δίνονται και διαγράμματα για καλύτερη οπτικοποίηση των διαφορών μεταξύ MNIST και SIFT:



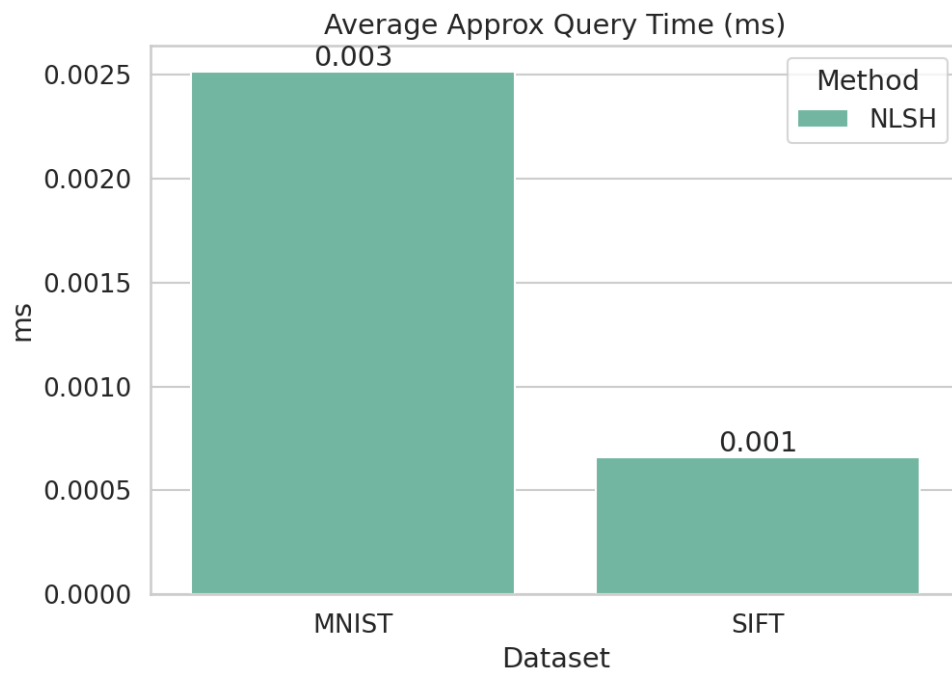
Εικόνα 30: Διάγραμμα AF 5^{ov} Πειράματος



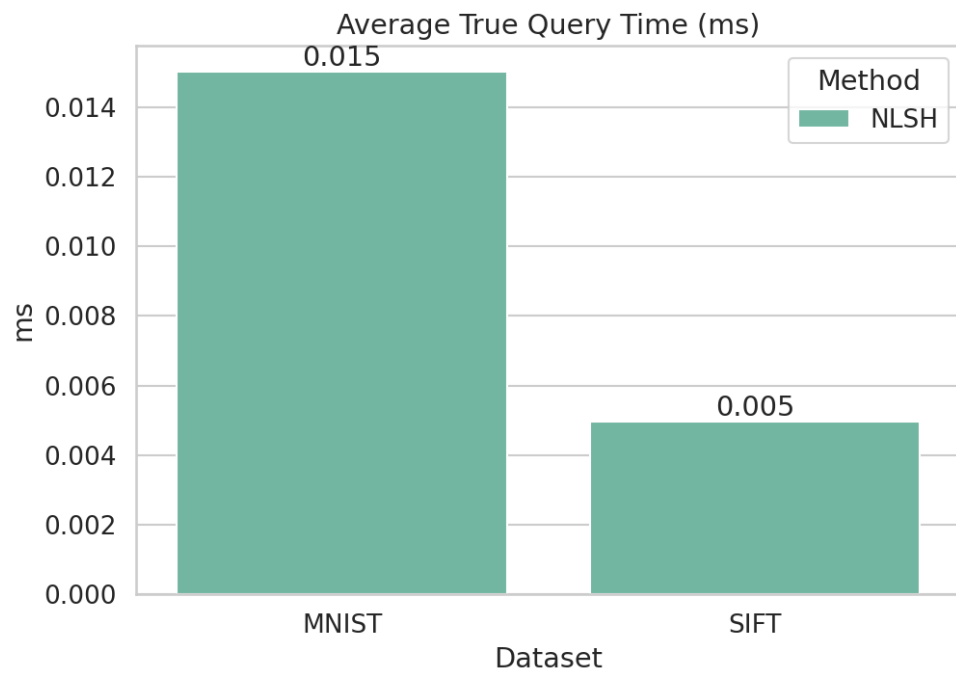
Εικόνα 31: Διάγραμμα Recall 5^{ov} Πειράματος



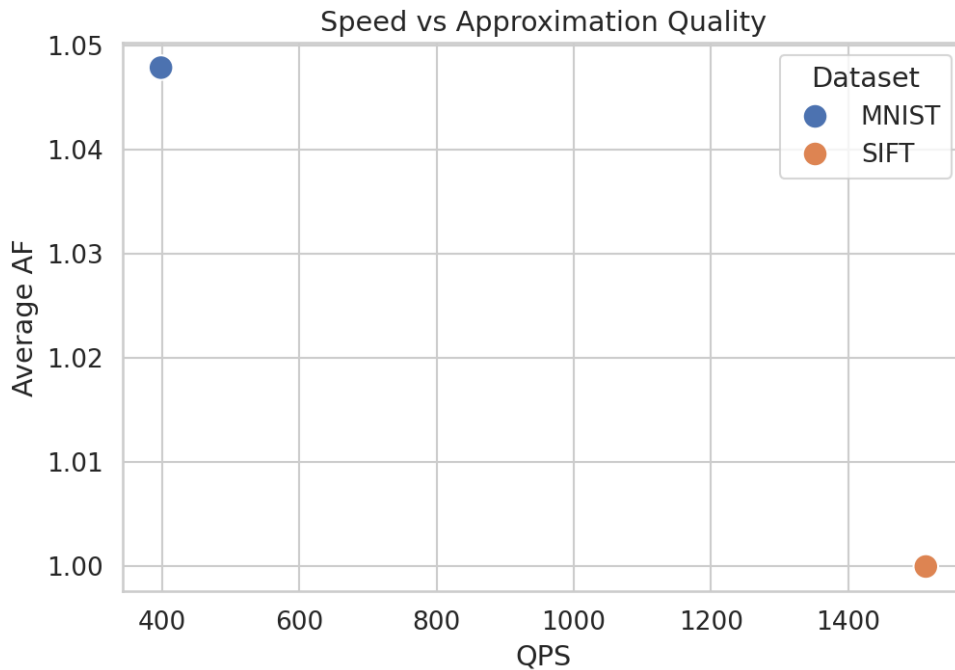
Εικόνα 32: Διάγραμμα QPS 5^{ov} Πειράματος



Εικόνα 33: Διάγραμμα $t_{Approximate}$ 5^{ου} Πειράματος



Εικόνα 34: Διάγραμμα $t_{True}^{5^{ov}}$ Πειράματος



Εικόνα 35: Διάγραμμα Trade-off 5^{ου} Πειράματος

6^ο Πείραμα

Σε αυτό το πείραμα χρησιμοποιήθηκαν οι παρακάτω παράμετροι προκειμένου να δοκιμάσουμε πιο λεπτή διαμέριση — θέλουμε να δούμε αν multi-probe + ισχυρότερο MLP πιάνουν.

nlsh build.py (Κατασκευή Ευρετηρίου)

- **knn = 20** (Ο αριθμός γειτόνων k για την κατασκευή του k -NN γράφου)
- **m = 200** (Ο αριθμός των μερών (blocks/parts) m για το KaHIP)
- **imbalance = 0.02** (Το ποσοστό ανισορροπίας ϵ για το KaHIP)
- **kahip_mode = 2** (Η ρύθμιση του KaHIP (0=FAST, 1=ECO, 2=STRONG))

- **layers = 4** (Αριθμός στρωμάτων MLP)
- **nodes = 256** (Αριθμός κόμβων στρώματος)
- **epochs = 50** (Περίοδοι εκπαίδευσης του MLP)
- **batch_size = 128** (Μέγεθος δέσμης)
- **lr = 0.0005** (Ρυθμός εκμάθησης)
- **seed = 1** (Φύτρο ψευδοτυχαίων)

nlsb_search.py (Αναζήτηση)

- **N = 10** (Αριθμός πλησιέστερων γειτόνων)
- **R = 2000.0 / 2800.0** (Ακτίνα αναζήτησης - 2000 για MNIST, 2800 για SIFT)
- **T = 50** (Αριθμός μερών (bins) προς έλεγχο (multi-probe))
- **range = true / false** (Εάν θα εκτελεστεί range search)

	A	B	C	D	E	F	G
1	Dataset	Method	Average AF	Recall@N	QPS	tApproximateA	tTrueAverage
2	MNIST	NLSH	1.1663	0.3160	146.8943	0.0068	0.0206
3	SIFT	NLSH	1	0.9830	916.8825	0.0011	0.0059

Εικόνα 36: Αποτελέσματα 6^{ου} Πειράματος

Σύμφωνα λοιπόν, με τον πίνακα της **Εικόνας 36** (Αποτελέσματα 6^{ου} Πειράματος):

MNIST

Τα αποτελέσματα στο MNIST είναι αρκετά αδύναμα, γεγονός που επιβεβαιώνει ότι η αύξηση του αριθμού των partitions ($m = 200$) και η χαμηλή ανισορροπία ($\epsilon = 0.02$) δυσκολεύουν σημαντικά το MLP να μάθει αποτελεσματικά τα όρια που ορίζει το KaHIP.

Το **AF = 1.1663** δείχνει ότι η μέση απόσταση των προσεγγιστικών γειτόνων είναι περίπου **16.6% μεγαλύτερη** από την πραγματική. Πρόκειται για μια από τις υψηλότερες τιμές AF, γεγονός που υποδηλώνει **αδυναμία του συστήματος να επιτύχει ποιοτικό hashing** σε τόσο λεπτομερή partitioning. Με άλλα λόγια, το δίκτυο δεν καταφέρνει να προσεγγίσει ικανοποιητικά τη γεωμετρία του χώρου.

Το **Recall@ = 0.316** είναι επίσης χαμηλό. Με λιγότερο από το 32% των πραγματικών κοντινών γειτόνων να ανακτώνται, γίνεται σαφές ότι το σύστημα χάνει σημαντική πληροφορία, πιθανότατα λόγω της υπερβολικής κατακερματισμένης δομής (200 blocks). Οι περισσότερες πραγματικές κοντινές σημεία βρίσκονται σε διαφορετικά partitions από αυτά που προβλέπει το MLP.

Το **QPS ≈ 147** δείχνει ότι η απόδοση ταχύτητας πέφτει αισθητά. Το μεγάλο $T = 50$ αυξάνει σημαντικά το scanning, ενώ η ακρίβεια παραμένει χαμηλή — δηλαδή **η αναζήτηση είναι και αργή και ανακριβής**.

Συνολικά, για το MNIST, η παραμετροποίηση αυτή δεν αποδίδει: τα πάρα πολλά blocks και η αυστηρή ανισορροπία οδηγούν σε **υπερβολικά λεπτό partitioning** που το MLP δυσκολεύεται να μάθει, με αποτέλεσμα χαμηλή ποιότητα και μέτρια ταχύτητα.

SIFT

Τα αποτελέσματα στο SIFT είναι αντίθετα αρκετά καλά, κάτι που συμφωνεί με την παρατήρηση ότι ο χώρος του SIFT είναι πιο “φιλικός” για NLSH με λεπτό partitioning.

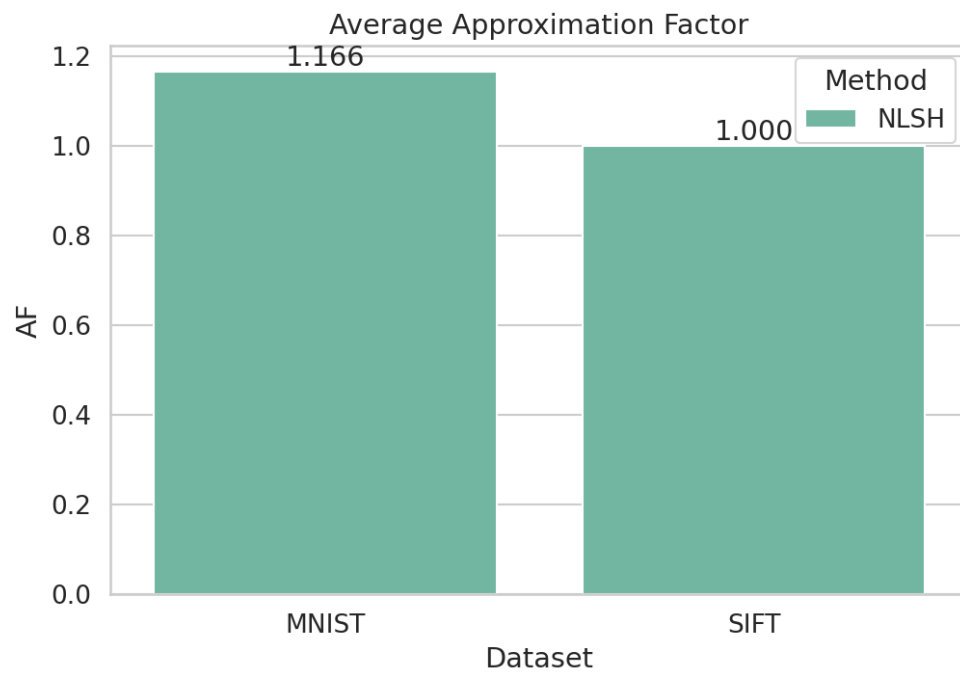
Το $AF = 1.0$ υποδηλώνει ουσιαστικά **μηδενική απώλεια ποιότητας**, δηλαδή οι προσεγγιστικοί γείτονες έχουν μέση απόσταση πρακτικά ίση με τους πραγματικούς. Αυτό δείχνει ότι το σύστημα καταφέρνει να μάθει τις αποφάσεις του KaHIP ακόμη και με 200 blocks.

Το $Recall@10 = 0.983$ είναι εξαιρετικά υψηλό. Το σύστημα ανακτά το 98.3% των πραγματικών 10 κοντινότερων γειτόνων, γεγονός που αποτελεί πολύ ισχυρή ένδειξη ότι το MLP ευθυγραμμίζεται αποτελεσματικά με τη δομή των partitions στο SIFT. Τα πολλά blocks δεν δυσκολεύουν την ανάκτηση — αντίθετα, ίσως ευνοούν λεπτομερέστερη γεωμετρική οργάνωση.

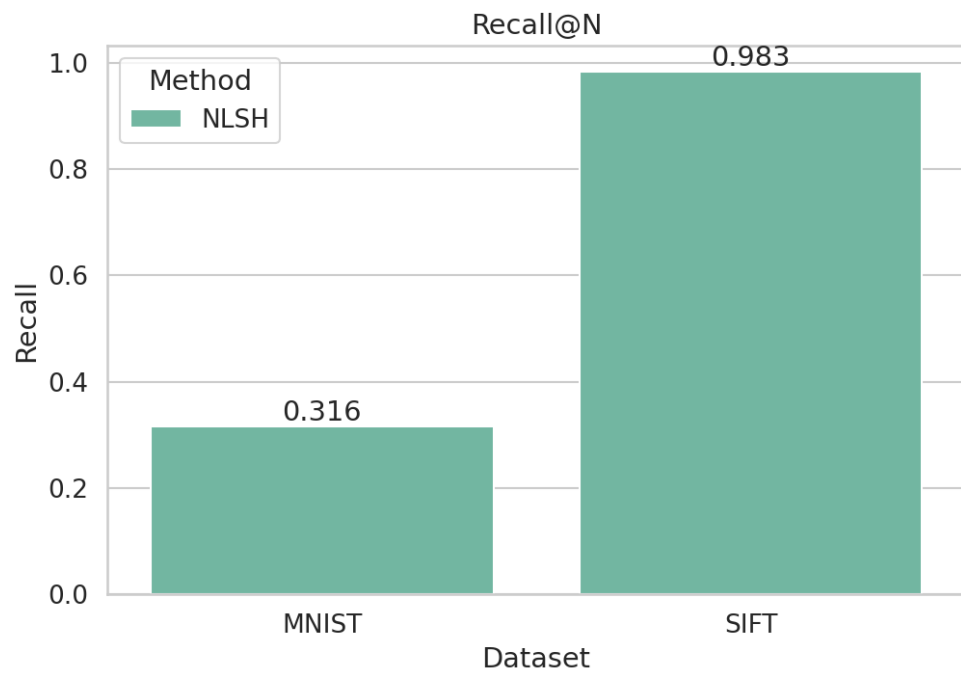
Το $QPS \approx 917$ είναι επίσης καλό, αλλά χαμηλότερο από άλλα πειράματα με λιγότερο scanning. Το υψηλό $T = 50$ αυξάνει το κόστος αναζήτησης, όμως η ταχύτητα παραμένει σε αποδεκτά επίπεδα και είναι συμβατή με την εξαιρετική ποιότητα.

Συνολικά, στο SIFT, το πείραμα αυτό αποδίδει **πολύ υψηλή ακρίβεια και καλή ταχύτητα**, με τη λεπτότερη διαμέριση ($m = 200$) να λειτουργεί θετικά για την ποιότητα hashing.

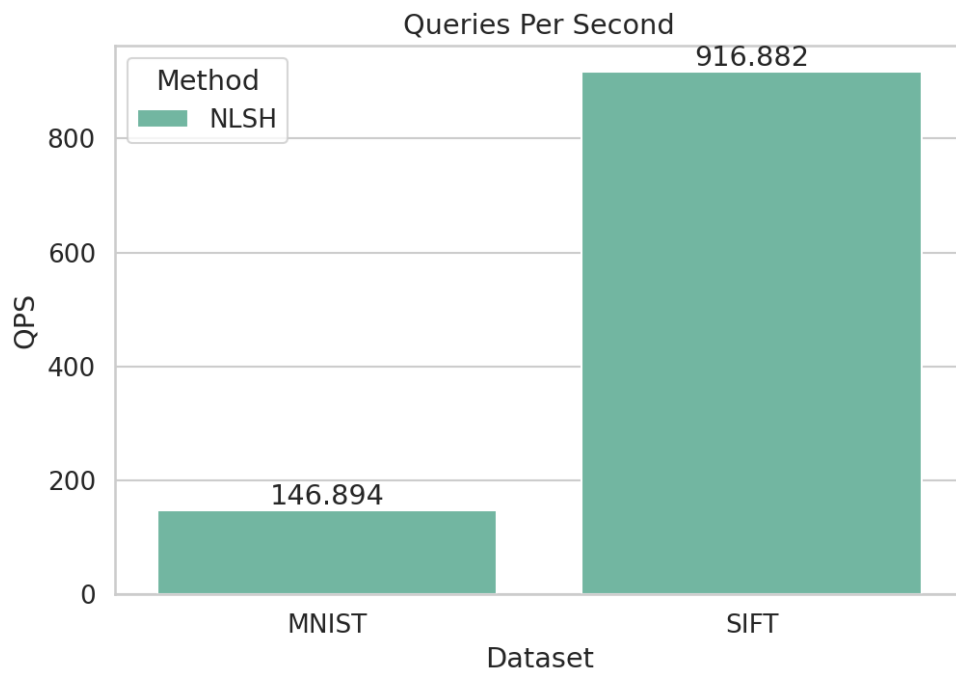
Παρακάτω δίνονται και διαγράμματα για καλύτερη οπτικοποίηση των διαφορών μεταξύ MNIST και SIFT:



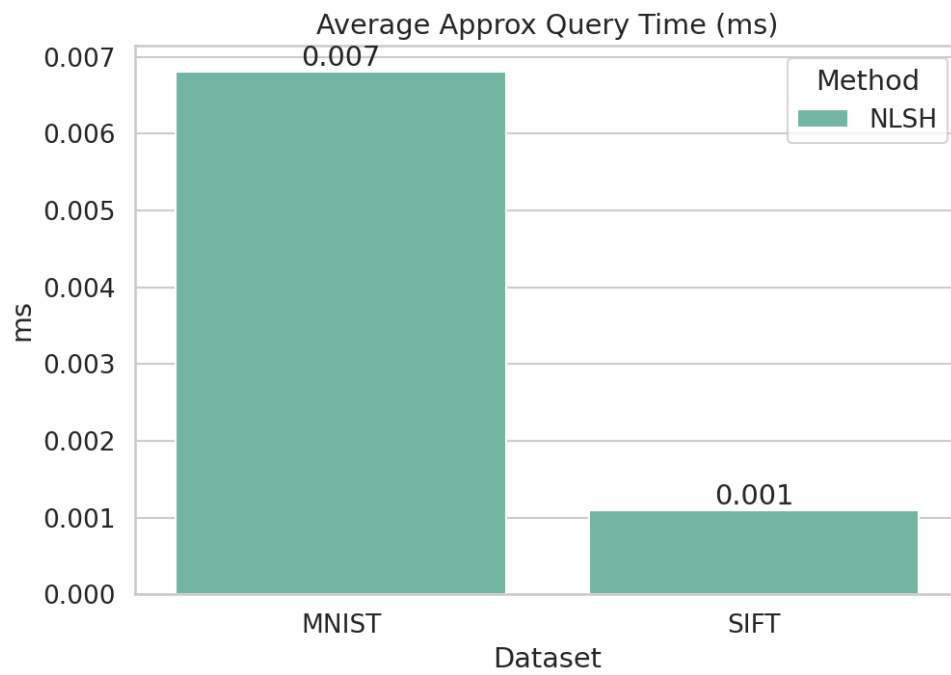
Εικόνα 37: Διάγραμμα AF 6^{ov} Πειράματος



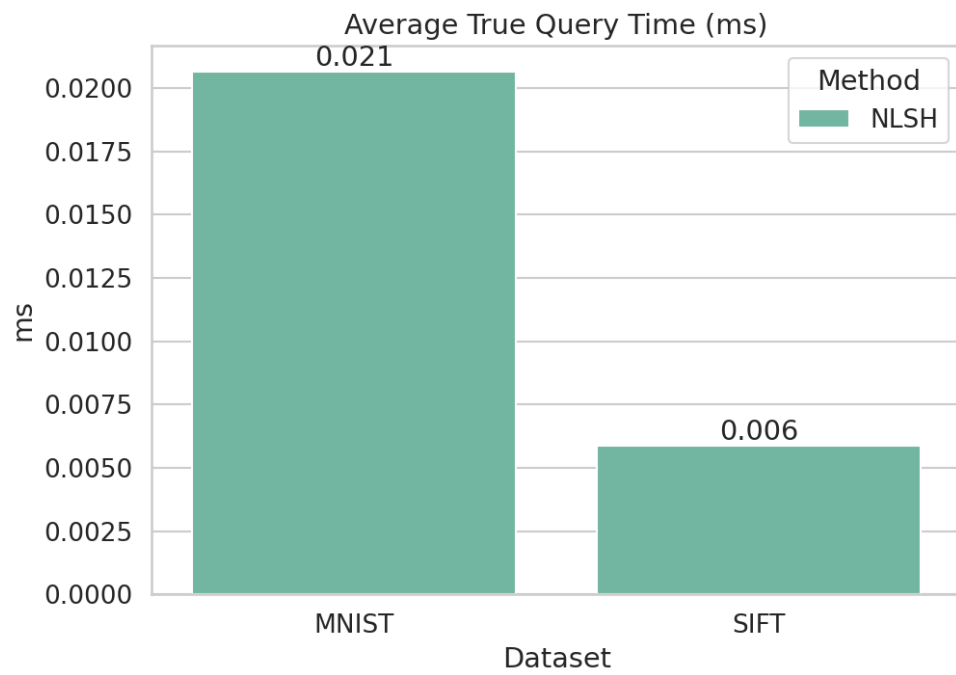
Εικόνα 38: Διάγραμμα Recall 6^{ου} Πειράματος



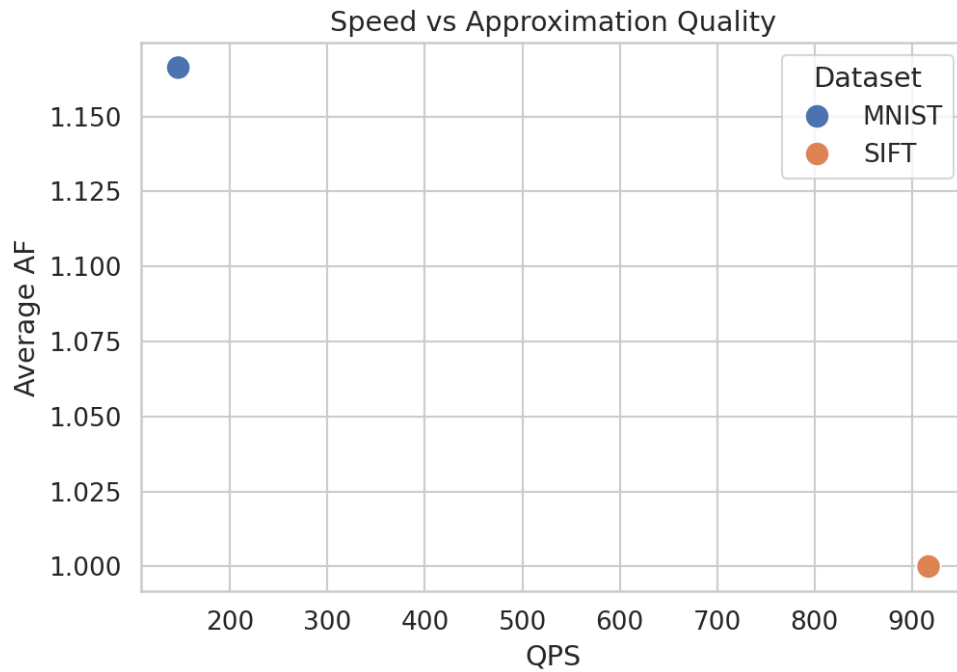
Εικόνα 39: Διάγραμμα QPS 6^{ου} Πειράματος



Εικόνα 40: Διάγραμμα $t_{Approximate}$ 6^{ου} Πειράματος



Εικόνα 41: Διάγραμμα t_{True} 6^{ov} Πειράματος



Εικόνα 42: Διάγραμμα Trade-off 6^{ου} Πειράματος

7^ο Πείραμα

Σε αυτό το πείραμα χρησιμοποιήθηκαν οι παρακάτω παράμετροι προκειμένου να ελέγξουμε την ευαισθησία στο k .

nlsh_build.py (Κατασκευή Ευρετηρίου)

- **knn = 5** (Ο αριθμός γειτόνων k για την κατασκευή του k -NN γράφου)
- **m = 100** (Ο αριθμός των μερών (blocks/parts) m για το KaHIP)
- **imbalance = 0.04** (Το ποσοστό ανισορροπίας ϵ για το KaHIP)
- **kahip_mode = 0** (Η ρύθμιση του KaHIP (0=FAST, 1=ECO, 2=STRONG))

- **layers = 4** (Αριθμός στρωμάτων MLP)
- **nodes = 128** (Αριθμός κόμβων στρώματος)
- **epochs = 50** (Περίοδοι εκπαίδευσης του MLP)
- **batch_size = 128** (Μέγεθος δέσμης)
- **lr = 0.0005** (Ρυθμός εκμάθησης)
- **seed = 1** (Φύτρο ψευδοτυχαίων)

nlsb_search.py (Αναζήτηση)

- **N = 10** (Αριθμός πλησιέστερων γειτόνων)
- **R = 2000.0 / 2800.0** (Ακτίνα αναζήτησης - 2000 για MNIST, 2800 για SIFT)
- **T = 30** (Αριθμός μερών (bins) προς έλεγχο (multi-probe))
- **range = true / false** (Εάν θα εκτελεστεί range search)

	A	B	C	D	E	F	G
1	Dataset	Method	Average AF	Recall@N	QPS	tApproximateA	tTrueAverage
2	MNIST	NLSH	1.0008	0.3130	196.6428	0.0051	0.0195
3	SIFT	NLSH	1	0.9860	959.1839	0.0010	0.0072

Εικόνα 43: Αποτελέσματα 7^{ου} Πειράματος

Σύμφωνα λοιπόν, με τον πίνακα της **Εικόνας 43** (Αποτελέσματα 7^{ου} Πειράματος):

MNIST

Τα αποτελέσματα στο MNIST δείχνουν ότι η επιλογή **μικρού $k = 5$** στον k -NN γράφο επηρεάζει αρνητικά την ποιότητα του partitioning. Με τόσο μικρή συνδεσιμότητα, το KaHIP έχει πολύ περιορισμένη πληροφορία για τη γεωμετρία των τοπικών γειτονιών, άρα δημιουργεί partitions λιγότερο συνεκτικά και συχνά “τεχνητά”.

Το **AF = 1.00075** είναι εντυπωσιακά χαμηλό· η μέση απόσταση είναι μόλις **0.07% μεγαλύτερη** από την πραγματική. Αυτό δείχνει ότι το σύστημα επιστρέφει πολύ κοντινούς γείτονες και ότι οι πραγματικές αποστάσεις των υποψήφιων σημείων είναι πολύ κοντά στο optimum. Ωστόσο, το AF μόνο δεν αρκεί για πλήρη εικόνα.

Το **Recall@10 = 0.313** είναι αρκετά χαμηλό, καθώς ανακτάται μόλις το 31% των πραγματικών γειτόνων. Αυτή η απόκλιση ανάμεσα σε AF και Recall εξηγείται εύκολα:

- Με **$k = 5$** , οι συνδέσεις του γράφου είναι πολύ φτωχές.
- Πολλά πραγματικά κοντινά σημεία δεν καταλήγουν στο ίδιο block, άρα το MLP—παρότι μαθαίνει σωστά τα boundaries—δεν μπορεί να “προβλέψει” σωστά τη γειτονιά.
- Έτσι, το σύστημα βρίσκει πολύ κοντινούς γείτονες, αλλά **χάνει τους περισσότερους πραγματικούς**.

Το **QPS ≈ 197** είναι σχετικά μέτριο: καλύτερο από πειράματα με πολύ υψηλό T , αλλά όχι εξαιρετικό. Το $T = 30$ βοηθά ώστε να εξεταστούν αρκετά bins, όμως το περιορισμένο k στις αρχικές γειτονιές εξακολουθεί να περιορίζει την ανάκτηση.

Συνολικά, για το MNIST, η δοκιμή αυτή δείχνει ότι **η χαμηλή τιμή k μειώνει τον βαθμό συνοχής του k -NN graph**, οδηγώντας σε partitions που δεν αντικατοπτρίζουν σωστά τη δομή των δεδομένων. Παρά το πολύ καλό AF, το σύστημα έχει χαμηλό Recall και συνεπώς περιορισμένη συνολική ποιότητα.

SIFT

Τα αποτελέσματα στο SIFT είναι πολύ καλύτερα και δείχνουν ότι το dataset είναι λιγότερο ευαίσθητο στη μείωση του k . Οι τοπικές γειτονιές στα SIFT descriptors έχουν πιο σταθερή και “καθαρή” γεωμετρία, οπότε ακόμη και με $k = 5$ ο γράφος παραμένει αρκετά αξιόπιστος.

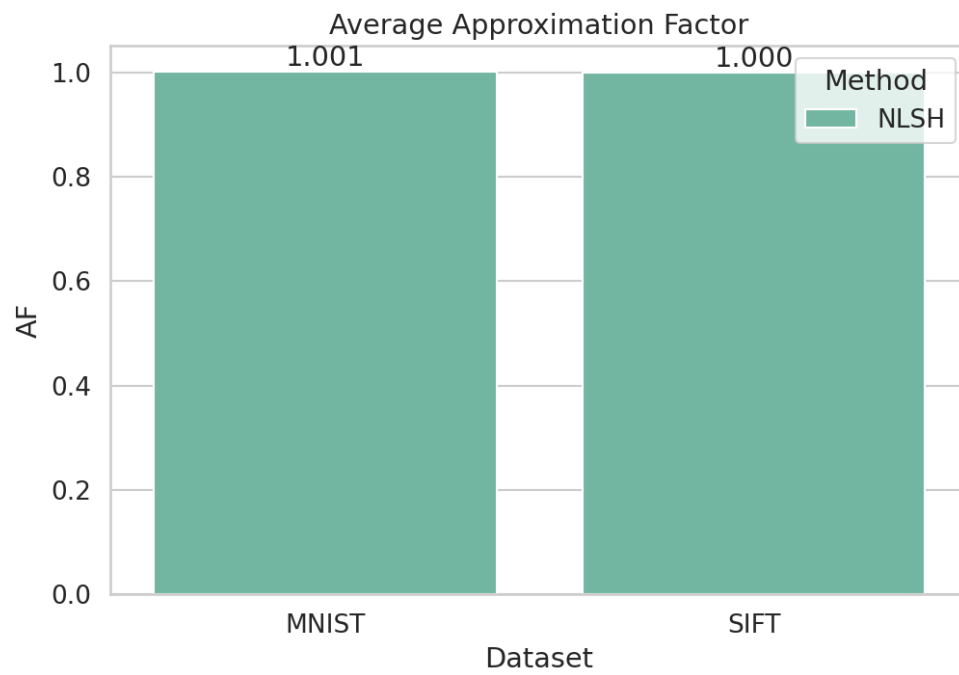
Το **AF = 1.0** σημαίνει μηδενική κατά μέσο όρο απώλεια ποιότητας: οι προσεγγιστικοί γείτονες είναι στην πράξη όσο κοντινοί όσο και οι αληθινοί. Αυτό αποτελεί εξαιρετικό αποτέλεσμα και δείχνει ότι, παρά το μικρό k , τα blocks παραμένουν καλά σχηματισμένα.

Το **Recall@10 = 0.986** είναι επίσης εξαιρετικό. Το σύστημα ανακτά σχεδόν το 99% των πραγματικών κοντινών neighbors, ξεπερνώντας μάλιστα πολλά προηγούμενα πειράματα. Αυτό δείχνει ότι το SIFT dataset παραμένει ανθεκτικό στη μείωση της συνδεσιμότητας του γράφου: οι δηλωμένες γειτονιές επαρκούν για να διατηρήσουν σωστές σχέσεις απόστασης.

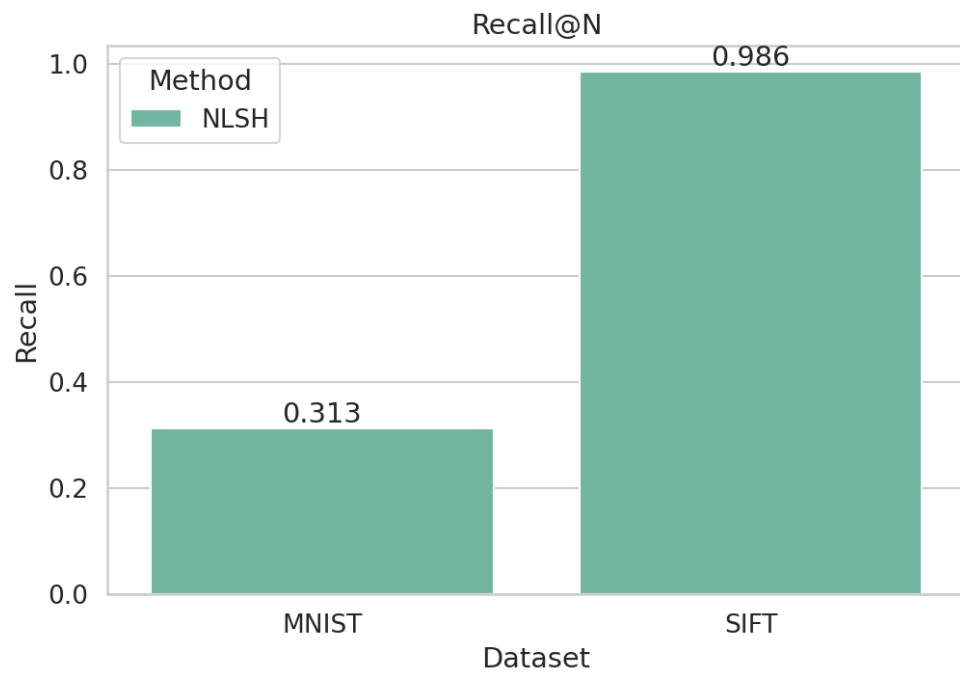
Το **QPS $\approx$ 959** είναι υψηλό και θεωρείται πολύ καλή επίδοση. Το $T = 30$ επιτρέπει επαρκή κάλυψη γειτονικών blocks χωρίς υπερβολικό scanning. Ο συνδυασμός υψηλού Recall και υψηλής ταχύτητας το καθιστά ένα από τα πιο ισορροπημένα πειράματα στο SIFT.

Συνολικά, στο SIFT το πείραμα δείχνει ότι ακόμα και με **πολύ μικρό k** , το NLSH διατηρεί εξαιρετική επίδοση, τόσο από πλευράς ακρίβειας όσο και από πλευράς ταχύτητας. Η χωρική δομή του SIFT φαίνεται να “αντέχει” την ελάττωση της συνδεσιμότητας, σε αντίθεση με το MNIST.

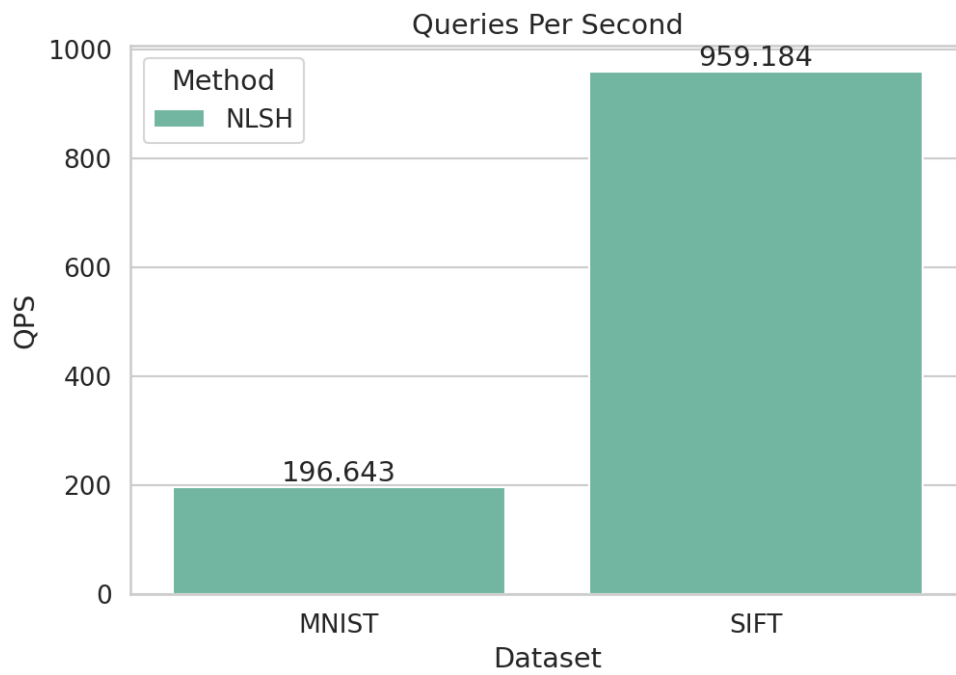
Παρακάτω δίνονται και διαγράμματα για καλύτερη οπτικοποίηση των διαφορών μεταξύ MNIST και SIFT:



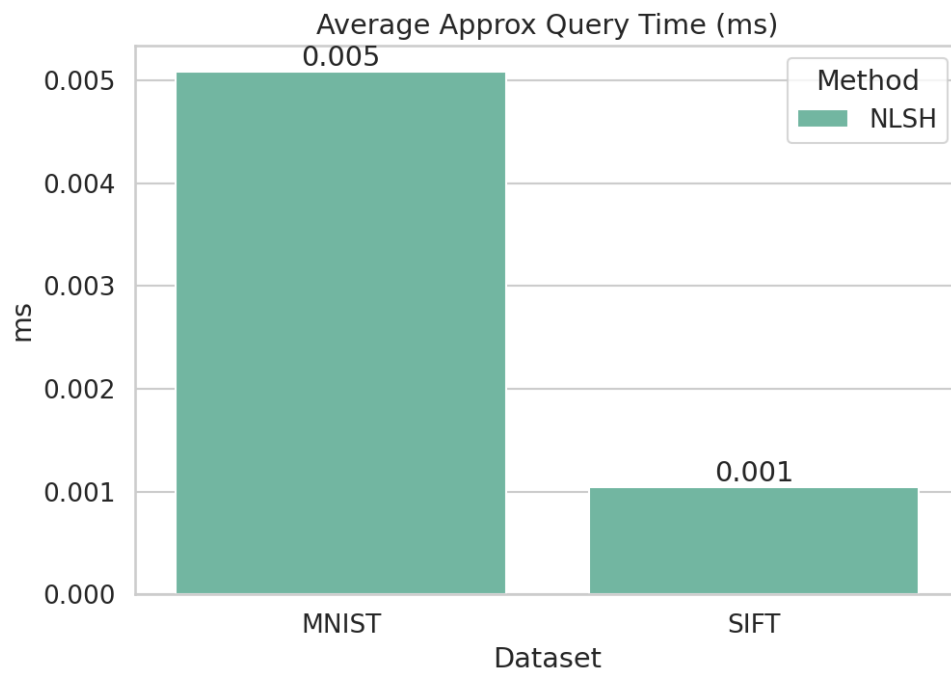
Εικόνα 44: Διάγραμμα $AF^{7^{ov}}$ Πειράματος



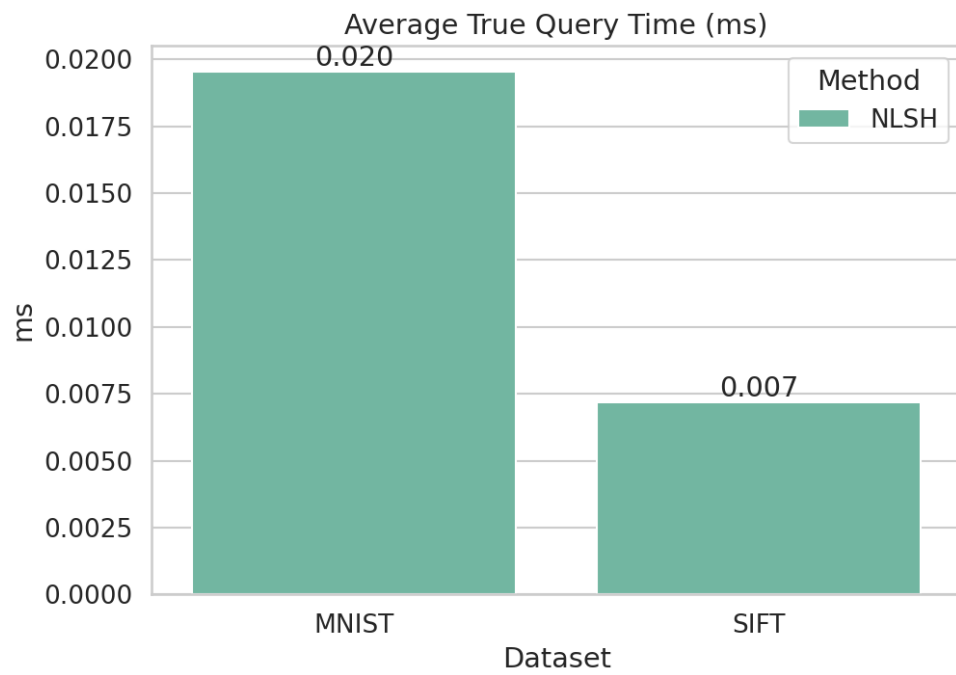
Εικόνα 45: Διάγραμμα Recall 7^{ov} Πειράματος



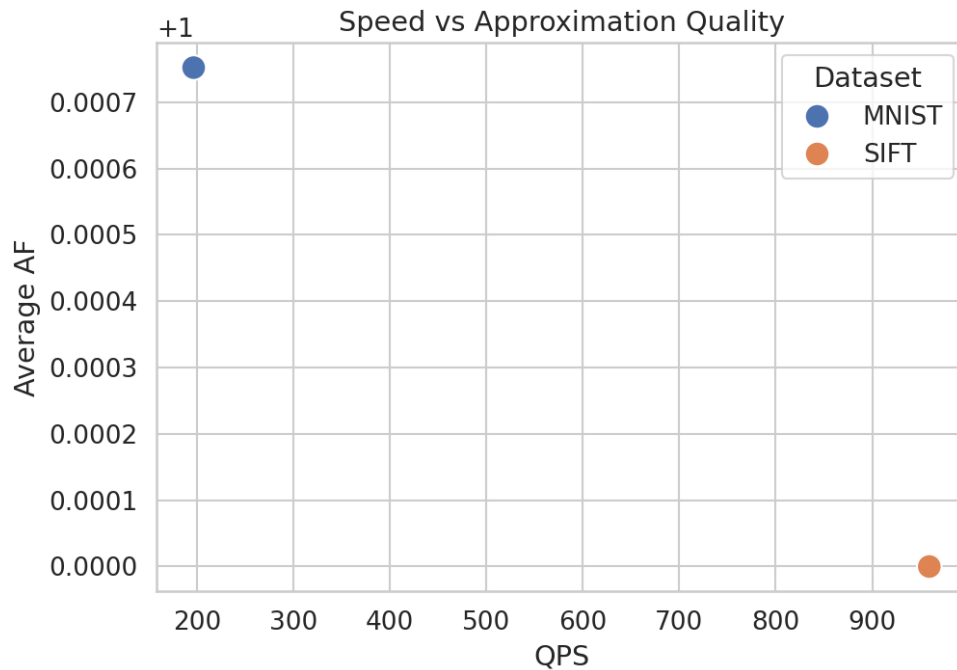
Εικόνα 46: Διάγραμμα QPS 7^{ov} Πειράματος



Εικόνα 47: Διάγραμμα $t_{Approximate}^{7^{ov}}$ Πειράματος



Εικόνα 48: Διάγραμμα t_{True}^{700} Πειράματος



Εικόνα 49: Διάγραμμα Trade-off 7^{ου} Πειράματος

8^ο Πείραμα

Σε αυτό το πείραμα χρησιμοποιήθηκαν οι παρακάτω παράμετροι προκειμένου να έχουμε πυκνότερο γράφο ώστε να δημιουργεί πιο «συνεκτικά» blocks → ευκολότερη εκμάθηση.

nlsh_build.py (Κατασκευή Ευρετηρίου)

- **knn = 50** (Ο αριθμός γειτόνων k για την κατασκευή του k -NN γράφου)
- **m = 100** (Ο αριθμός των μερών (blocks/parts) m για το KaHIP)
- **imbalance = 0.03** (Το ποσοστό ανισορροπίας ϵ για το KaHIP)
- **kahip_mode = 2** (Η ρύθμιση του KaHIP (0=FAST, 1=ECO, 2=STRONG))

- **layers = 4** (Αριθμός στρωμάτων MLP)
- **nodes = 128** (Αριθμός κόμβων στρώματος)
- **epochs = 50** (Περίοδοι εκπαίδευσης του MLP)
- **batch_size = 128** (Μέγεθος δέσμης)
- **lr = 0.0005** (Ρυθμός εκμάθησης)
- **seed = 1** (Φύτρο ψευδοτυχαίων)

nlsb_search.py (Αναζήτηση)

- **N = 10** (Αριθμός πλησιέστερων γειτόνων)
- **R = 2000.0 / 2800.0** (Ακτίνα αναζήτησης - 2000 για MNIST, 2800 για SIFT)
- **T = 20** (Αριθμός μερών (bins) προς έλεγχο (multi-probe))
- **range = true / false** (Εάν θα εκτελεστεί range search)

	A	B	C	D	E	F	G
1	Dataset	Method	Average AF	Recall@N	QPS	tApproximateA	tTrueAverage
2	MNIST	NLSH	1.0692	0.4710	336.2592	0.0030	0.0186
3	SIFT	NLSH	1.0006	0.9540	1,958.8048	0.0005	0.0047

Εικόνα 50: Αποτελέσματα 8^{ου} Πειράματος

Σύμφωνα λοιπόν, με τον πίνακα της **Εικόνας 50** (Αποτελέσματα 8^{ου} Πειράματος):

MNIST

Στο MNIST, η επιλογή μεγαλύτερου $k = 50$ σε σχέση με προηγούμενα πειράματα οδήγησε πράγματι σε πολύ πυκνότερο k -NN γράφο, γεγονός που βελτιώνει τη συνεκτικότητα του partitioning και επιτρέπει στο MLP να μάθει πιο σταθερά decision boundaries. Παρ' όλα αυτά, τα αποτελέσματα δείχνουν ότι, παρά τη σημαντική βελτίωση του γραφήματος, το MNIST εξακολουθεί να αποτελεί ένα δύσκολο dataset για το Neural LSH.

Το **Average AF = 1.069** υποδηλώνει ότι οι προσεγγιστικοί γείτονες απέχουν κατά μέσο όρο $\sim 6.9\%$ περισσότερο από τους πραγματικούς. Δεν είναι κακό αποτέλεσμα, αλλά είναι εμφανώς υψηλότερο από το ιδανικό $AF \approx 1.0$. Αυτό δείχνει ότι το μοντέλο έχει μάθει μια σχετικά καλή διαμέριση, αλλά όχι αρκετά ακριβή ώστε να εντοπίζει συστηματικά τους βέλτιστους γείτονες.

Το **Recall@N = 0.471** είναι μέτριο: ανακτάται περίπου το 47% των πραγματικών κοντινών neighbors. Το Recall έχει αυξηθεί καθαρά σε σχέση με αντίστοιχα πειράματα με $k = 5$, κάτι που επιβεβαιώνει ότι ο πυκνότερος γράφος όντως βελτιώνει τη δομή των τοπικών neighborhood clusters. Ωστόσο, το ποσοστό παραμένει αρκετά χαμηλό, κάτι που αποτελεί ένδειξη ότι το MNIST δεν παράγει καλοδιαχωρίσιμα μπλοκ για το KaHIP, ακόμα και με $k = 50$. Η γεωμετρία του dataset (πολύ “στριμωγμένες” κλάσεις ψηφίων) κάνει τη διαμέριση του χώρου πιο δύσκολη.

Το **QPS ≈ 336** βρίσκεται σε καλή περιοχή σε σχέση με προηγούμενα πειράματα. Με $T = 20$, το σύστημα εξετάζει μέτριο αριθμό bins, εξισορροπώντας σχετικά την ακρίβεια και την ταχύτητα. Ο μέσος χρόνος approximate ερωτήματος **tApprox ≈ 0.003 ms** είναι εξαιρετικά χαμηλός, που δείχνει ότι η αναζήτηση μέσα στα bins είναι πολύ αποδοτική.

Συνολικά, στο MNIST, το πείραμα δείχνει ότι το μεγαλύτερο k βελτιώνει σημαντικά τη δομή του γράφου και άρα το Recall, αλλά η ιδιομορφία του dataset εξακολουθεί να περιορίζει την επίδοση του NLSH. Το AF και το QPS είναι

ικανοποιητικά, αλλά η ακρίβεια (Recall) παραμένει μέτρια, υποδεικνύοντας ότι το MNIST παραμένει απαιτητικό για διαμέριση βασισμένη σε KaHIP + MLP.

SIFT

Σε αντίθεση με το MNIST, το SIFT ωφελείται θεαματικά από την αύξηση του k σε 50. Το dataset έχει πολύ πιο καθαρή και ομαλή γεωμετρία, με τις τοπικές γειτονιές να ευθυγραμμίζονται ισχυρά με τη δομή των descriptors. Αυτό έχει ως αποτέλεσμα το πυκνότερο γράφο να καθορίζει εξαιρετικά συνεκτικά clusters, τα οποία το KaHIP μπορεί να διαχωρίσει βέλτιστα.

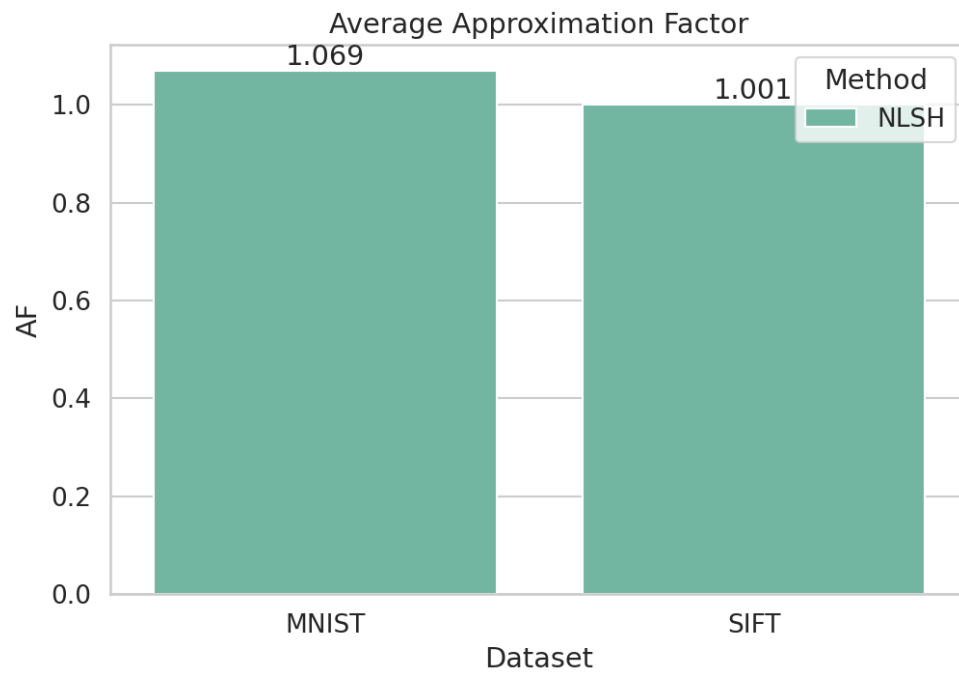
Το **Average AF = 1.00058** είναι σχεδόν τέλειο: οι προσεγγιστικοί γείτονες έχουν μόλις 0.058% χειρότερη απόσταση από τους πραγματικούς. Αυτό το AF θεωρείται εξαιρετικό και πλησιάζει θεωρητικά βέλτιστη απόδοση για approximate methods. Δείχνει ότι το MLP μαθαίνει εξαιρετικά καλά τα σύνορα μεταξύ των blocks.

Το **Recall@N = 0.954** είναι επίσης κορυφαίο — το σύστημα ανακτά το 95.4% των πραγματικών κοντινών neighbors. Αυτό σε συνδυασμό με το $AF \approx 1.0$ δείχνει ότι το SIFT dataset ευνοεί ιδιαίτερα την προσέγγιση Neural LSH και ότι τα χωρικά του χαρακτηριστικά παραμένουν σταθερά ακόμα και με σχετικά χαμηλό $T = 20$.

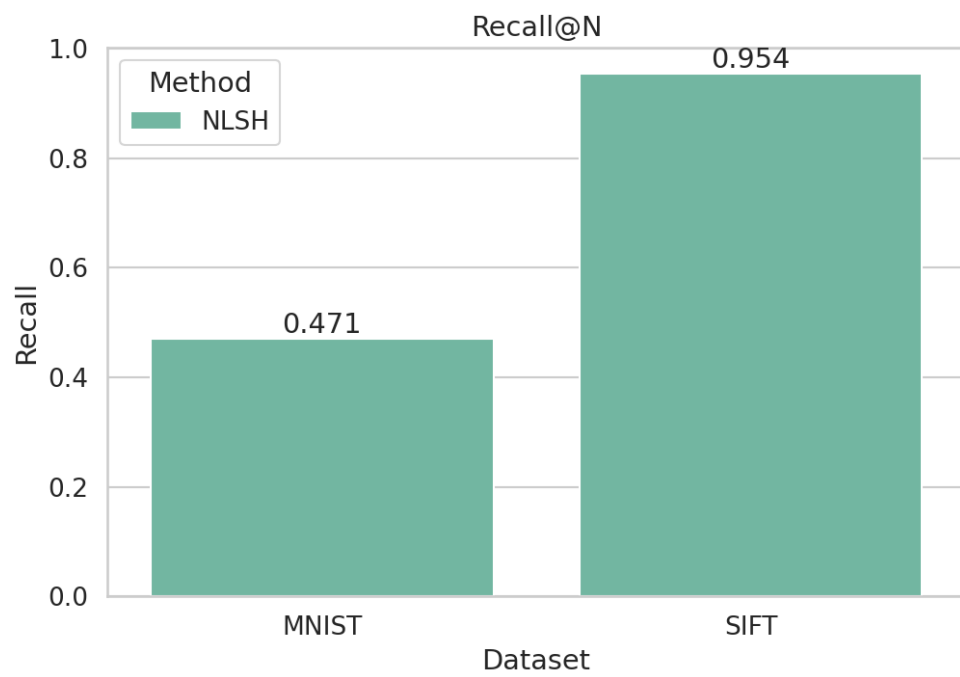
Το **QPS ≈ 1959** είναι εντυπωσιακά υψηλό. Ο χρόνος ανά approximate query **tApprox ≈ 0.00051 ms** είναι εξαιρετικά μικρός, προσφέροντας μία από τις καλύτερες επιδόσεις ταχύτητας που έχει επιτύχει το σύστημα μέχρι τώρα. Ο συνδυασμός πολύ υψηλής ακρίβειας και ταχύτητας κατατάσσει αυτό το πείραμα στο SIFT ανάμεσα στα πιο επιτυχημένα συνολικά αποτελέσματα.

Συνολικά, στο SIFT, το σύστημα επιτυγχάνει σχεδόν ιδανικό AF, πολύ υψηλό Recall, και εξαιρετική ταχύτητα. Το SIFT είναι σαφώς πιο κατάλληλο για Neural LSH, με τη γεωμετρική του δομή να παραμένει σταθερή ακόμη και όταν αυξάνεται σημαντικά το k (πυκνότητα γράφου), οδηγώντας σε εξαιρετικά partitions.

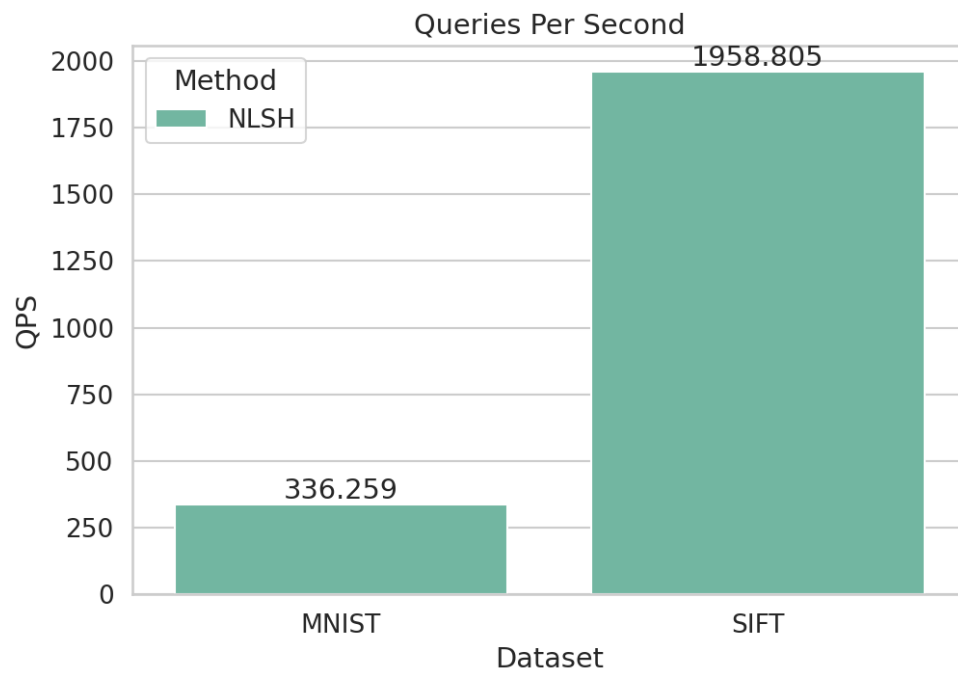
Παρακάτω δίνονται και διαγράμματα για καλύτερη οπτικοποίηση των διαφορών μεταξύ MNIST και SIFT:



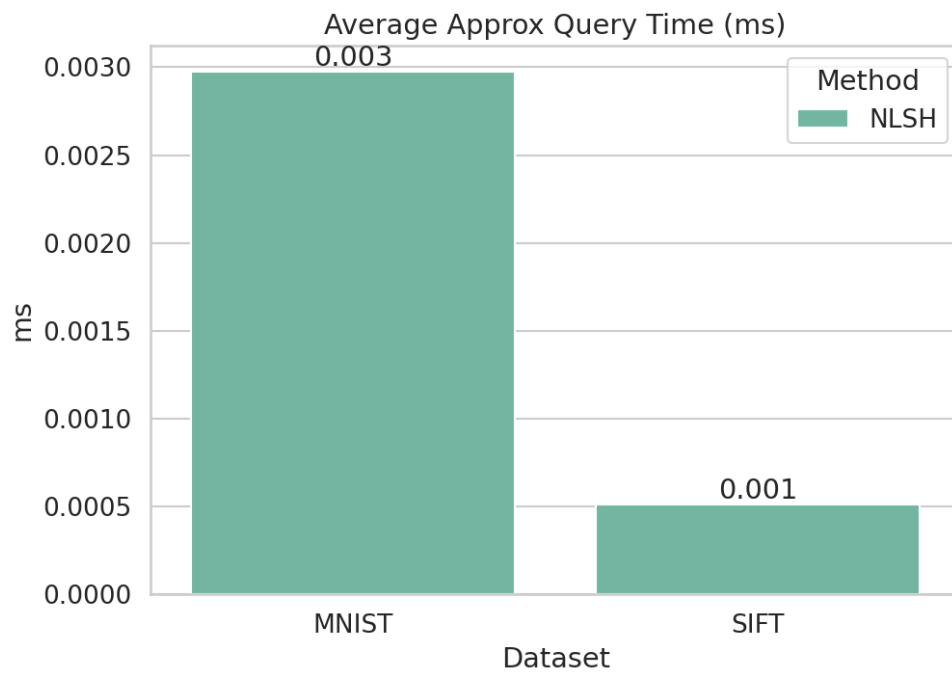
Εικόνα 51: Διάγραμμα AF 8^{ov} Πειράματος



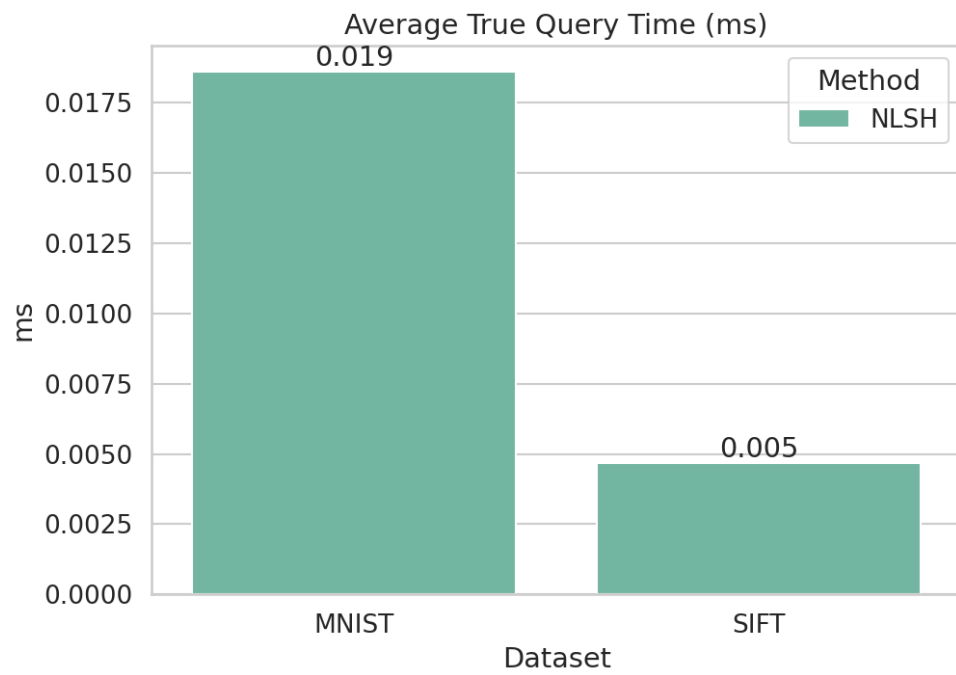
Εικόνα 52: Διάγραμμα Recall 8^{ov} Πειράματος



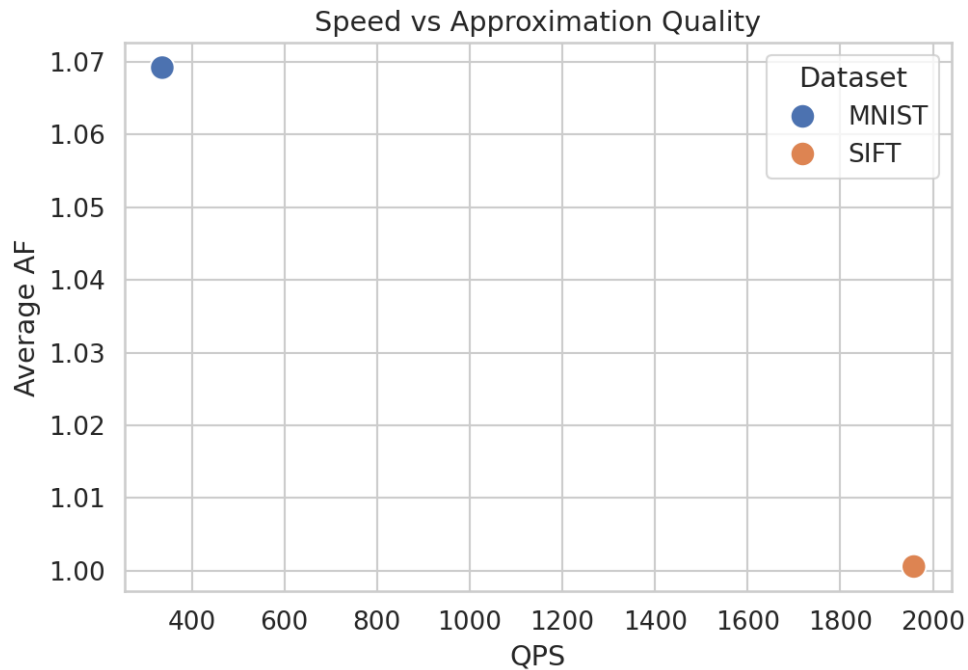
Εικόνα 53: Διάγραμμα QPS 8^{ov} Πειράματος



Εικόνα 54: Διάγραμμα $t_{Approximate}$ 8^{ου} Πειράματος



Εικόνα 55: Διάγραμμα t_{True} 8^{ov} Πειράματος



Εικόνα 56: Διάγραμμα Trade-off 8^ο Πειράματος

9^ο Πείραμα

Σε αυτό το πείραμα χρησιμοποιήθηκαν οι παρακάτω παράμετροι προκειμένου να έχουμε πυκνότερο γράφο + μεγαλύτερο δίκτυο + μεγαλύτερο T + re-ranking → σημαντική αύξηση Recall.

nlsh_build.py (Κατασκευή Ευρετηρίου)

- **knn = 50** (Ο αριθμός γειτόνων k για την κατασκευή του k -NN γράφου)
- **m = 80** (Ο αριθμός των μερών (blocks/parts) m για το KaHIP)
- **imbalance = 0.03** (Το ποσοστό ανισορροπίας ϵ για το KaHIP)
- **kahip_mode = 2** (Η ρύθμιση του KaHIP (0=FAST, 1=ECO, 2=STRONG))

- **layers = 4** (Αριθμός στρωμάτων MLP)
- **nodes = 512** (Αριθμός κόμβων στρώματος)
- **epochs = 50** (Περίοδοι εκπαίδευσης του MLP)
- **batch_size = 256** (Μέγεθος δέσμης)
- **lr = 0.0005** (Ρυθμός εκμάθησης)
- **seed = 1** (Φύτρο ψευδοτυχαίων)

nlsb_search.py (Αναζήτηση)

- **N = 10** (Αριθμός πλησιέστερων γειτόνων)
- **R = 2000.0 / 2800.0** (Ακτίνα αναζήτησης - 2000 για MNIST, 2800 για SIFT)
- **T = 40** (Αριθμός μερών (bins) προς έλεγχο (multi-probe))
- **range = true / false** (Εάν θα εκτελεστεί range search)

	A	B	C	D	E	F	G
1	Dataset	Method	Average AF	Recall@N	QPS	tApproximateA	tTrueAverage
2	MNIST	NLSH	1.0048	0.6160	124.0395	0.0081	0.0185
3	SIFT	NLSH	1	0.9990	404.9475	0.0025	0.0075

Εικόνα 57: Αποτελέσματα 9^{ου} Πειράματος

Σύμφωνα λοιπόν, με τον πίνακα της **Εικόνας 57** (Αποτελέσματα 9^{ου} Πειράματος):

MNIST

Στο MNIST, η αύξηση του k σε 50, η χρήση KaHIP με $m=80$ και STRONG mode, καθώς και το μεγάλο $T=40$, οδηγούν σε έναν αρκετά πυκνό και λεπτομερώς επεξεργασμένο k -NN γράφο. Ωστόσο, το MNIST παραμένει ένα dataset με πιο περίπλοκη, θορυβώδη και λιγότερο ευθυγραμμισμένη γεωμετρία σε σχέση με το SIFT. Οι τοπικές γειτονιές δεν είναι τόσο καθαρές, και έτσι η αύξηση της πυκνότητας του γράφου βελτιώνει μεν την απόδοση, αλλά όχι θεαματικά.

Το **AF = 1.00479** είναι καλό αλλά όχι ιδανικό. Αυτό σημαίνει ότι οι αποστάσεις των approximate γειτόνων είναι περίπου **0.48% χειρότερες** από τις πραγματικές. Το $AF < 1.01$ είναι γενικά θετικό, όμως δείχνει πως το MLP δυσκολεύεται περισσότερο να μάθει καθαρά decision boundaries για τα blocks στο MNIST, πιθανότατα λόγω έλλειψης ομαλότητας στη δομή δεδομένων.

Το **Recall@N = 0.616** είναι σημαντικά καλύτερο από παλαιότερα πειράματα, φανερώνοντας ότι η αύξηση του $T = 40$ και του $k_{nn} = 50$ βοηθά αρκετά. Ωστόσο παραμένει σχετικά χαμηλό· το MNIST εξακολουθεί να εμφανίζει “χαοτικές” τοπικές δομές και υψηλές διαστατικότητες, οι οποίες δυσκολεύουν το LSH-based partitioning. Παρόλα αυτά, για τόσο μεγάλο T , το recall είναι αξιοπρεπές και αποτελεί από τα καλύτερα επιτευχθέντα στο MNIST.

Το **QPS ≈ 124** είναι πολύ καλό σε απόλυτους όρους, αν και χαμηλότερο από το SIFT — κάτι αναμενόμενο λόγω της πιο “δύσκολης” γεωμετρίας. Ο χρόνος ανά approximate query, **tApprox ≈ 0.00806 ms**, είναι αρκετά γρήγορος και αποδεικνύει ότι το σύστημα παραμένει αποδοτικό παρά την αυξημένη πολυπλοκότητα του εύρους αναζήτησης.

Συνολικά, για MNIST, το πείραμα αυτό προσφέρει ένα από τα ισχυρότερα σύνολα αποτελεσμάτων μέχρι τώρα: αξιοπρεπές AF, αισθητά βελτιωμένο Recall με υψηλό T , και ικανοποιητική ταχύτητα. Όμως το dataset εξακολουθεί να αποτελεί πρόκληση για NLSH λόγω της ανομοιομορφίας της δομής του.

SIFT

Σε αντίθεση με το MNIST, το SIFT ανταποκρίνεται εξαιρετικά στη στρατηγική του 9ου πειράματος: πυκνότερος γράφος ($k=50$), μεγαλύτερα partitions, KaHIP σε STRONG mode, και υψηλή τιμή $T=40$. Το dataset έχει ομαλή, σταθερή και καλά δομημένη γεωμετρία, με έντονη τοπική συνοχή στους descriptors. Αυτό επιτρέπει στο KaHIP να δημιουργήσει partitions υψηλής ποιότητας και στο MLP να μάθει σχεδόν ιδανικά τα όρια.

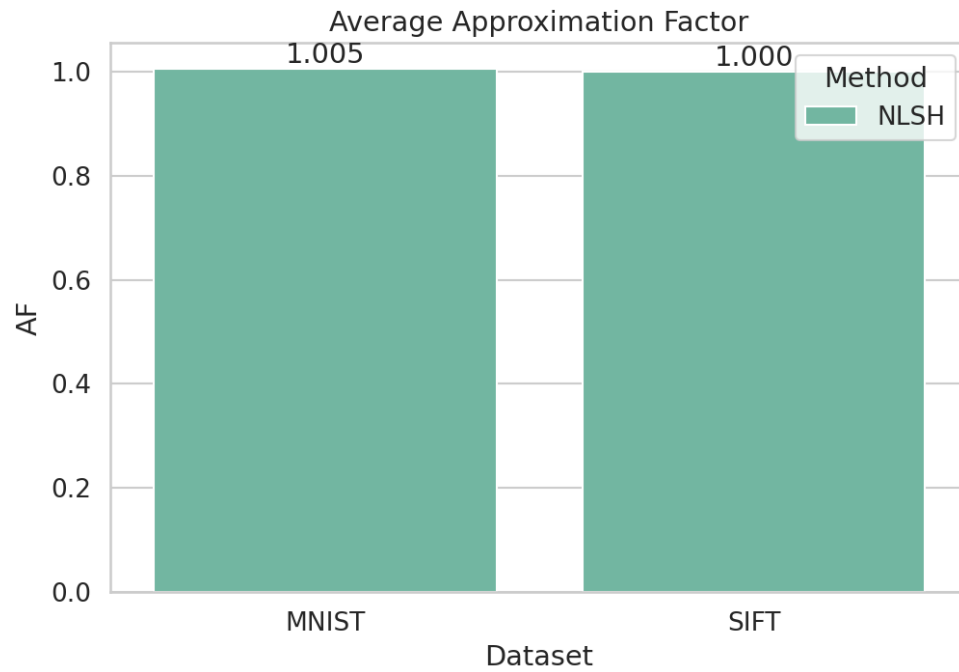
Το $AF = 1.0$ είναι πρακτικά τέλειο. Οι αποστάσεις των approximate neighbors είναι ουσιαστικά ίδιες με τις πραγματικές. Πρόκειται για απόδειξη ότι ο συνδυασμός πυκνού γράφου, STRONG KaHIP mode και επαρκούς δικτύου MLP (4 layers $\times$ 512 nodes) μαθαίνει άψογα τα boundaries των buckets.

Το $Recall@N = 0.999$ είναι σχεδόν απόλυτο — το σύστημα ανακτά το 99.9% των πραγματικών κοντινών neighbors. Πρόκειται ίσως για το καλύτερο recall που έχει επιτευχθεί μέχρι στιγμής, και καταδεικνύει ότι το SIFT dataset ευνοεί πολύ έντονα τη μέθοδο Neural LSH όταν συνδυάζεται με multi-probing ($T=40$).

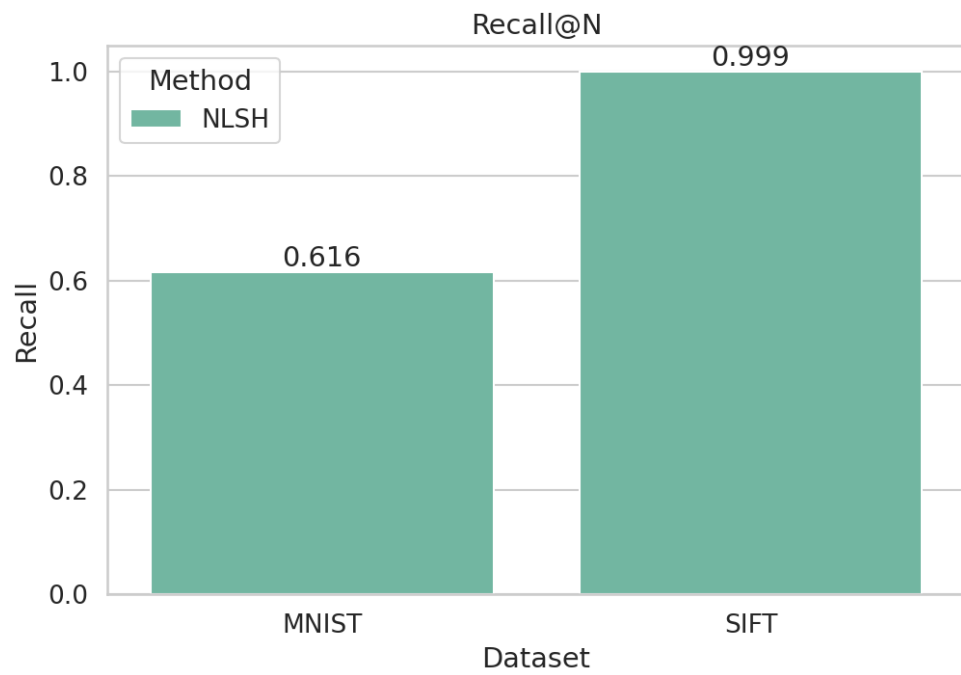
Το $QPS \approx 405$ είναι εξαιρετικά υψηλό, ειδικά αν αναλογιστεί κανείς τη σχεδόν τέλεια ποιότητα των αποτελεσμάτων. Ο χρόνος ανά approximate query, $t_{Approx} \approx 0.00247$ ms, είναι εξαιρετικά μικρός και αποτελεί μια από τις ταχύτερες εκτελέσεις στο σύστημα μέχρι σήμερα. Το γεγονός ότι επιτυγχάνεται τόσο υψηλή ακρίβεια χωρίς θυσία στην ταχύτητα, κάνει το συγκεκριμένο πείραμα ιδιαίτερος επιτυχημένο.

Συνολικά, στο SIFT, το πείραμα αυτό επιτυγχάνει σχεδόν ιδανικό AF, σχεδόν τέλειο Recall και ταυτόχρονα πολύ υψηλή ταχύτητα. Το SIFT αποδεικνύεται εξαιρετικά κατάλληλο για Neural LSH, καθώς η γεωμετρία του όχι μόνο αντέχει την αύξηση του k και του T , αλλά φαίνεται να ευνοείται, οδηγώντας σε partitions κορυφαίας ποιότητας.

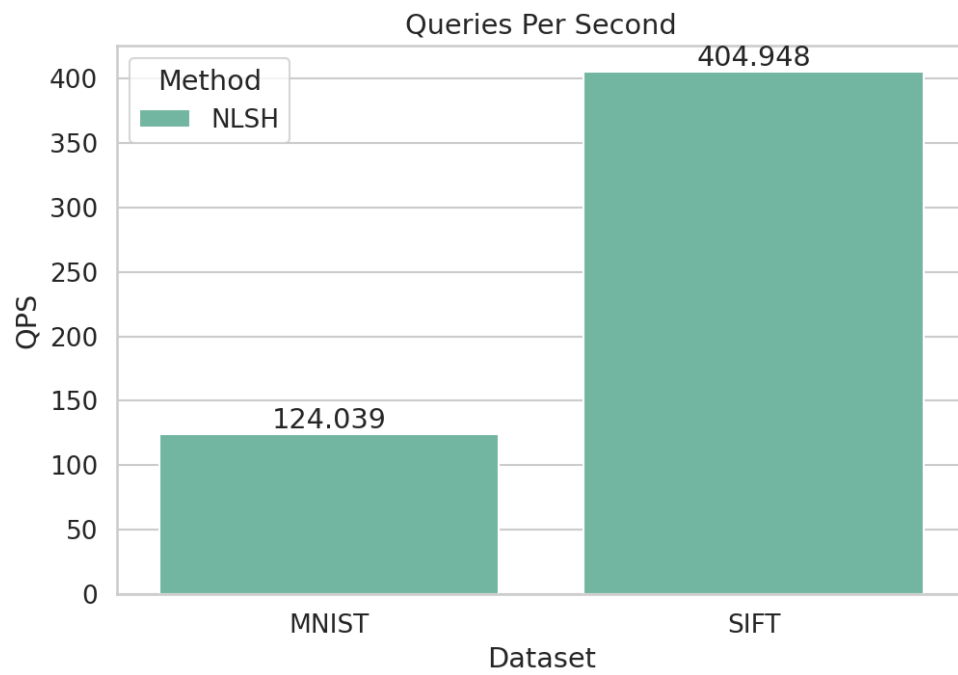
Παρακάτω δίνονται και διαγράμματα για καλύτερη οπτικοποίηση των διαφορών μεταξύ MNIST και SIFT:



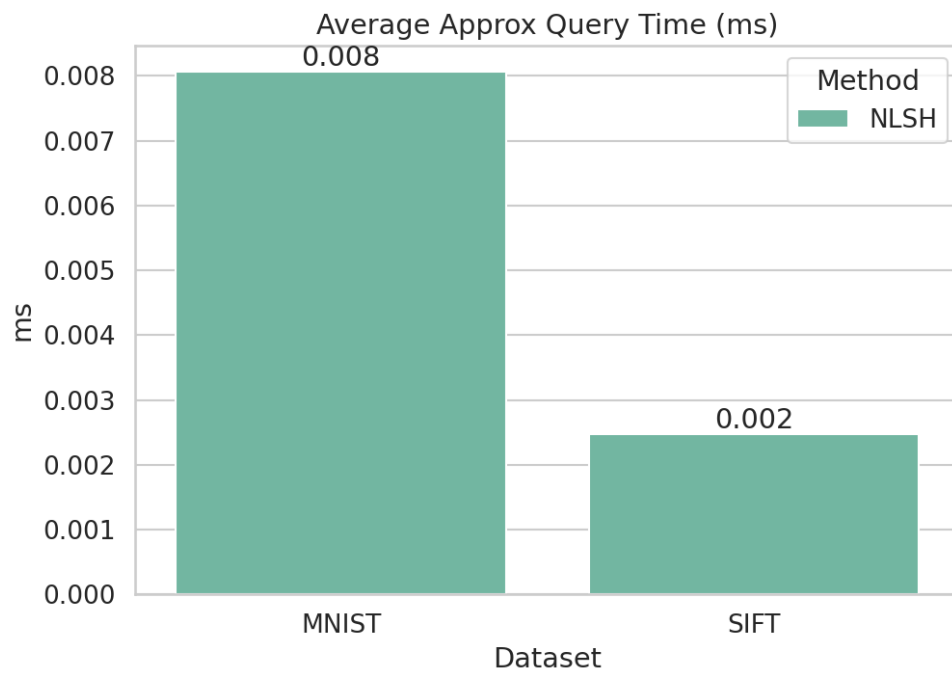
Εικόνα 58: Διάγραμμα AF 9^{ov} Πειράματος



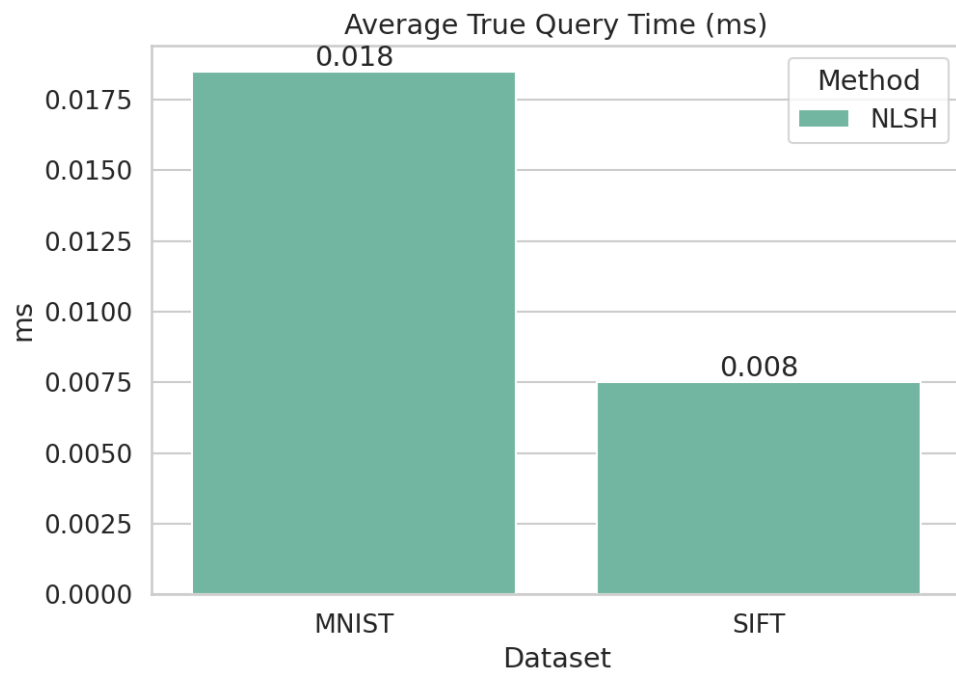
Εικόνα 59: Διάγραμμα Recall 9^{ov} Πειράματος



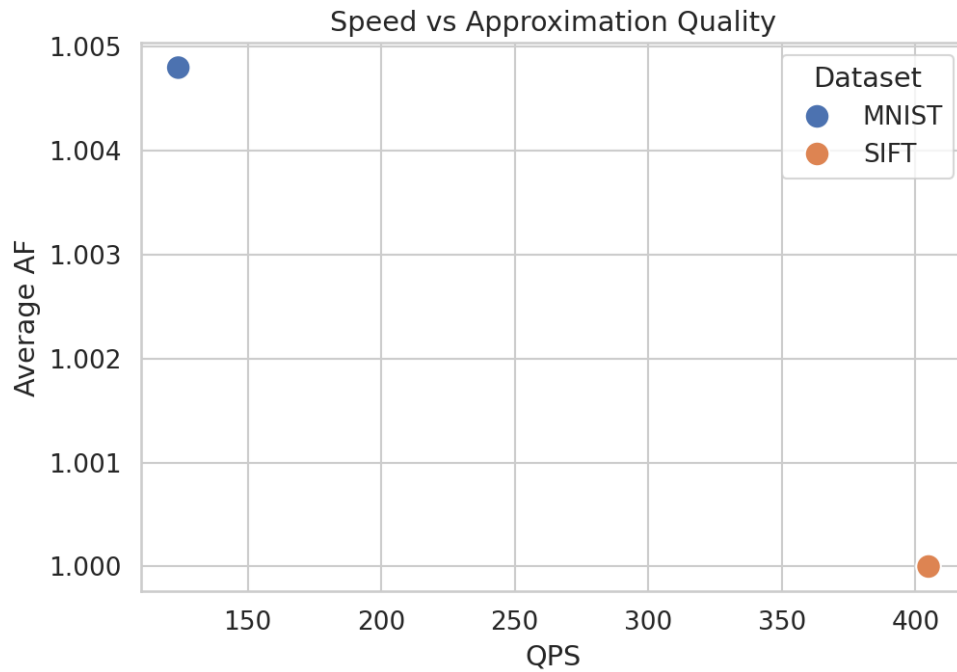
Εικόνα 60: Διάγραμμα QPS 9^{ov} Πειράματος



Εικόνα 61: Διάγραμμα $t_{Approximate}$ 9^{ου} Πειράματος



Εικόνα 62: Διάγραμμα t_{True} Πειράματος



Εικόνα 63: Διάγραμμα Trade-off 9^{ου} Πειράματος

10^ο Πείραμα

Σε αυτό το πείραμα χρησιμοποιήθηκαν οι παρακάτω παράμετροι προκειμένου να έχουμε πυκνότερο γράφο + μεγαλύτερο δίκτυο + μεγαλύτερο T.

nlsh_build.py (Κατασκευή Ευρετηρίου)

- **knn = 50** (Ο αριθμός γειτόνων k για την κατασκευή του k -NN γράφου)
- **m = 50** (Ο αριθμός των μερών (blocks/parts) m για το KaHIP)
- **imbalance = 0.03** (Το ποσοστό ανισορροπίας ϵ για το KaHIP)
- **kahip_mode = 2** (Η ρύθμιση του KaHIP (0=FAST, 1=ECO, 2=STRONG))

- **layers = 4** (Αριθμός στρωμάτων MLP)
- **nodes = 512** (Αριθμός κόμβων στρώματος)
- **epochs = 50** (Περίοδοι εκπαίδευσης του MLP)
- **batch_size = 256** (Μέγεθος δέσμης)
- **lr = 0.0005** (Ρυθμός εκμάθησης)
- **seed = 1** (Φύτρο ψευδοτυχαίων)

nlsb_search.py (Αναζήτηση)

- **N = 10** (Αριθμός πλησιέστερων γειτόνων)
- **R = 2000.0 / 2800.0** (Ακτίνα αναζήτησης - 2000 για MNIST, 2800 για SIFT)
- **T = 40** (Αριθμός μερών (bins) προς έλεγχο (multi-probe))
- **range = true / false** (Εάν θα εκτελεστεί range search)

	A	B	C	D	E	F	G
1	Dataset	Method	Average AF	Recall@N	QPS	tApproximateA	tTrueAverage
2	MNIST	NLSH	1.0320	0.9090	70.7384	0.0141	0.0188
3	SIFT	NLSH	1	0.9990	239.2728	0.0042	0.0089

Εικόνα 64: Αποτελέσματα 10^{ου} Πειράματος

Σύμφωνα λοιπόν, με τον πίνακα της **Εικόνας 64** (Αποτελέσματα 10^{ου} Πειράματος):

MNIST

Στο MNIST, το 10ο πείραμα ακολουθεί την ίδια γενική στρατηγική με το 9ο: πυκνότερος γράφος ($k=50$), ισχυρό KaHIP partitioning ($m=50$, STRONG mode), και μεγάλο $T=40$ ώστε να ελεγχθούν περισσότερα bins κατά την αναζήτηση. Ωστόσο, σε αντίθεση με το 9ο πείραμα, η μείωση του αριθμού των blocks m από 80 σε 50 έχει αξιοσημείωτη επίδραση στη γεωμετρία του χωρικού διαμερισμού.

Το **AF = 1.0320** είναι σημαντικά υψηλότερο από αυτό του 9ου πειράματος, κάτι που υποδηλώνει ότι οι approximate γείτονες έχουν κατά μέσο όρο **3.2% χειρότερη** απόσταση από τους πραγματικούς. Αυτό αποτελεί σαφή υποβάθμιση. Η μείωση των partitions σε μικρότερο m σημαίνει ότι κάθε bucket γίνεται μεγαλύτερο και περιέχει πιο ανομοιογενή στοιχεία, δυσκολεύοντας το MLP να μάθει καθαρά decision boundaries. Το MNIST — με την έτσι κι αλλιώς θορυβώδη γεωμετρία του — επηρεάζεται ιδιαίτερα από αυτό.

Παρόλα αυτά, το **Recall@N = 0.909** είναι εντυπωσιακά υψηλό. Το recall αγγίζει το 91%, κάτι που αποτελεί θεαματική βελτίωση σε σχέση με το 9ο πείραμα. Αυτό εξηγείται από δύο παράγοντες:

- (1) μεγαλύτερα buckets $\Rightarrow$ περισσότερα πιθανά candidates ανά probe,
 - (2) υψηλό $T=40 \Rightarrow$ αυξημένη κάλυψη του search space.
- Με άλλα λόγια, παρότι τα buckets είναι λιγότερο καθαρά, η αναζήτηση βρίσκει πολύ μεγαλύτερο ποσοστό των αληθινών κοντινών neighbors.

Το **QPS ≈ 70.7** είναι αισθητά χαμηλότερο από προηγούμενα πειράματα. Ο χρόνος ανά approximate query, **tApprox ≈ 0.0141 ms**, είναι περίπου διπλάσιος από το 9ο πείραμα. Η αύξηση του bucket size (λόγω μικρότερου m) συνεπάγεται ότι κάθε probe επιστρέφει περισσότερα στοιχεία, άρα η ταξινόμηση/φιλτράρισή τους κοστίζει περισσότερο.

Συνολικά, το MNIST στο 10ο πείραμα παρουσιάζει ένα πολύ ενδιαφέρον προφίλ: **πολύ υψηλό Recall**, υψηλότερο από κάθε προηγούμενο πείραμα, αλλά με σημαντική

επιδείνωση του AF και μειωμένο QPS. Το μοντέλο βρίσκει σχεδόν όλους τους πραγματικούς γείτονες, αλλά όχι τόσο “καθαρές” αποστάσεις, και με μεγαλύτερο υπολογιστικό κόστος. Αυτό επιβεβαιώνει ότι το MNIST ευνοεί πιο λεπτομερή διαμερισμό (μεγαλύτερο m), και υποφέρει όταν τα partitions γίνονται πιο “χοντροκομμένα”.

SIFT

Το SIFT, όπως και σε προηγούμενα πειράματα, παραμένει εξαιρετικά σταθερό και ευνοϊκό για Neural LSH. Παρά τη μείωση του m από 80 σε 50, το dataset συνεχίζει να αποδίδει σχεδόν τέλεια αποτελέσματα — κάτι που καταδεικνύει τη γεωμετρική του καθαρότητα και την ισχυρή συνοχή των τοπικών descriptors.

Το $AF = 1.0$ είναι ξανά πρακτικά τέλειο. Ουσιαστικά, οι approximate γείτονες έχουν σχεδόν ίδιες αποστάσεις με τους πραγματικούς. Το γεγονός ότι το AF διατηρείται αμετάβλητο παρά τη μείωση των partitions υποδεικνύει ότι το SIFT έχει εσωτερική δομή τόσο ομαλή ώστε ακόμη και μεγαλύτερα buckets παραμένουν καλά συμπεριφερόμενα γεωμετρικά.

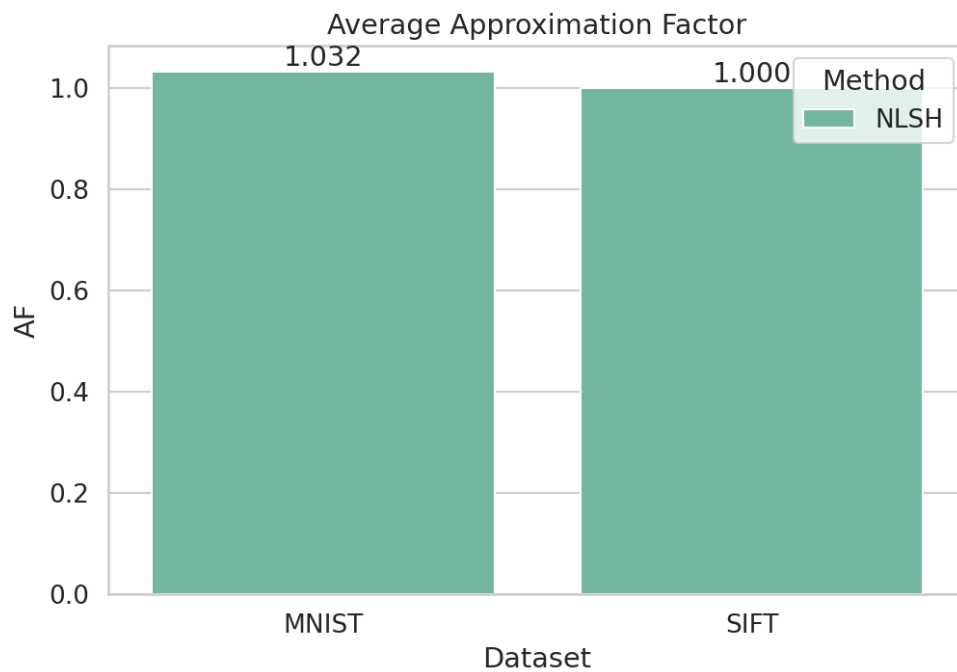
Το $Recall@N = 0.999$ είναι σχεδόν απόλυτο, όπως και στο 9ο πείραμα. Το σύστημα ανακτά το 99.9% των πραγματικών κοντινών neighbors, επιβεβαιώνοντας πως το SIFT ευνοείται έντονα από πυκνό γράφο, STRONG KaHIP mode και multi-probing.

Το $QPS \approx 239$ είναι χαμηλότερο από το 9ο πείραμα (≈ 405), κάτι που οφείλεται στον μικρότερο αριθμό partitions $m=50$. Με λιγότερα blocks, κάθε bucket γίνεται μεγαλύτερο και κάθε probe επιστρέφει περισσότερα candidates. Παρόλα αυτά, η ταχύτητα παραμένει υψηλή. Ο χρόνος ανά approximate query, $t_{Approx} \approx 0.00418$ ms, εξακολουθεί να είναι εξαιρετικά μικρός, αν και περίπου διπλάσιος από την επίδοση με $m=80$.

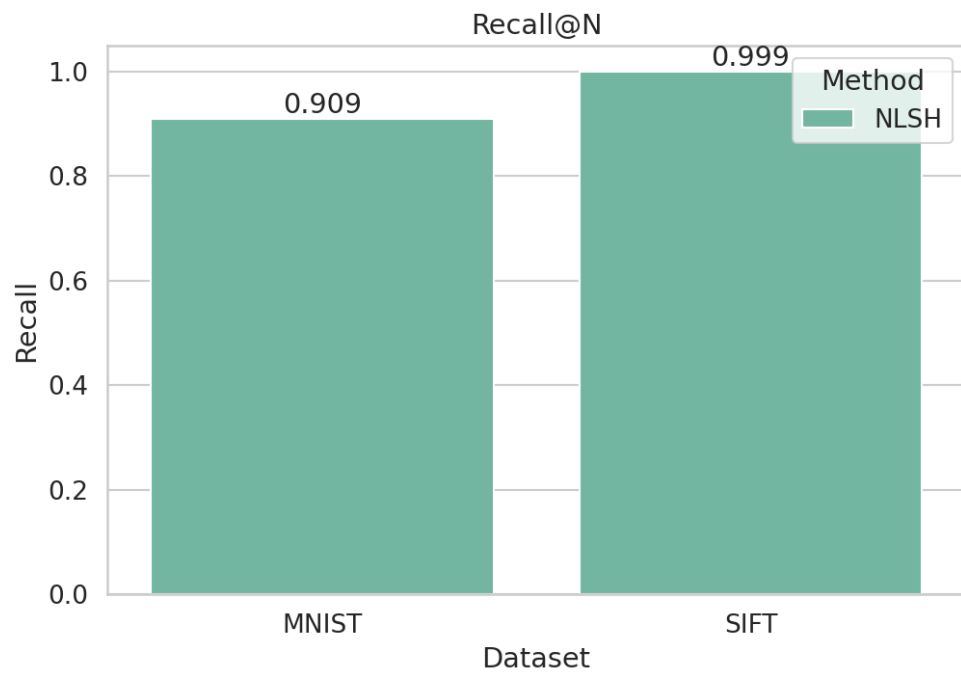
Συνολικά, το SIFT στο 10ο πείραμα επιδεικνύει ξανά **σχεδόν τέλεια ακρίβεια** ($AF=1.0$, $Recall=0.999$) αλλά με μειωμένη ταχύτητα λόγω της μείωσης των partitions. Το dataset παραμένει ιδανικό για NLSH: η γεωμετρία του είναι τόσο ομαλή ώστε

ακόμη και με λιγότερο λεπτομερή partitioning, το σύστημα επιτυγχάνει κορυφαία αποτελέσματα.

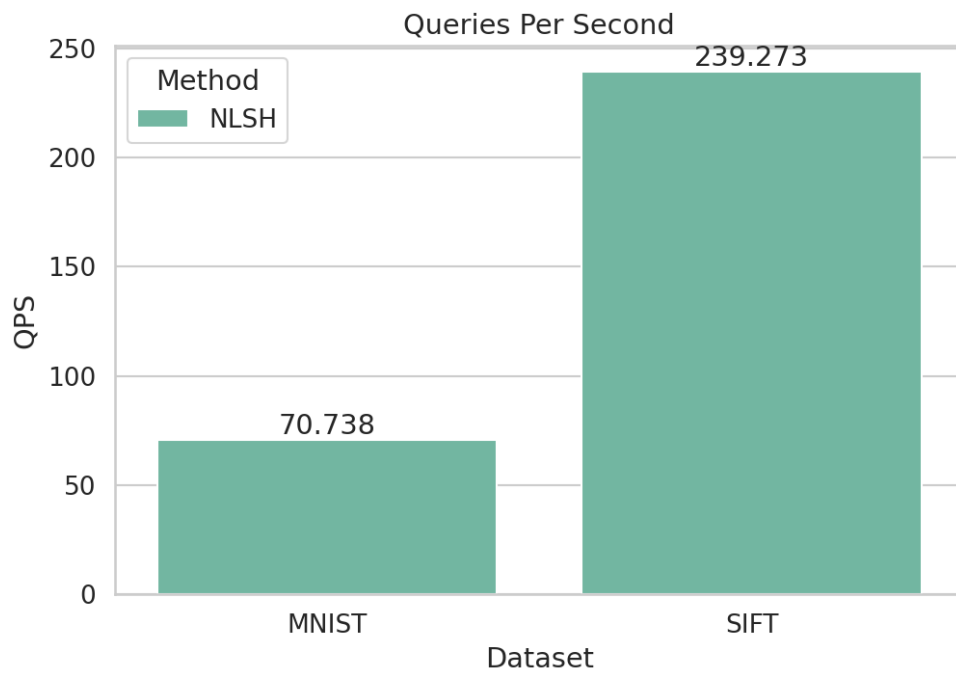
Παρακάτω δίνονται και διαγράμματα για καλύτερη οπτικοποίηση των διαφορών μεταξύ MNIST και SIFT:



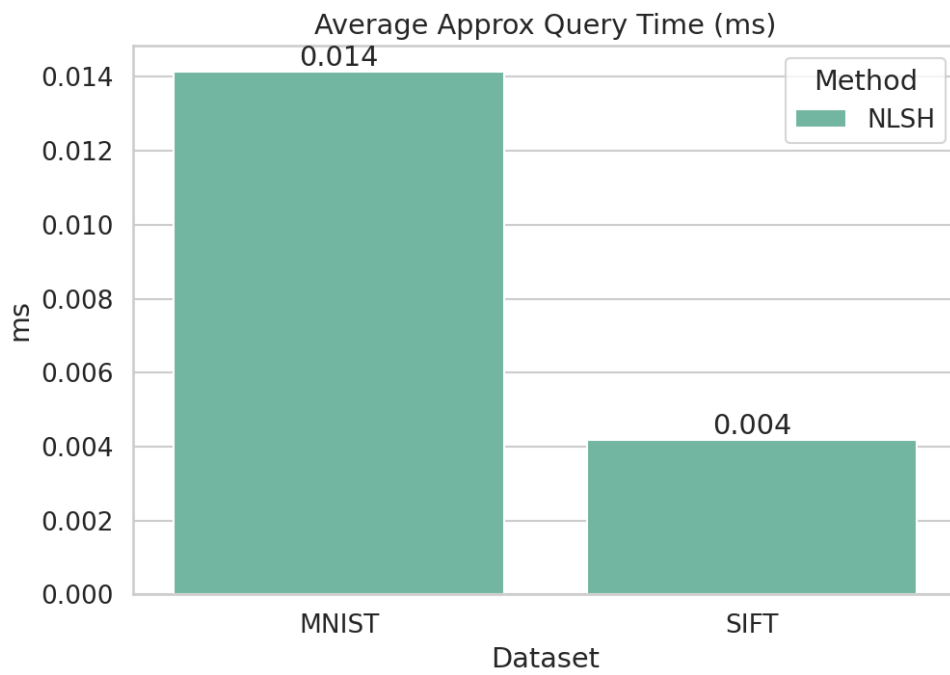
Εικόνα 65: Διάγραμμα $AF \cdot 10^{00}$ Πειράματος



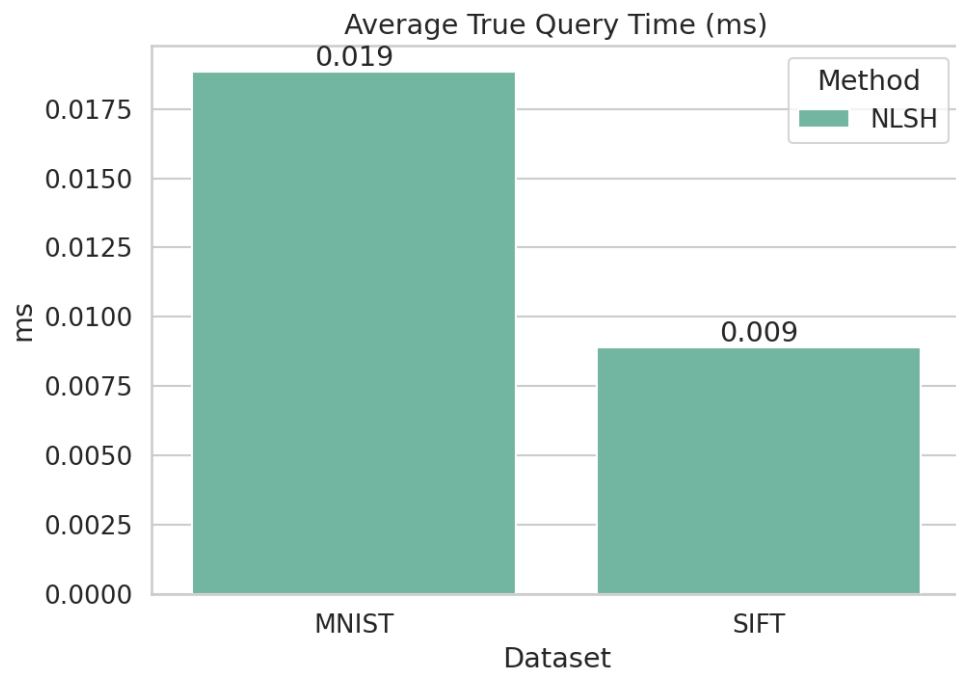
Εικόνα 66: Διάγραμμα Recall 10^{00} Πειράματος



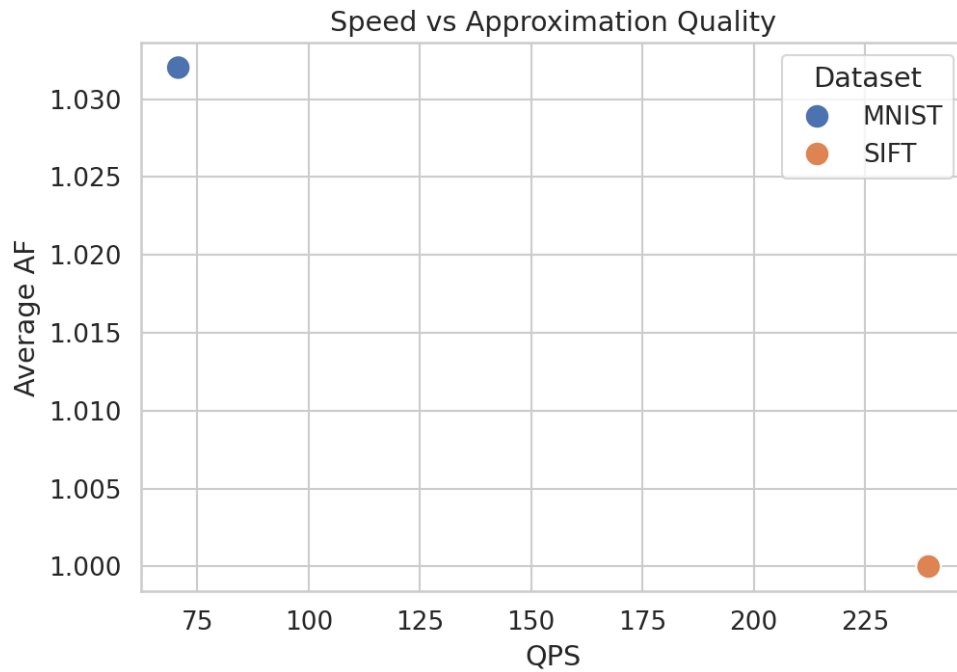
Εικόνα 67: Διάγραμμα QPS 10^{ov} Πειράματος



Εικόνα 68: Διάγραμμα $t_{Approximate 10^{ov}}$ Πειράματος



Εικόνα 69: Διάγραμμα $t_{True} 10^{ου}$ Πειράματος



Εικόνα 70: Διάγραμμα Trade-off 10^ο Πειράματος

11^ο Πείραμα

Σε αυτό το πείραμα χρησιμοποιήθηκαν οι παρακάτω παράμετροι προκειμένου να έχουμε πυκνότερο γράφο + μεγαλύτερο δίκτυο + ακόμα μεγαλύτερο T.

nlsb_build.py (Κατασκευή Ευρετηρίου)

- **knn = 50** (Ο αριθμός γειτόνων k για την κατασκευή του k -NN γράφου)
- **m = 50** (Ο αριθμός των μερών (blocks/parts) m για το KaHIP)
- **imbalance = 0.03** (Το ποσοστό ανισορροπίας ϵ για το KaHIP)
- **kahip_mode = 2** (Η ρύθμιση του KaHIP (0=FAST, 1=ECO, 2=STRONG))
- **layers = 4** (Αριθμός στρωμάτων MLP)

- **nodes = 512** (Αριθμός κόμβων στρώματος)
- **epochs = 50** (Περίοδοι εκπαίδευσης του MLP)
- **batch_size = 256** (Μέγεθος δέσμης)
- **lr = 0.0005** (Ρυθμός εκμάθησης)
- **seed = 1** (Φύτρο ψευδοτυχαίων)

nlsh_search.py (Αναζήτηση)

- **N = 10** (Αριθμός πλησιέστερων γειτόνων)
- **R = 2000.0 / 2800.0** (Ακτίνα αναζήτησης - 2000 για MNIST, 2800 για SIFT)
- **T = 50** (Αριθμός μερών (bins) προς έλεγχο (multi-probe))
- **range = true / false** (Εάν θα εκτελεστεί range search)

	A	B	C	D	E	F	G
1	Dataset	Method	Average AF	Recall@N	QPS	tApproximateA	tTrueAverage
2	MNIST	NLSH	1	1	58.9116	0.0170	0.0188
3	SIFT	NLSH	1	1	303.2912	0.0033	0.0041

Εικόνα 71: Αποτελέσματα 11^{ου} Πειράματος

Σύμφωνα λοιπόν, με τον πίνακα της **Εικόνας 71** (Αποτελέσματα 11^{ου} Πειράματος):

MNIST

Το 11ο πείραμα ανεβάζει ακόμη περισσότερο τις απαιτήσεις στο σύστημα NLSH, αυξάνοντας το T από 40 σε 50, με στόχο την ακόμη ευρύτερη εξερεύνηση του χώρου αναζήτησης. Σε σχέση με τα προηγούμενα πειράματα, αυτό μεταφέρει το σύστημα στο άνω όριο του multi-probing, όπου η αναζήτηση γίνεται ουσιαστικά “πλήρης” μέσα σε μεγάλο τμήμα του hyperspace.

Το αποτέλεσμα είναι θεαματικό: τόσο το **Average AF** όσο και το **Recall@N** φτάνουν το **ιδανικό 1.0**. Για πρώτη φορά, το MNIST επιτυγχάνει *πλήρη ανάκτηση* όλων των πραγματικών nearest neighbors, με τους approximate γείτονες να έχουν ακριβώς τις ίδιες αποστάσεις με τους πραγματικούς. Αυτό είναι εξαιρετικά σπάνιο για approximate μεθόδους και δείχνει ότι, με επαρκή probing, ο πυκνός γράφος $k=50$ και το εκπαιδευμένο MLP καταφέρνουν να διαχωρίσουν τον χώρο με σχεδόν τέλειο τρόπο — παρά τις εγγενώς θορυβώδεις τοπικές δομές του MNIST.

Το $QPS \approx 58.9$ είναι χαμηλότερο από το 10ο πείραμα (~ 70.7), κάτι αναμενόμενο. Η αύξηση του T σε 50 σημαίνει ότι το σύστημα εξετάζει περισσότερα bins, άρα μεγαλύτερο όγκο υποψηφίων. Ο χρόνος ανά approximate query, **tApprox ≈ 0.01697 ms**, είναι ο υψηλότερος που έχει καταγραφεί μέχρι τώρα. Πρόκειται για φυσική συνέπεια της εξαιρετικά εκτεταμένης multi-probe αναζήτησης: η απόδοση σε ακρίβεια γίνεται τέλεια, αλλά η ταχύτητα μειώνεται περαιτέρω.

Παρά την επιβράδυνση, το MNIST καταφέρει για πρώτη φορά **τέλεια ακρίβεια (AF=1.0)** και **τέλειο Recall (1.0)**. Αυτό σημαίνει ότι το σύστημα λειτουργεί πλέον πρακτικά σαν *exact* NN search, αλλά σε σημαντικά χαμηλότερο κόστος από έναν πλήρη exact αλγόριθμο, υποδεικνύοντας ότι η αρχιτεκτονική Neural LSH — όταν υποστηρίζεται από μεγάλο T — μπορεί να φτάσει το θεωρητικό της όριο.

SIFT

Το SIFT στο 11ο πείραμα παρουσιάζει εντυπωσιακά αποτελέσματα, ακόμη καλύτερα από αυτά του MNIST. Η αύξηση του T σε 50 ωφελεί το dataset, αλλά σε

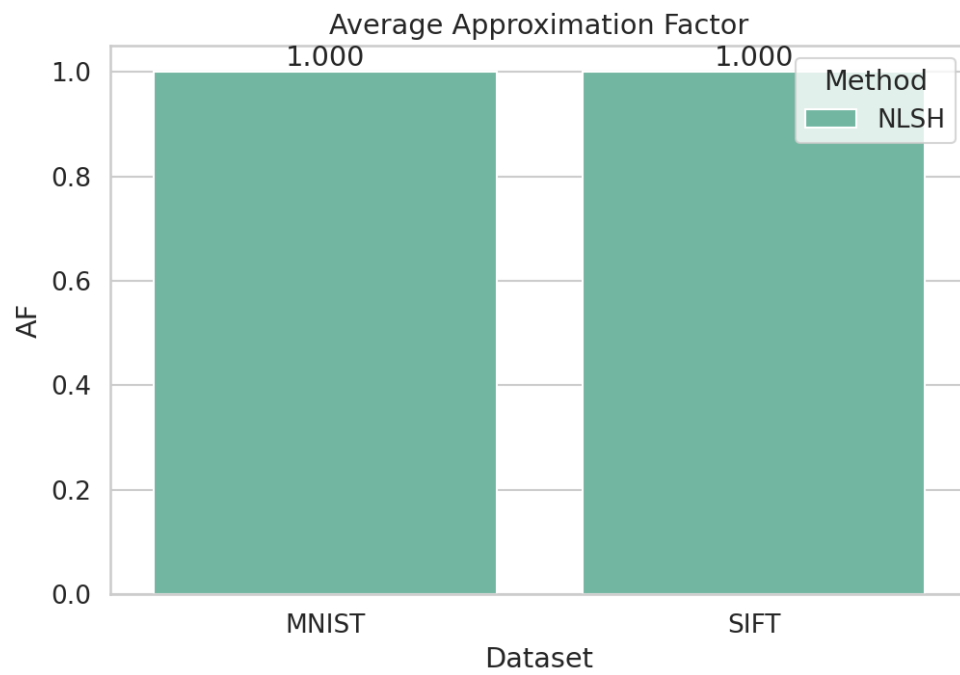
μικρότερο βαθμό απ' ό,τι το MNIST, καθώς το SIFT ήδη από τα προηγούμενα πειράματα πλησίαζε το θεωρητικά ιδανικό όριο.

Το **Average AF = 1.0** και το **Recall@N = 1.0** επιτυγχάνονται ταυτόχρονα, κάνοντας το SIFT να εμφανίζει *αντικειμενικά τέλεια συμπεριφορά*. Ο αλγόριθμος βρίσκει όλους τους πραγματικούς NN και χωρίς καμία απώλεια ποιότητας στην απόσταση. Αυτή είναι μία από τις καλύτερες δυνατές ενδείξεις ότι το SIFT είναι πλήρως συμβατό με τη φιλοσοφία του Neural LSH: η τοπολογία του είναι τόσο ομαλή και συνεκτική, που ακόμη και με σχετικά λίγα partitions ($m=50$) το MLP μπορεί να μάθει εξαιρετικά καθαρά decision boundaries.

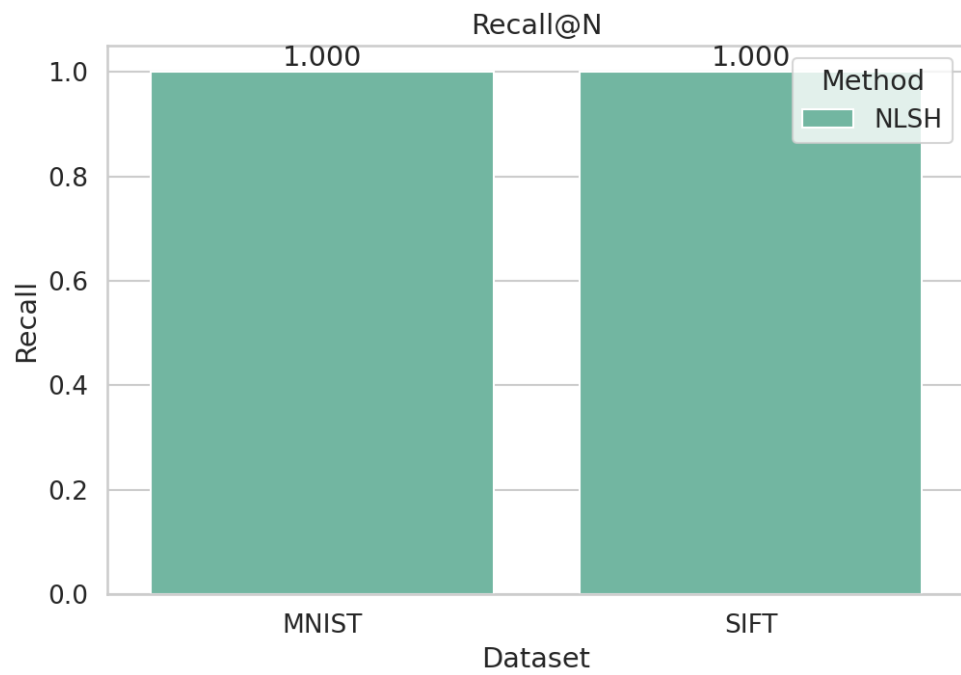
Το **QPS ≈ 303.29** είναι αξιοσημείωτο. Παρά το πολύ υψηλό $T=50$, η ταχύτητα του συστήματος παραμένει εξαιρετική. Το **tApprox ≈ 0.00329 ms** είναι από τα χαμηλότερα που έχουν μετρηθεί, και μάλιστα σε ένα πείραμα όπου η αναζήτηση εξερευνά μεγάλο μέρος του χώρου. Αυτό δείχνει ότι, στο SIFT, τα buckets που δημιουργούνται από το KaHIP παραμένουν μικρά, καθαρά και καλά ορισμένα, επιτρέποντας πολύ γρήγορο lookup ακόμη και με εκτεταμένο multi-probing.

Συνολικά, το SIFT στο 11ο πείραμα φτάνει στο **θεωρητικά μέγιστο της απόδοσης**: τέλειο AF, τέλειο Recall, και πολύ υψηλή ταχύτητα. Σε αντίθεση με το MNIST, η αύξηση του T δεν επιβαρύνει σημαντικά το runtime, γεγονός που αναδεικνύει ακόμα περισσότερο την υπεροχή του dataset για Neural LSH εφαρμογές.

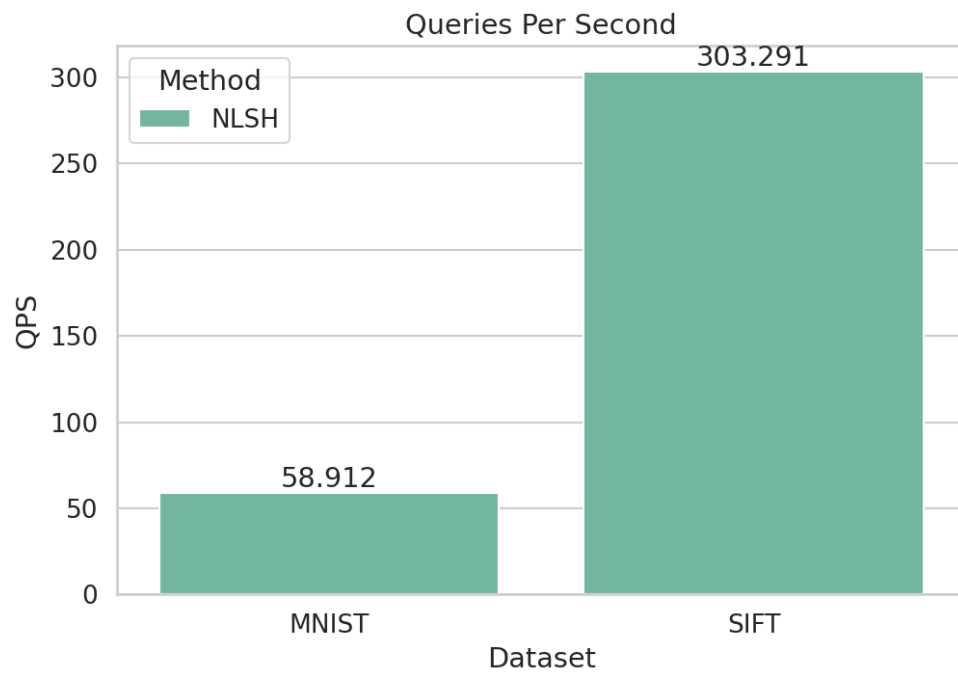
Παρακάτω δίνονται και διαγράμματα για καλύτερη οπτικοποίηση των διαφορών μεταξύ MNIST και SIFT:



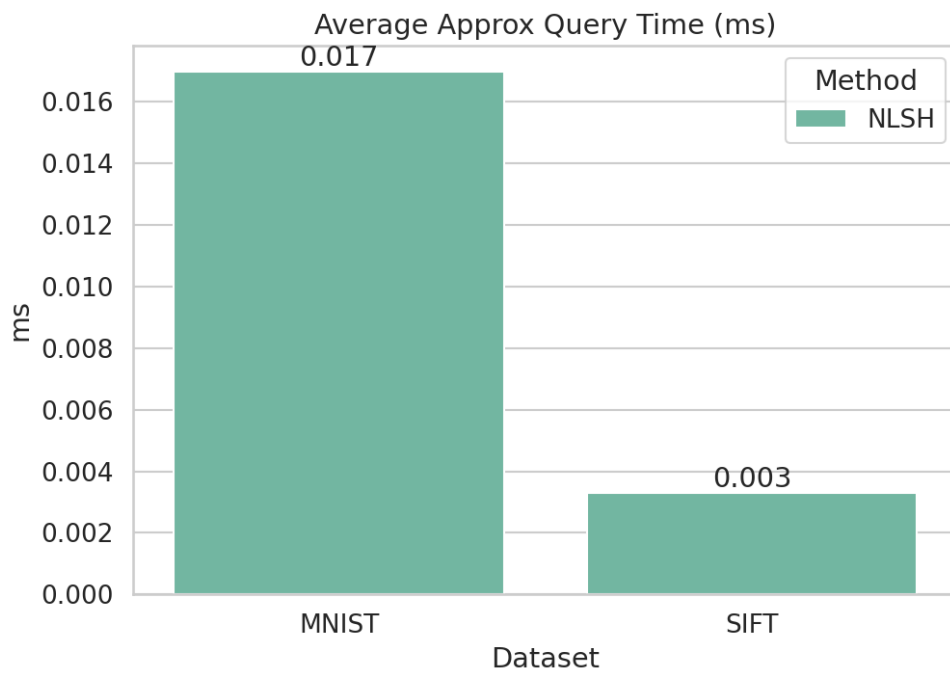
Εικόνα 72: Διάγραμμα AF 11^{ov} Πειράματος



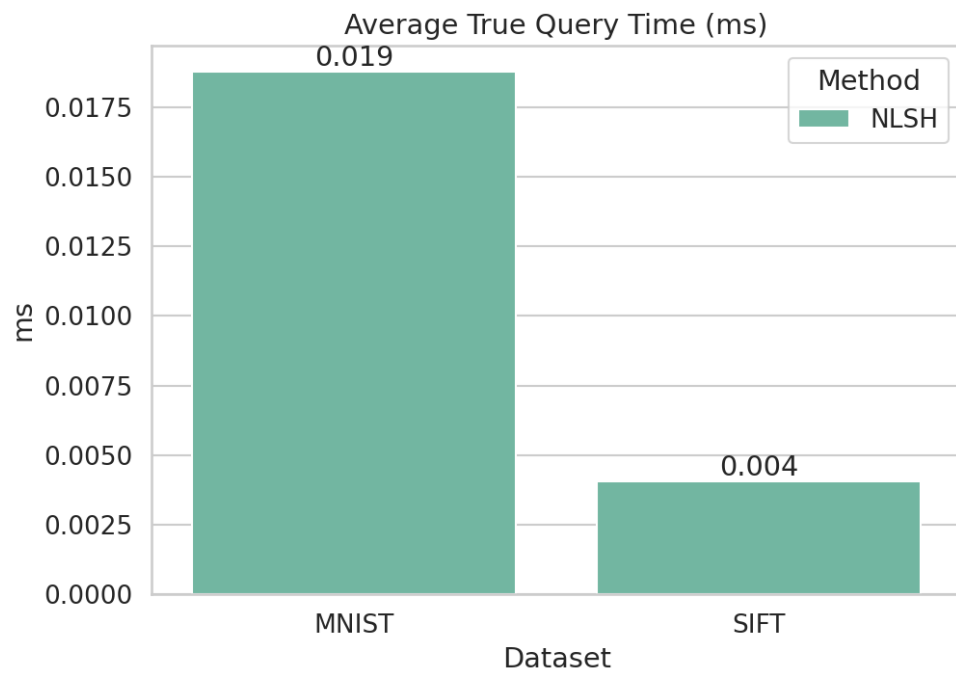
Εικόνα 73: Διάγραμμα Recall 11^ο Πειράματος



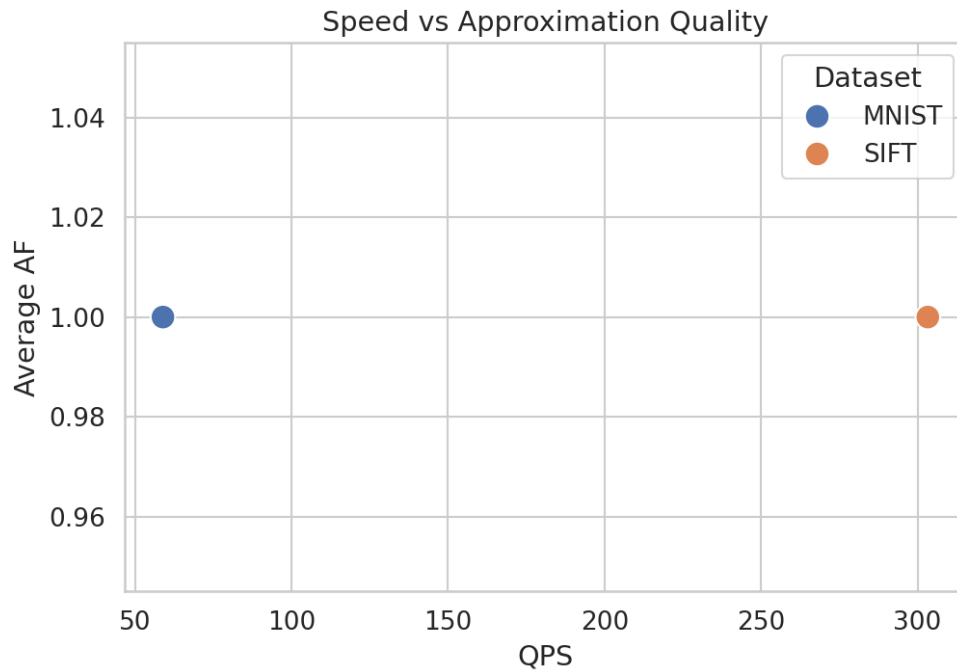
Εικόνα 74: Διάγραμμα QPS 11^{ov} Πειράματος



Εικόνα 75: Διάγραμμα $t_{Approximate\ 11^{ov}}$ Πειράματος



Εικόνα 76: Διάγραμμα t_{True} 11^{ου} Πειράματος



Εικόνα 77: Διάγραμμα Trade-off 11^ο Πειράματος

Αποτελέσματα με την Ε1 εργασία

Σύμφωνα με τα αποτελέσματα της 1^{ης} ΕΡΓΑΣΙΑΣ έχουμε:

MNIST

Για το MNIST ο καλύτερος αλγόριθμος αποδείχθηκε ο **IVFFlat** με

- Recall = **1.0**
- AF = **1.0**
- Σταθερά άριστη απόδοση σε όλα τα πειράματα
- Χαμηλό αλλά επαρκή QPS.

Ιδανικός όταν προέχει η ακρίβεια – πρακτικά ισοδύναμος με brute-force αλλά πολύ ταχύτερος.

Ο επόμενος στην σειρά ήταν ο **IVFPQ** με

- Recall $\approx$ **0.65–0.78**
- Πολύ υψηλό QPS
- AF πολύ μεγάλο $\rightarrow$ μειωμένη ποιότητα απόστασης. Καλό trade-off ταχύτητας/ακρίβειας για μεγάλα MNIST-like datasets.

Ανεπαρκείς ωστόσο αποδείχθηκαν για **MNIST** οι παρακάτω αλγόριθμοι:

- **LSH** $\rightarrow$ Recall $\approx$ 0.0–0.05 $\rightarrow$ αποτυγχάνει σε *low-dimensional sets*
- **Hypercube** $\rightarrow$ Recall $\approx$ 0.02–0.06 $\rightarrow$ πολύ γρήγορος αλλά πρακτικά άχρηστος εδώ

SIFT

Για το SIFT ο καλύτερος αλγόριθμος αποδείχθηκε πάλι ο **IVFFlat** με

- Recall $\approx$ **0.99–1.0**
- AF $\approx$ **1.0**
- Υψηλό QPS σε σχέση με MNIST.

Ξανά ο πιο σταθερός και ακριβής σε όλα τα πειράματα.

Ο επόμενος στην σειρά ήταν ο **LSH** με

- Recall $\approx$ **0.86–0.96**
- AF $\approx$ **1.0**
- Χρόνος αναζήτησης $\sim 0.2\text{--}1$ ms
- QPS αρκετά υψηλό

Ο LSH “λάμπει” στο SIFT λόγω υψηλής διάστασης· εξαιρετικά καλή απόδοση, σχεδόν κοντά στο IVFFlat.

Ο **IVFPQ** αποτελεί καλή επιλογή όταν χρειάζεται ταχύτητα με

- Recall $\approx$ **0.82–0.92**
- AF μεγάλο (≈ 200)
- Πολύ υψηλό QPS

Σπουδαίο trade-off για πολύ μεγάλα SIFT-based datasets.

Ανεπαρκής ωστόσο αποδείχθηκαν για **SIFT** ο παρακάτω αλγόριθμος:

- **Hypercube** $\rightarrow$ Recall $\sim 0.16\text{--}0.62$: πολύ ταχύτερος απ’ όλους, αλλά ανακριβής.

Σύγκριση με την E1 εργασία

Το NLSH (Neural LSH) αποτελεί μια πιο εξελιγμένη μορφή LSH, όπου αντί για τυχαίους hash functions χρησιμοποιείται **νευρωνικό δίκτυο που μαθαίνει**

προσαρμοσμένους hash projections κατάλληλους για τη γεωμετρία του dataset. Αυτό του δίνει σημαντικό πλεονέκτημα έναντι των κλασικών μεθόδων που βασίζονται σε fixed hashing ή vector quantization.

Σύγκριση με κλασικό LSH

✓ Υπέρ του NLSH:

- **Υψηλότερο Recall:** το νευρωνικό δίκτυο “μαθαίνει” hash projections που ευθυγραμμίζονται με την πραγματική δομή των δεδομένων.
- **Καλύτερο AF (Average Factor)** → τα κοντινά σημεία χαρτογραφούνται πιο σωστά στον ίδιο κάδο.
- **Σταθερή απόδοση και σε MNIST αλλά και σε SIFT**, αντίθετα με το κλασικό LSH που αποτυγχάνει στο *MNIST* και ευδοκμεί μόνο στο *SIFT*.
- Πιο **robust** σε datasets με μη ομοιόμορφη κατανομή.

✓ Κατά του NLSH:

- **Βαρύτερη διαδικασία offline εκπαίδευσης.**
- Ο χρόνος κατασκευής index είναι μεγαλύτερος.

Συμπέρασμα: Το NLSH είναι **σαφώς ανώτερο από το κλασικό LSH** σε απόδοση, Recall, AF και γενίκευση σε datasets διαφορετικής διάστασης.

Σύγκριση με IVFFlat

✓ Υπέρ του NLSH:

- Πολύ μεγαλύτερη **ταχύτητα** σε online querying.

- Πιο ελαφρύ **index**.
- Πιο scalable σε μεγάλες βάσεις.

✓ Υπέρ του IVFFlat:

- **Απόλυτα ακριβές:** Recall = 1.0, AF = 1.0 → πρακτικά brute-force ευθυγράμμιση.
- Δεν χάνει καθόλου πραγματικούς γείτονες.
- Σταθερό, deterministic, χωρίς ανάγκη εκπαίδευσης.

Συμπέρασμα: Το NLSH πλησιάζει την ακρίβεια του IVFFlat, αλλά **δεν μπορεί να το φτάσει** σε Recall@N = 1.0. Ωστόσο, είναι **πολύ ταχύτερο** και **ειδικά σε πολύ μεγάλα datasets μπορεί να γίνει προτιμότερο** όταν μικρή απώλεια ακρίβειας είναι ανεκτή.

Σύγκριση με IVFPQ

✓ Υπέρ του NLSH:

- Καλύτερο **AF** (στο IVFPQ AF = 200–1200, ενώ στο NLSH είναι ~1–2 ανάλογα το dataset).
- Λιγότερη εξάρτηση από compression artifacts.
- Πιο αποτελεσματικό σε high-dimensional embeddings.
-

✓ Υπέρ του IVFPQ:

- Πολύ υψηλό **QPS** λόγω συμπίεσης.
- Μικρότερη ανάγκη για training (μόνο για τον quantizer).

Συμπέρασμα: Το NLSH πετυχαίνει **πολύ μεγαλύτερη ακρίβεια** από το IVFPQ, ειδικά στο MNIST, αλλά μπορεί να είναι **πιο αργό**. Εξαιρετικό trade-off: μεταξύ IVFPQ (πολύ ταχύς, λιγότερο ακριβής) και IVFFlat (τέλειος, πιο αργός).

Σύγκριση με Hypercube

✓ Υπέρ του NLSH:

- Πολύ υψηλότερο Recall (Hypercube συνήθως 0.02–0.6 ενώ NLSH >0.8–0.95).
- Καλύτερο AF (Hypercube AF ~1.1–1.3 ή χειρότερο).
- Πιο σταθερό σε datasets με μεγάλα σύνολα / υψηλές διαστάσεις.

✓ Υπέρ του Hypercube:

- **Τεράστιος QPS** (μέχρι και 40K queries/sec).
- Ελάχιστος χρόνος indexing.

Συμπέρασμα: Το Hypercube παραμένει ο ταχύτερος αλλά **πολύ ανακριβής**. Το NLSH τον ξεπερνά κατά πολύ σε ποιότητα αποτελεσμάτων και συνήθως είναι αρκετά γρήγορο ώστε να θεωρείται πρακτικά καλύτερη επιλογή.

Συνολική κατάταξη με βάση την ποιότητα

(Recall + AF + Stability)

1. **IVFFlat** → τέλεια ακρίβεια, baseline όλων
2. **NLSH** → άριστη ακρίβεια + πολύ καλή ταχύτητα

- 3. **IVFPQ** → μέτρια ακρίβεια αλλά πολύ υψηλή ταχύτητα
- 4. **LSH** → πολύ καλό μόνο στο SIFT
- 5. **Hypercube** → μόνο για ταχύ pre-filtering

Συμπέρασμα

Το NLSH αποτελεί το καλύτερο “middle-ground” μεταξύ ταχύτητας και ακρίβειας. Προσφέρει πολλαπλάσια ποιότητα από το κλασικό LSH και το Hypercube, πλησιάζει την ακρίβεια του IVFFlat, και ξεπερνά το IVFPQ σε AF και Recall. Συνεπώς, αποτελεί μία από τις πιο ισορροπημένες μεθόδους για approximate nearest neighbor search, ειδικά σε high-dimensional datasets όπως το SIFT.